



ECOLE  
**POLYTECHNIQUE**  
DE BRUXELLES

# 06.Textures, Normal Mapping and Depth Compositing

INFO-H502

**Daniele Bonatto**

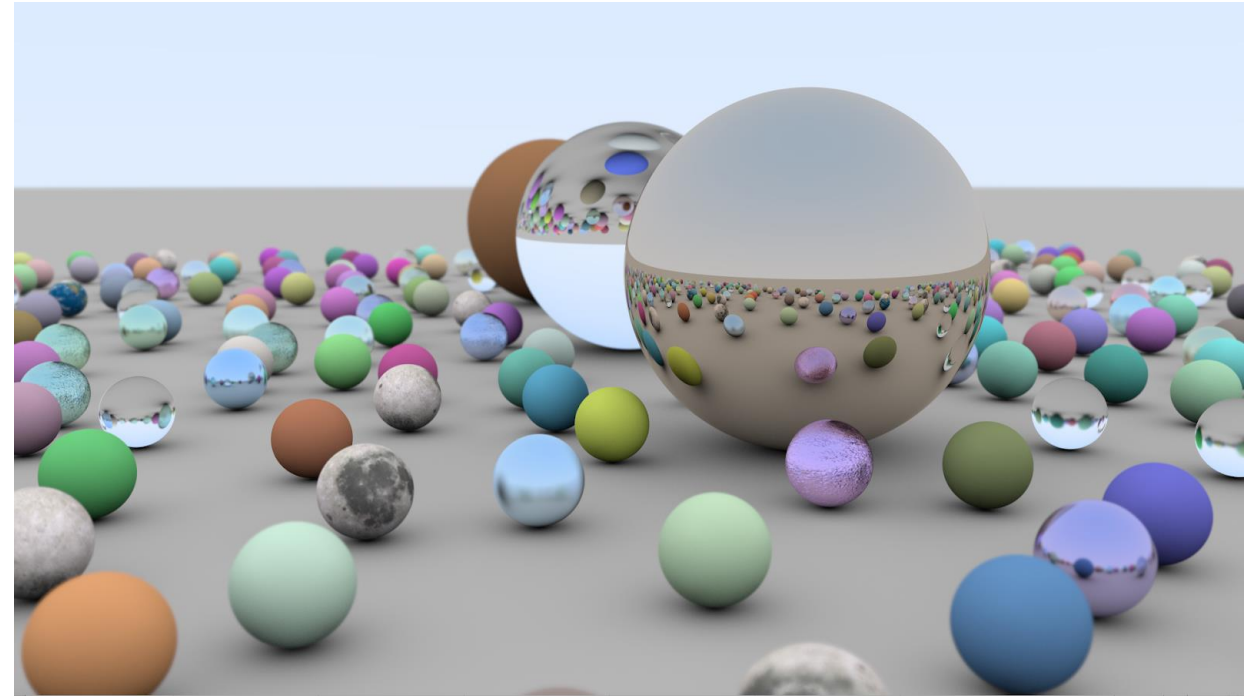
LISA-VR

2024

# Previously



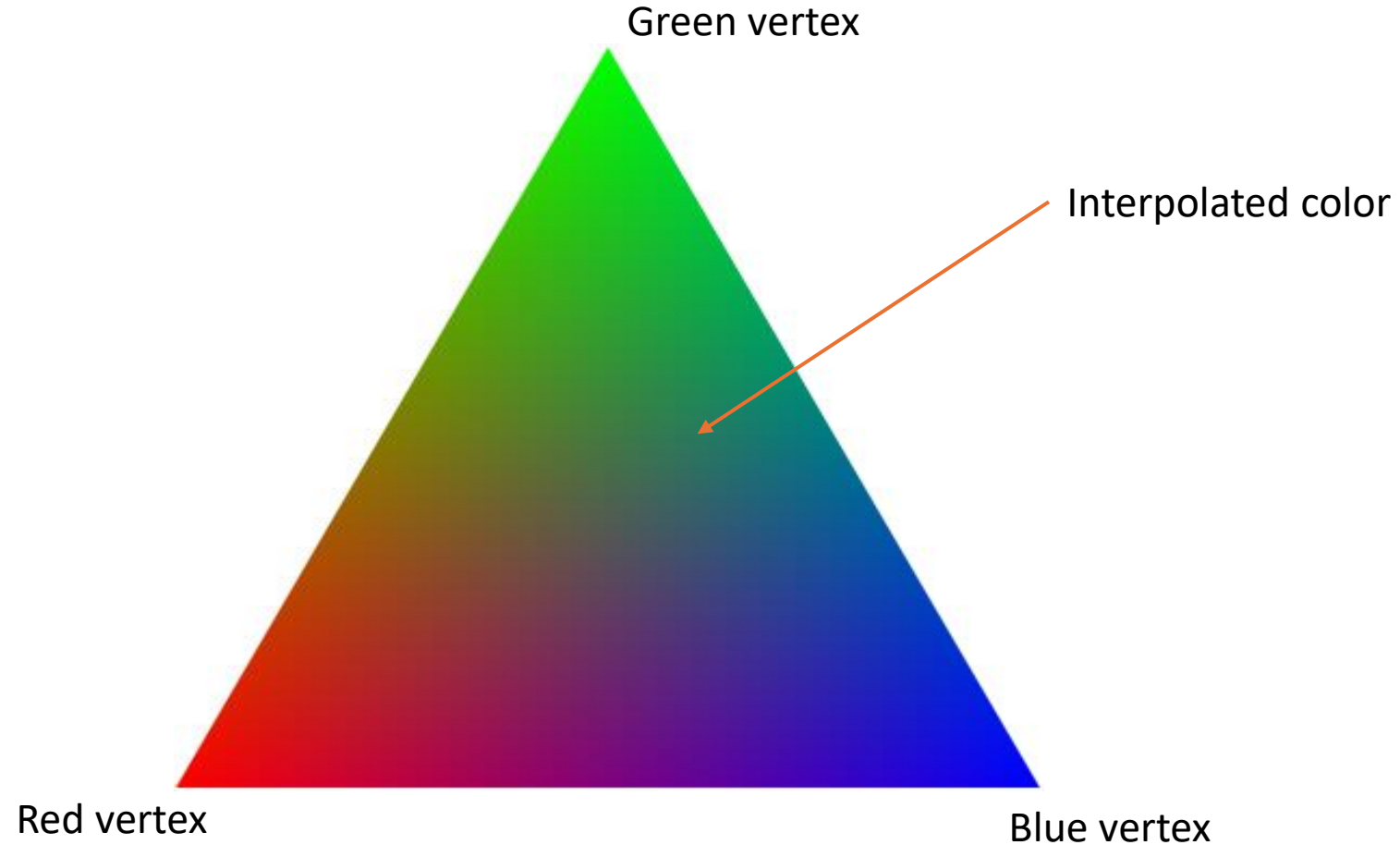
Lights



Raytracing

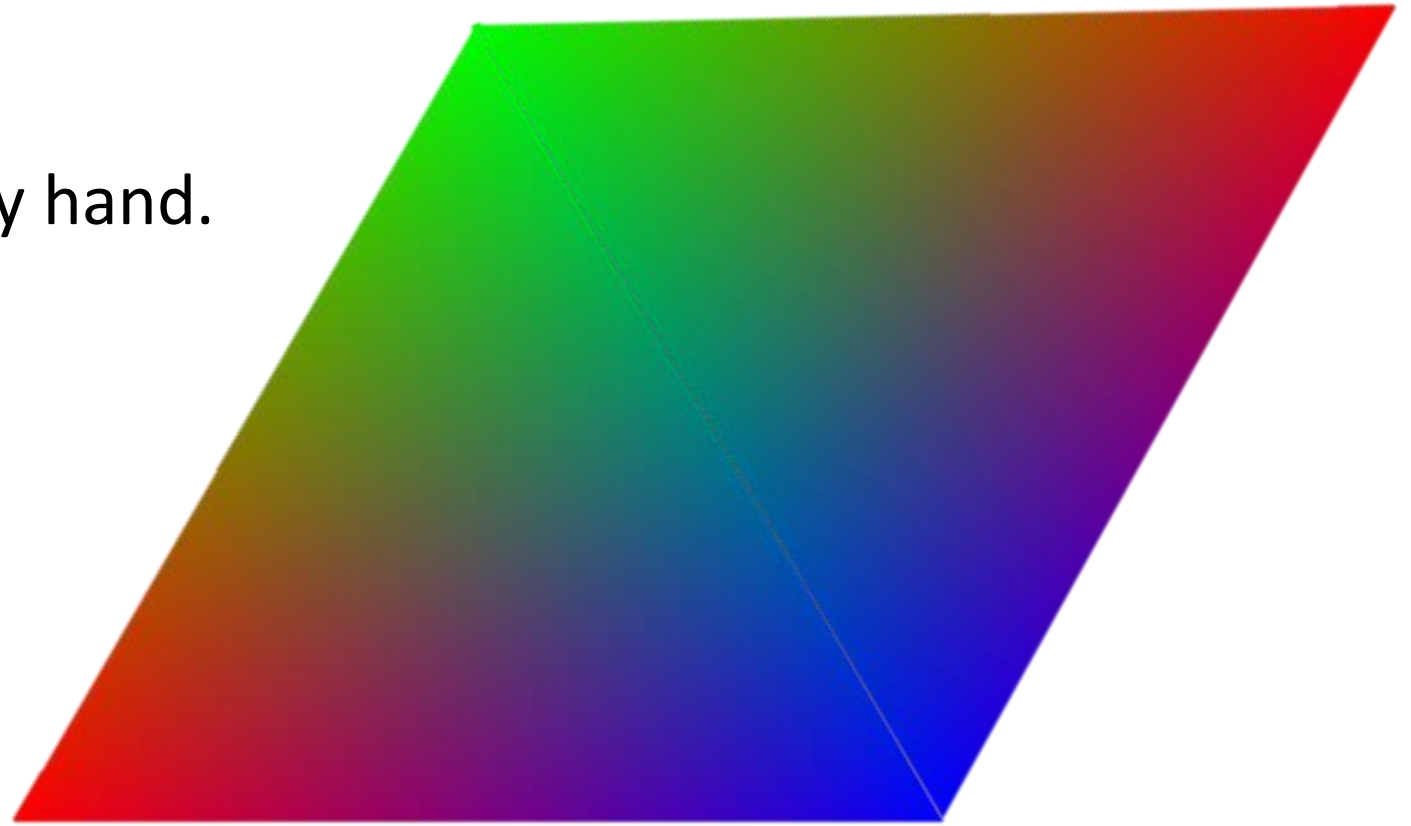
# Colors and objects

- Up to now, we set a color to the vertices and we interpolate the colors in the fragments of the object.



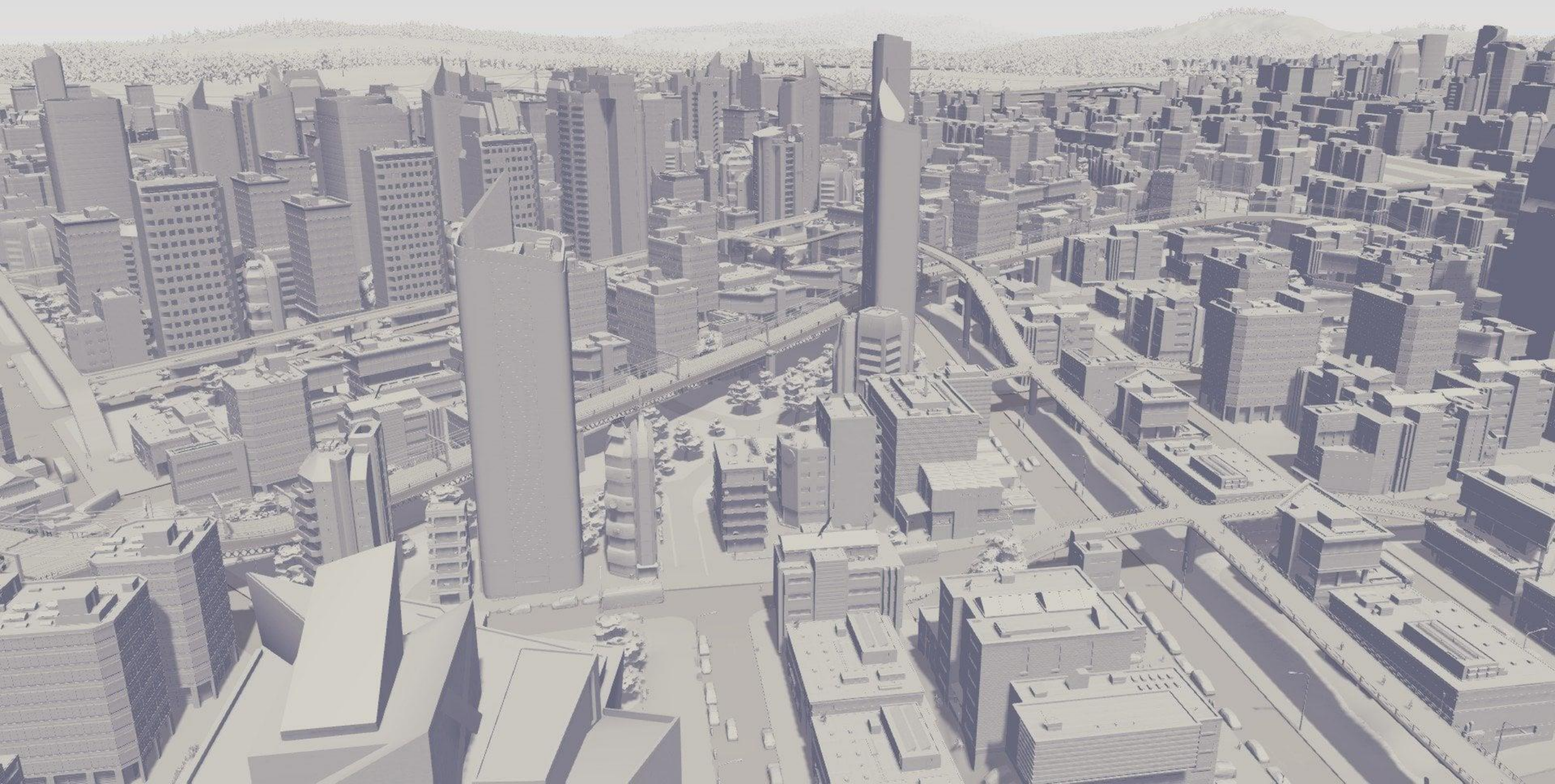
# Colors and objects

- Surely this creates a beautiful triangle
- And with some creativity, we can create intricate patterns
- But.. It is kind of hard to do it by hand.
- Is there an easier way ?

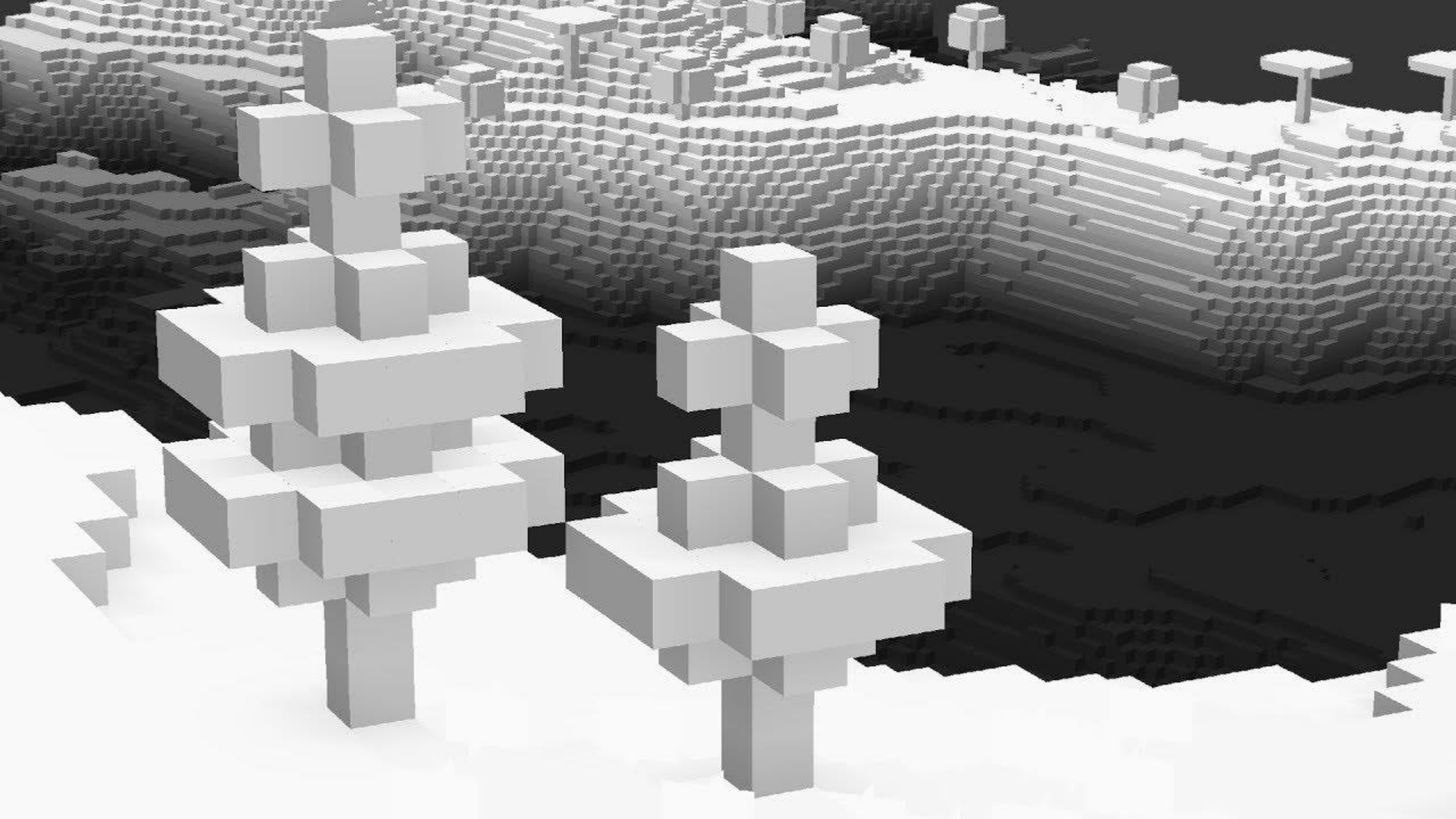


# Typical scene

- Yes!
- We can add something that is called a **texture**
- It is a 2D image which will be mapped to our objects.
- Computer Graphics softwares without textures lack realism !
- Let's see some examples



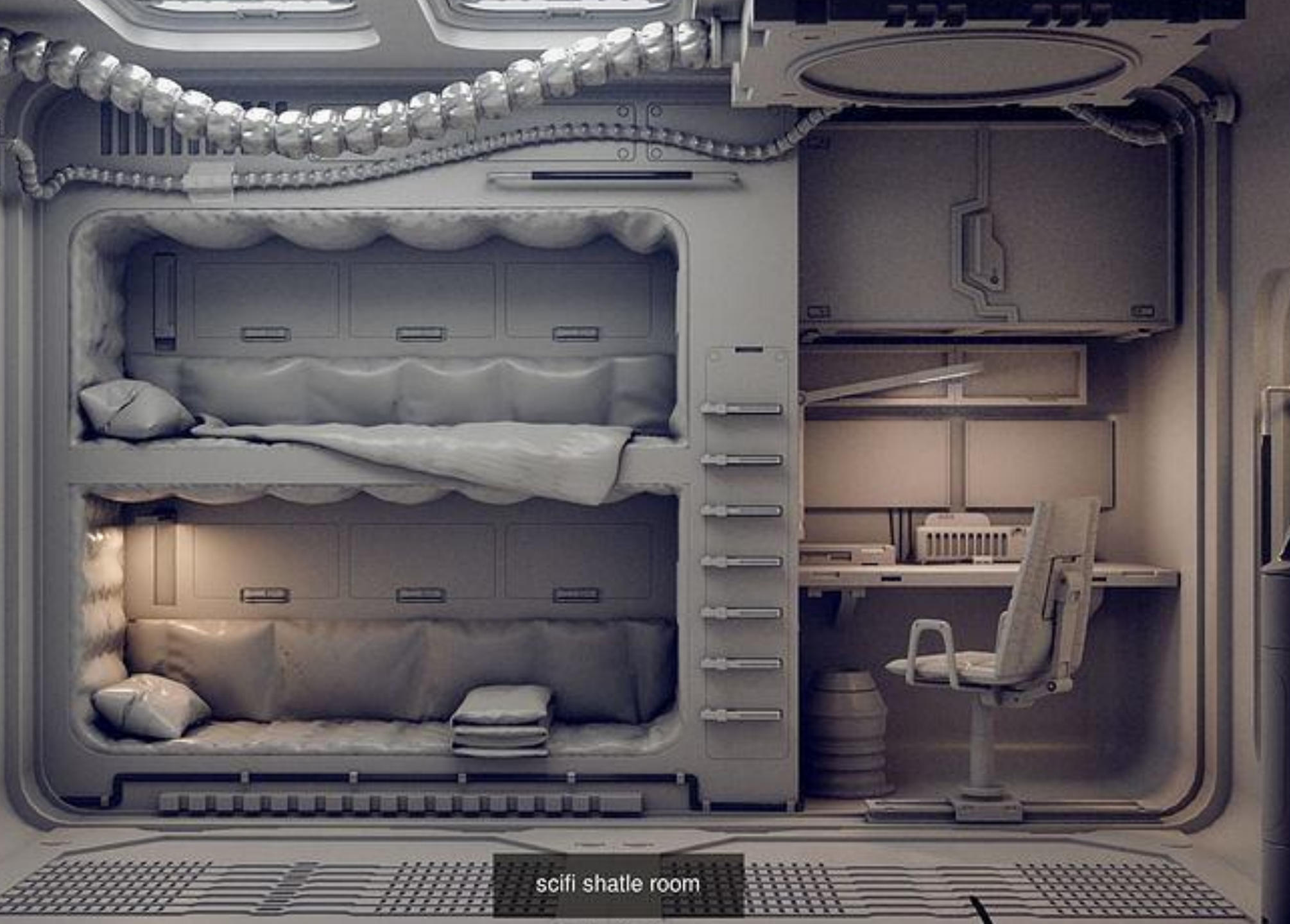




blender

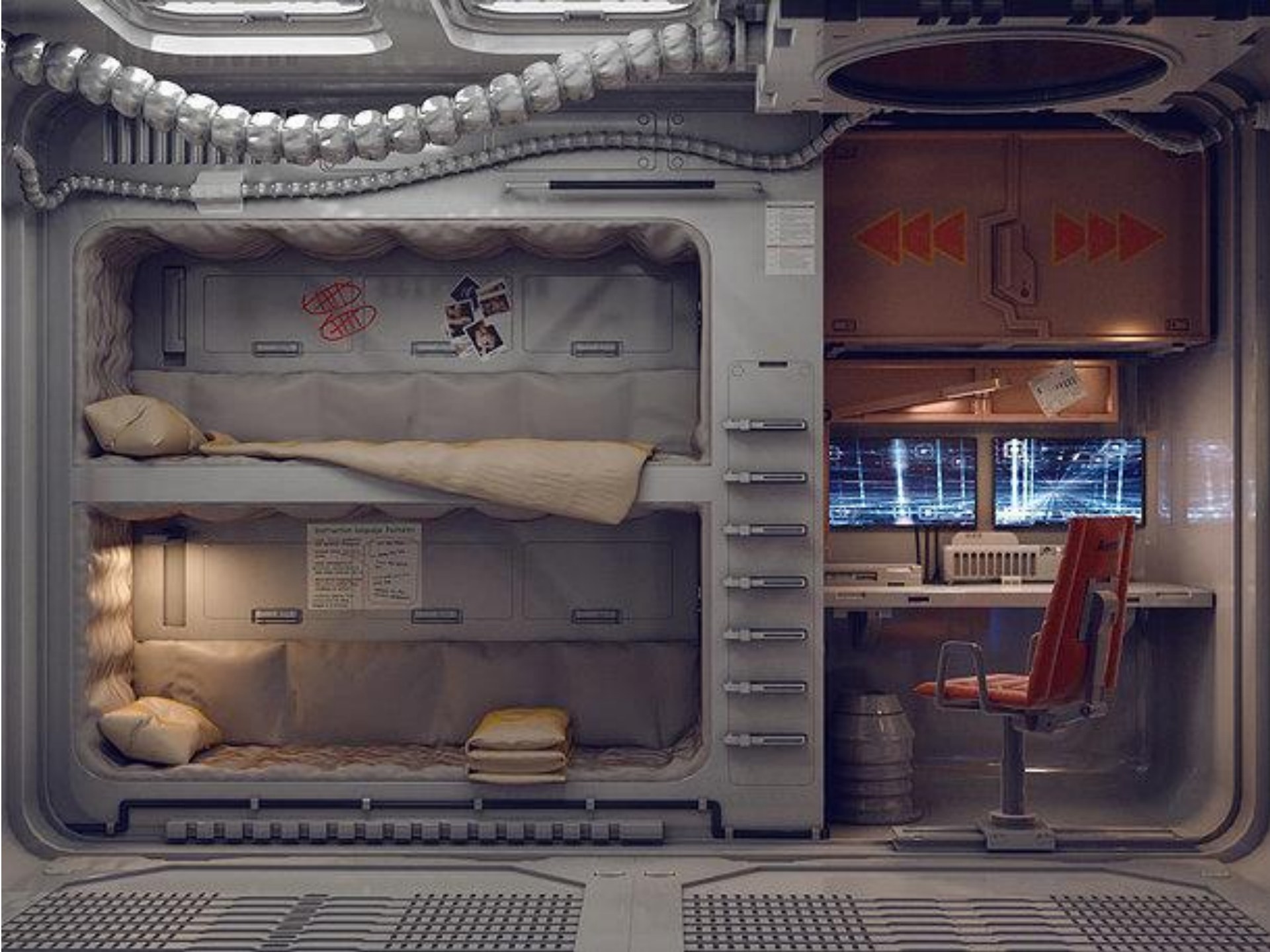




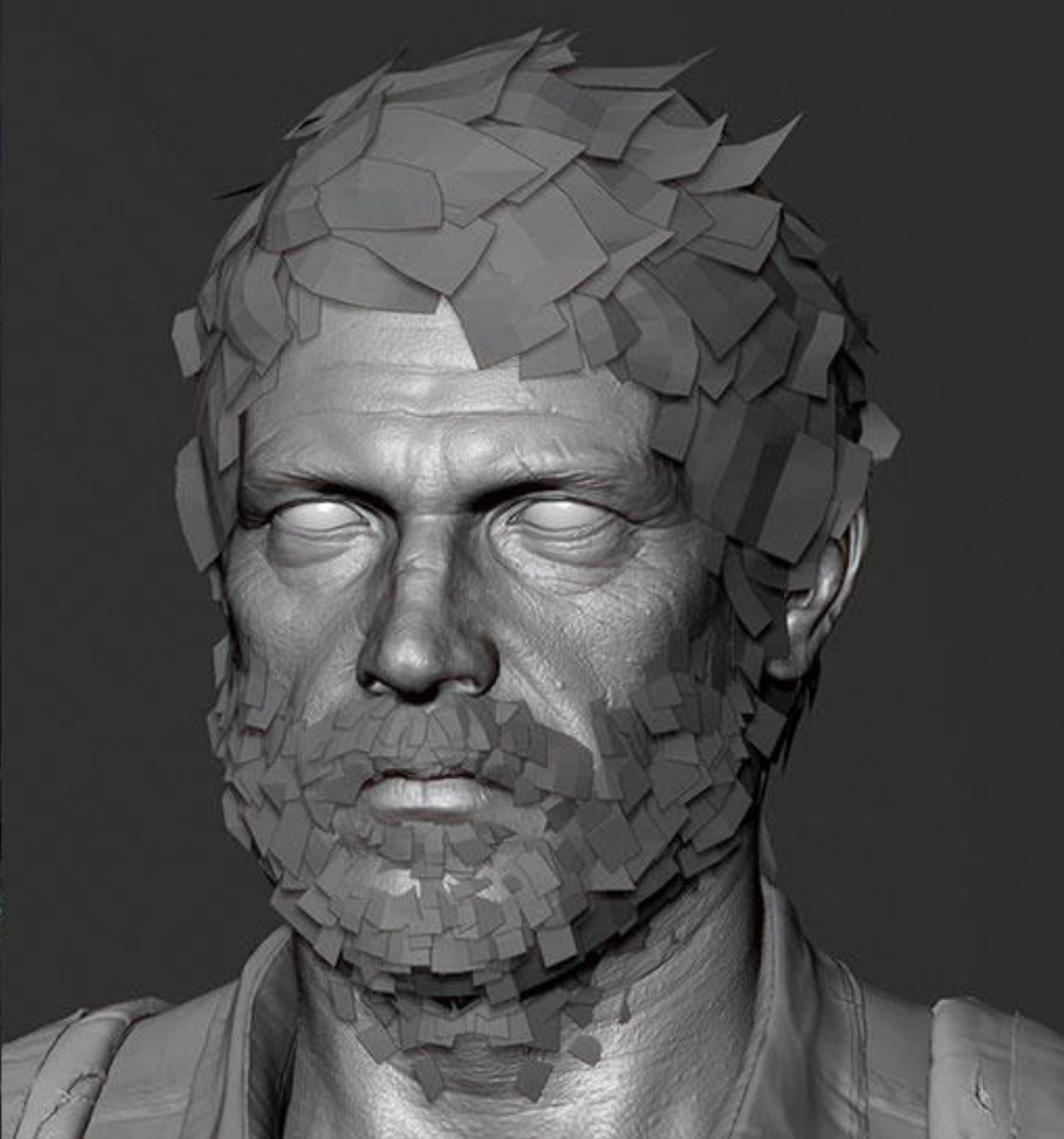


- Even with few textures

scifi shuttle room



- Even with few textures
- The scene becomes suddenly better



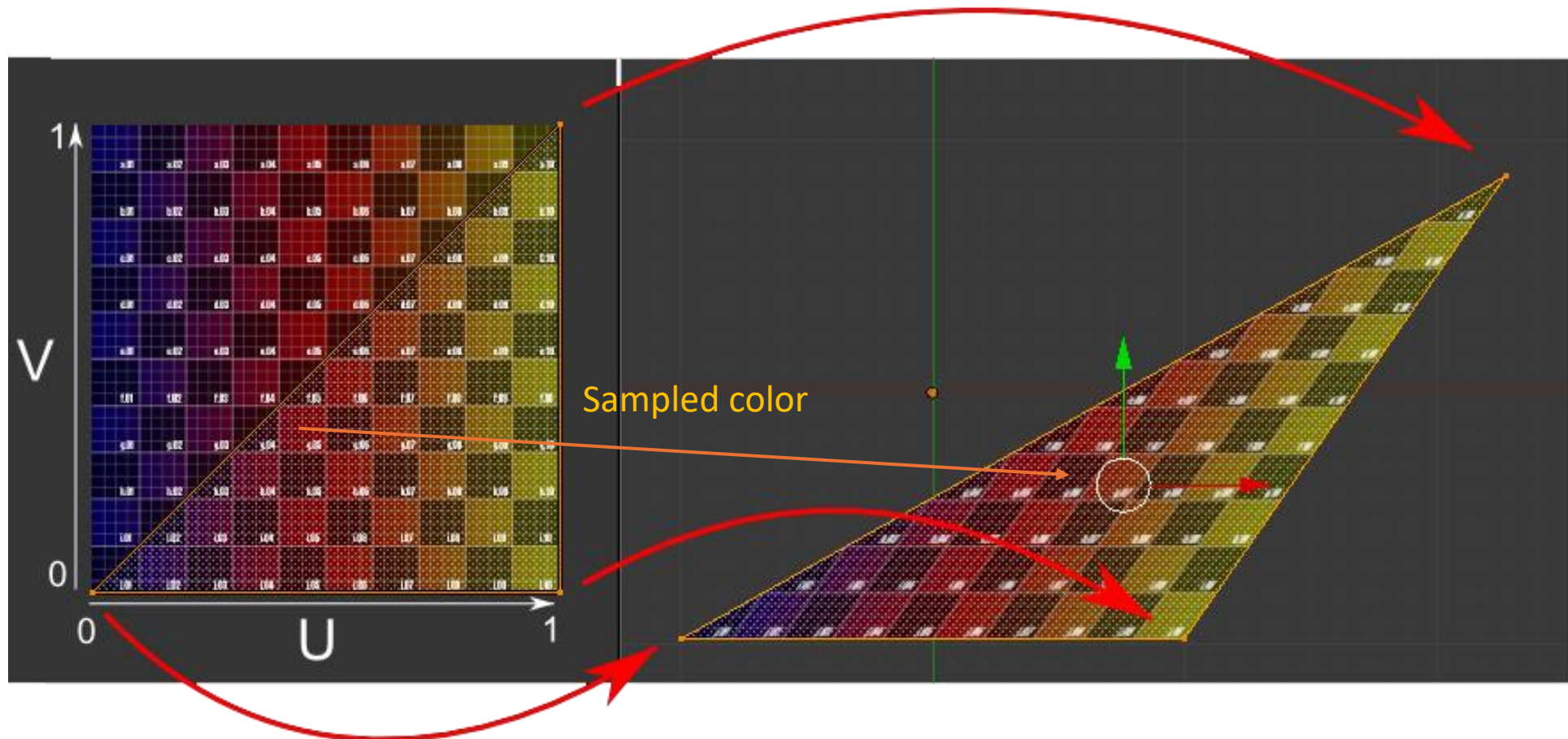




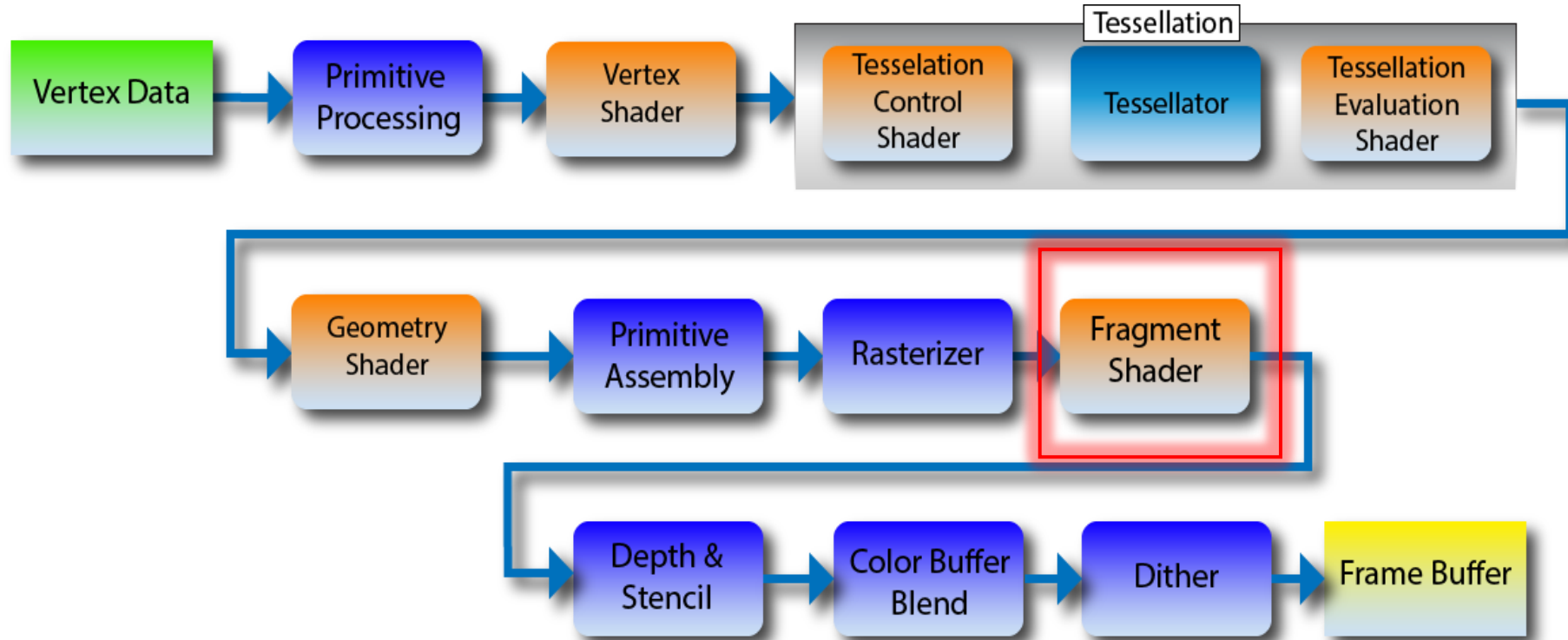


# What is a texture?

- So, to add realism, we want to apply an *image (texture)* to an object.
- Therefore, we map coordinates in the image to vertices (texture coordinates), and *sample* the color to be applied to a fragment in the texture



- We work again in the fragment shader



- The shader receive a texture (image to be applied to our model)
- And our model with texture coordinates

# Creating Textures

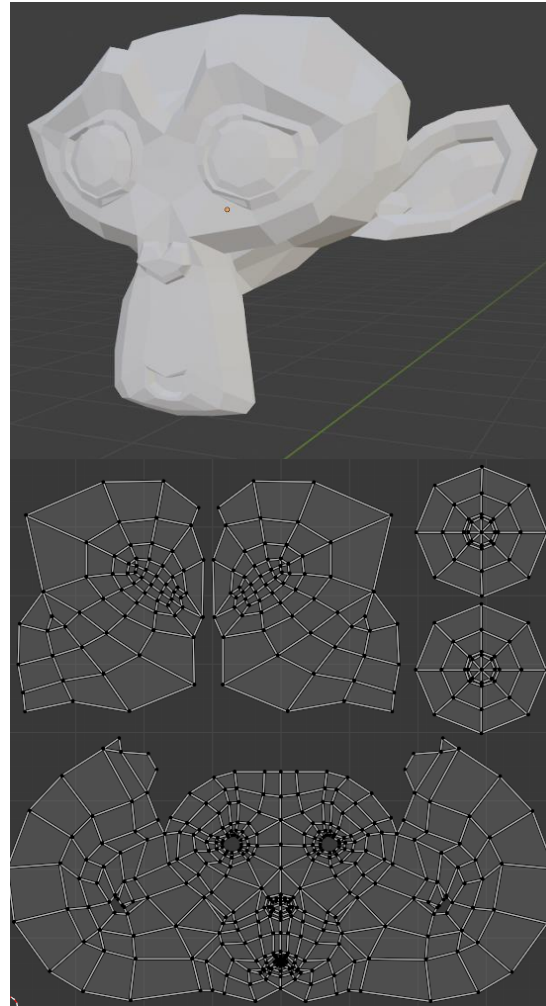
- However.. Textures are 2D image and our object is in 3D
- How to correctly map texture coordinates in a 2D file to the vertices in 3D ?

# Creating Textures

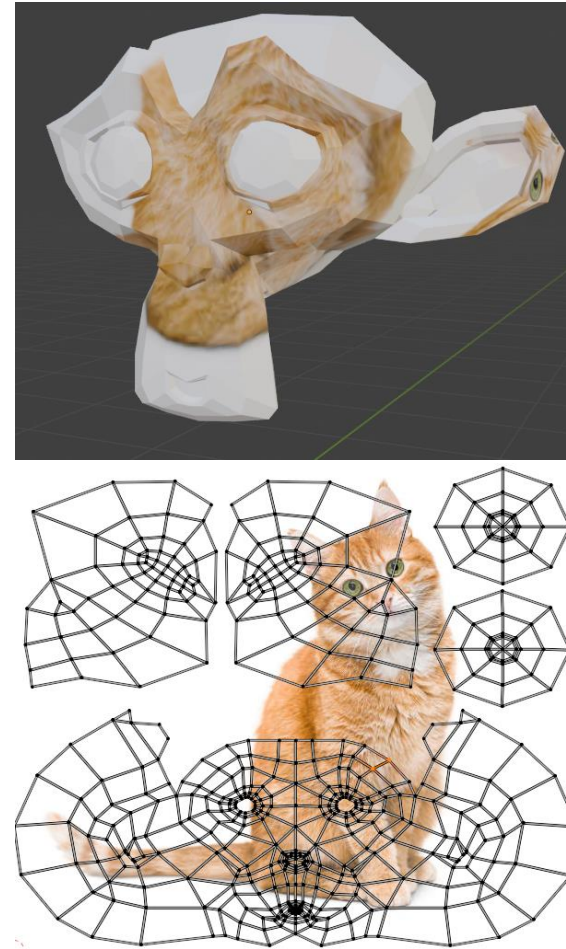
- However.. Textures are 2D image and our object is in 3D
- How to correctly map texture coordinates in a 2D file to the vertices in 3D ?
- We need to *unroll/unwrap* the 3D objects so that it is in 2D.
- This is not an easy task and requires differential geometry.
- Lucky for us.. Blender do it

# How are textures made ?

- We need a way to « unroll » the 3D object, and make a texture correspond to it



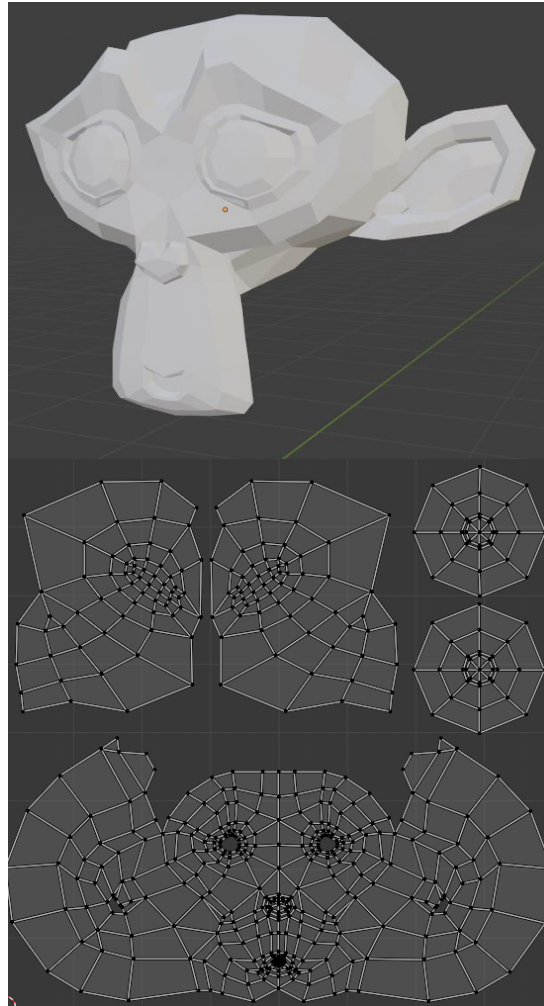
Unrolled object



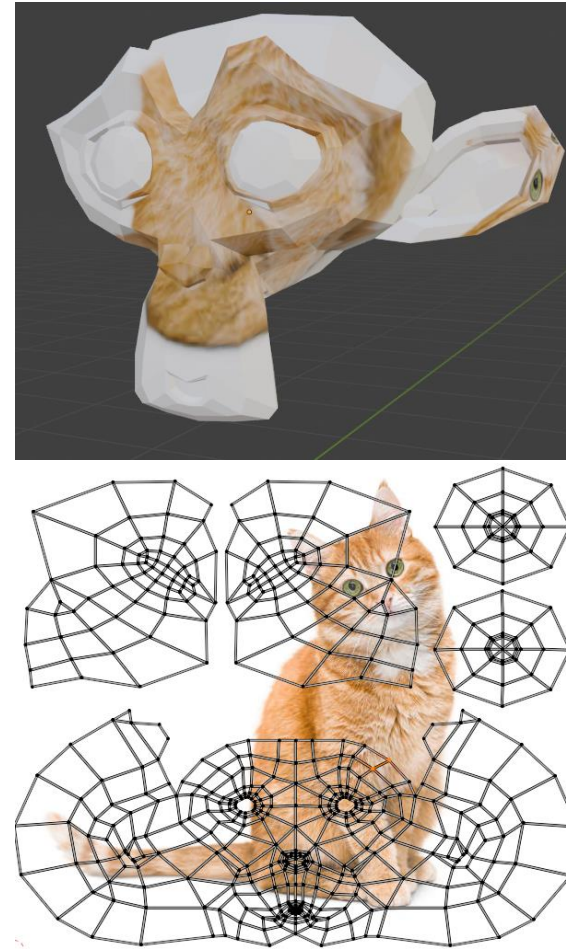
# How are textures made ?

- But.. Even if it is unrolled.. We still need to find a matching picture ?

3D model

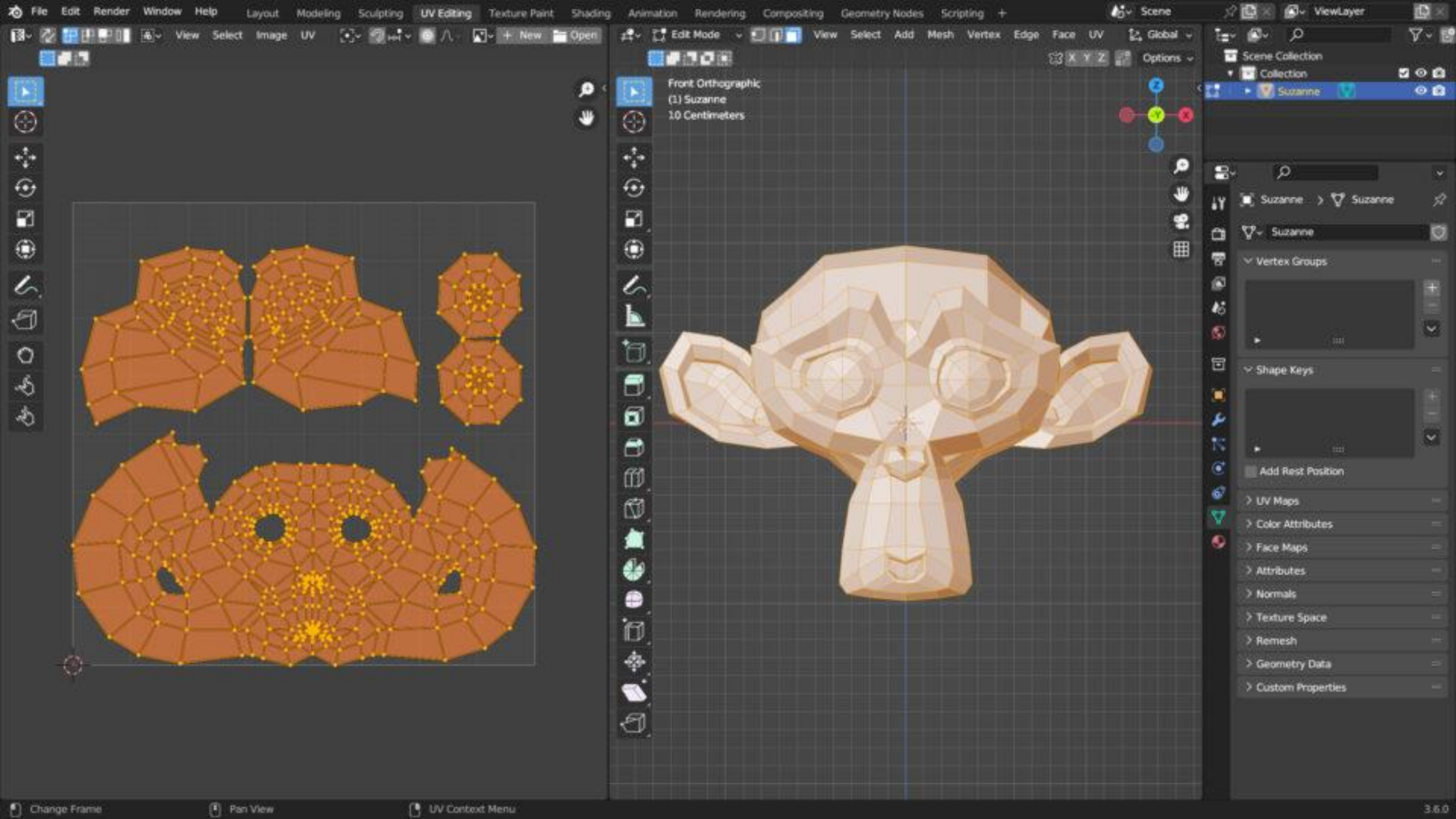


Unrolled object



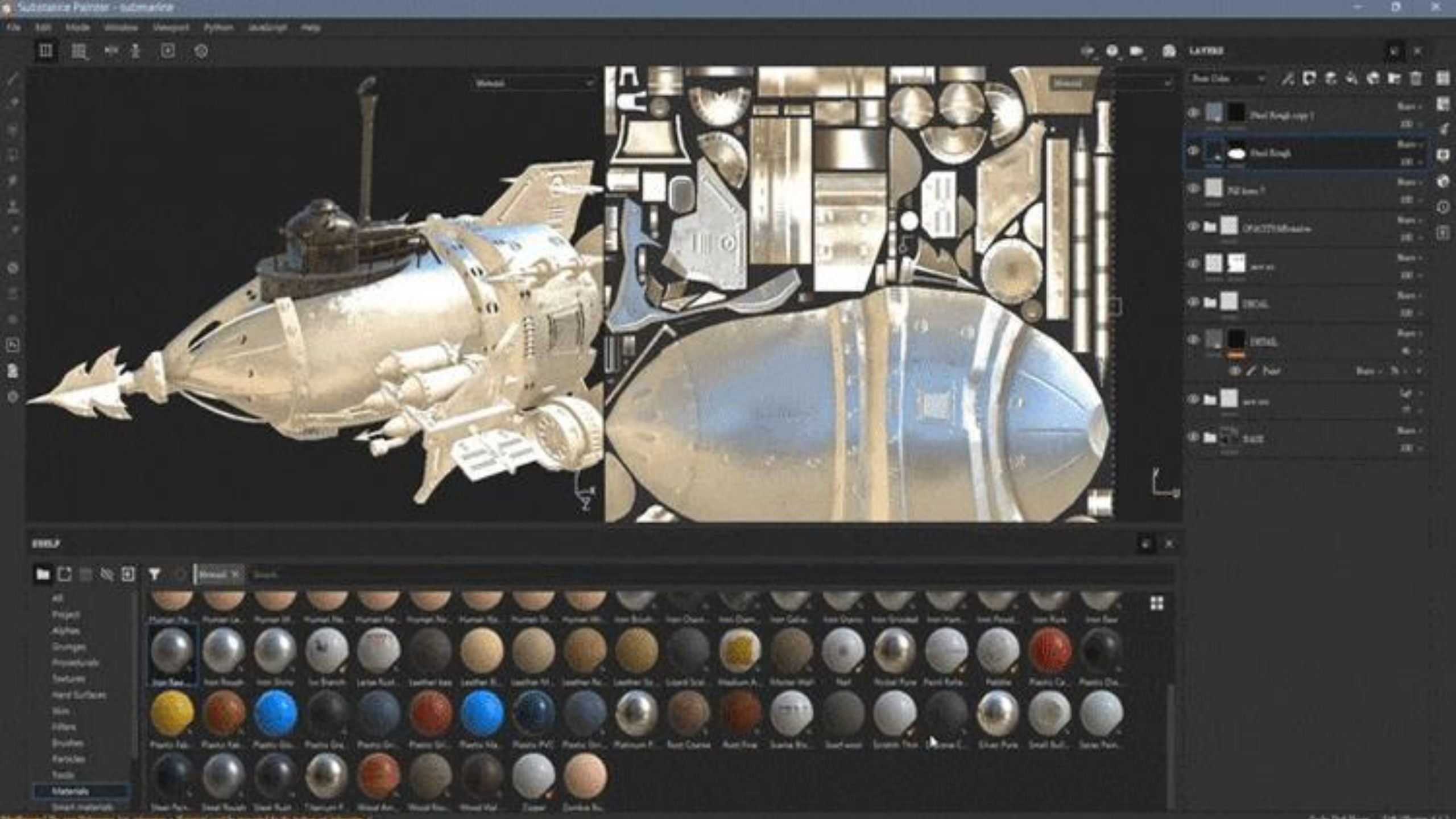
Why the eyes are white?

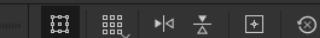
Why the ear contains the cat eyes ?



# How are textures made ?

- But.. Even if it is unrolled.. We still need to find a matching picture ?
- Not really, Blender allows us to paint in 3D on our object, and unroll the texture for us too !
- There are more specialized softwares to do that such as **Substance Painter**





材质



材质



日志 着色器设置 X TEXTURE SBT纹理列表

撤销

重做

AZNPRlit\_Unify

链接 名称

body



着色器参数

Common Parameters

AO Intensity

0.75

RampTex

选择纹理

Matcap

body\_metalMap



图层 TEXTURE SBT纹理设置 X



常规属性

名称

body1

描述

描述

大小

1024



1024

PROPERTIES - 填充



材质

color

emissi

nonmet

fixsha

metals

materi

rampte

材质模式

无可选择资源

或

ramptexture

均一颜色

SHSLP展架



All全部

Project项目

Alphas透贴

Grunges脏迹

Procedurals程序纹理

Textures贴图

Hard Surfaces硬表面

Skin皮肤

Filters滤镜

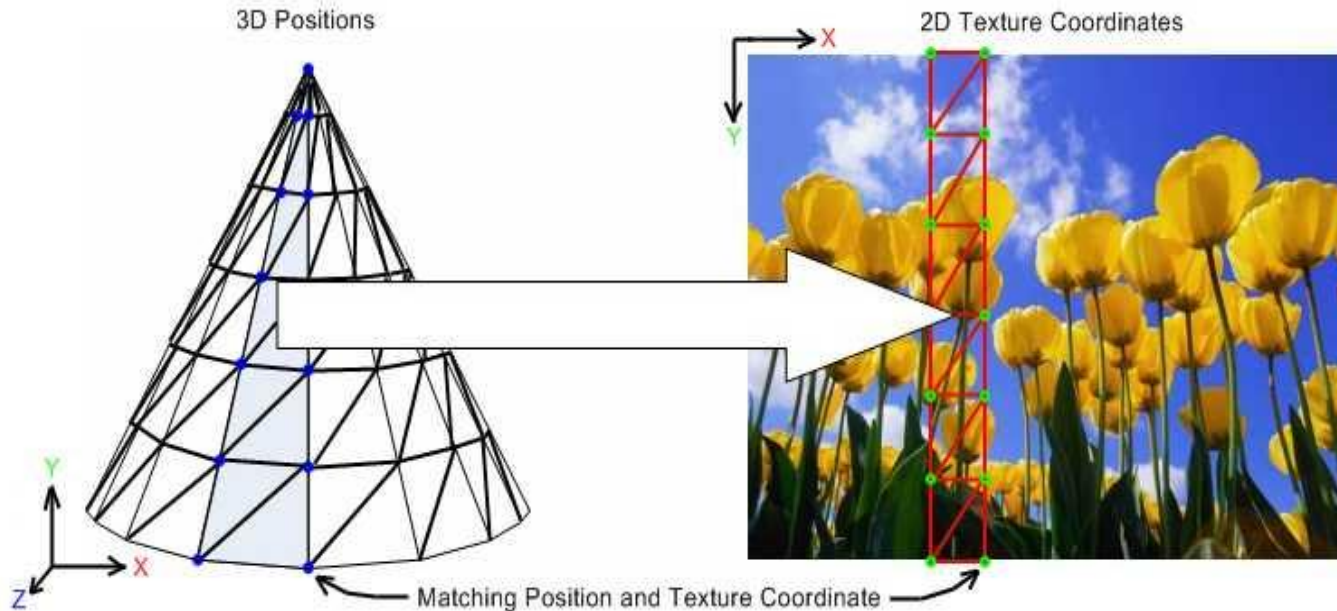
Brushes笔刷

Proj... X 搜索...

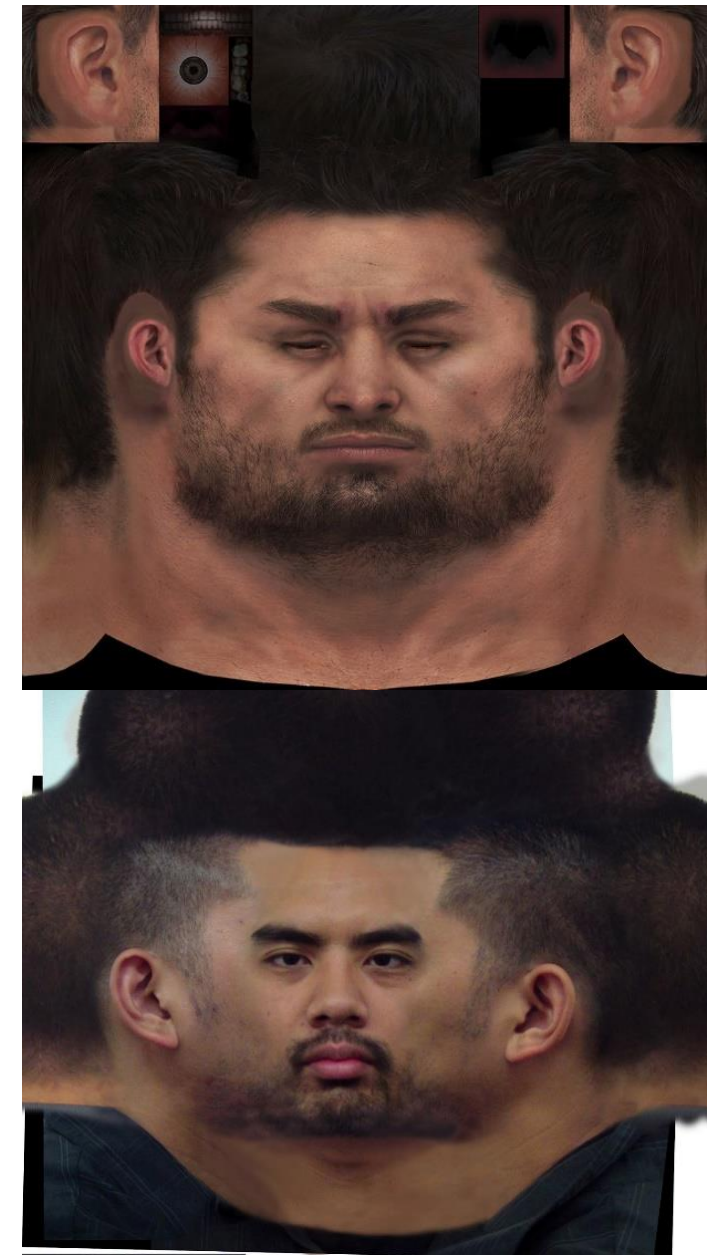


# How are textures made ?

- Texturing can be hard !



- And the texture can be pretty ugly to look at.
- Let's look at some examples of unwrapped textures.

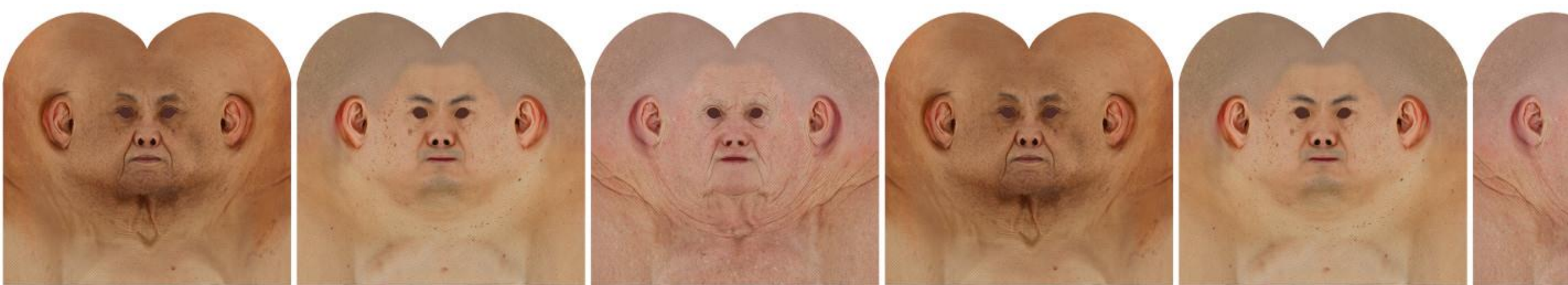


Maths: Heckbert, Survey of Texture Mapping, 1986

and: [https://nbviewer.org/github/gpeyre/numerical-tours/blob/master/python/meshdeform\\_1\\_parameterization.ipynb](https://nbviewer.org/github/gpeyre/numerical-tours/blob/master/python/meshdeform_1_parameterization.ipynb)

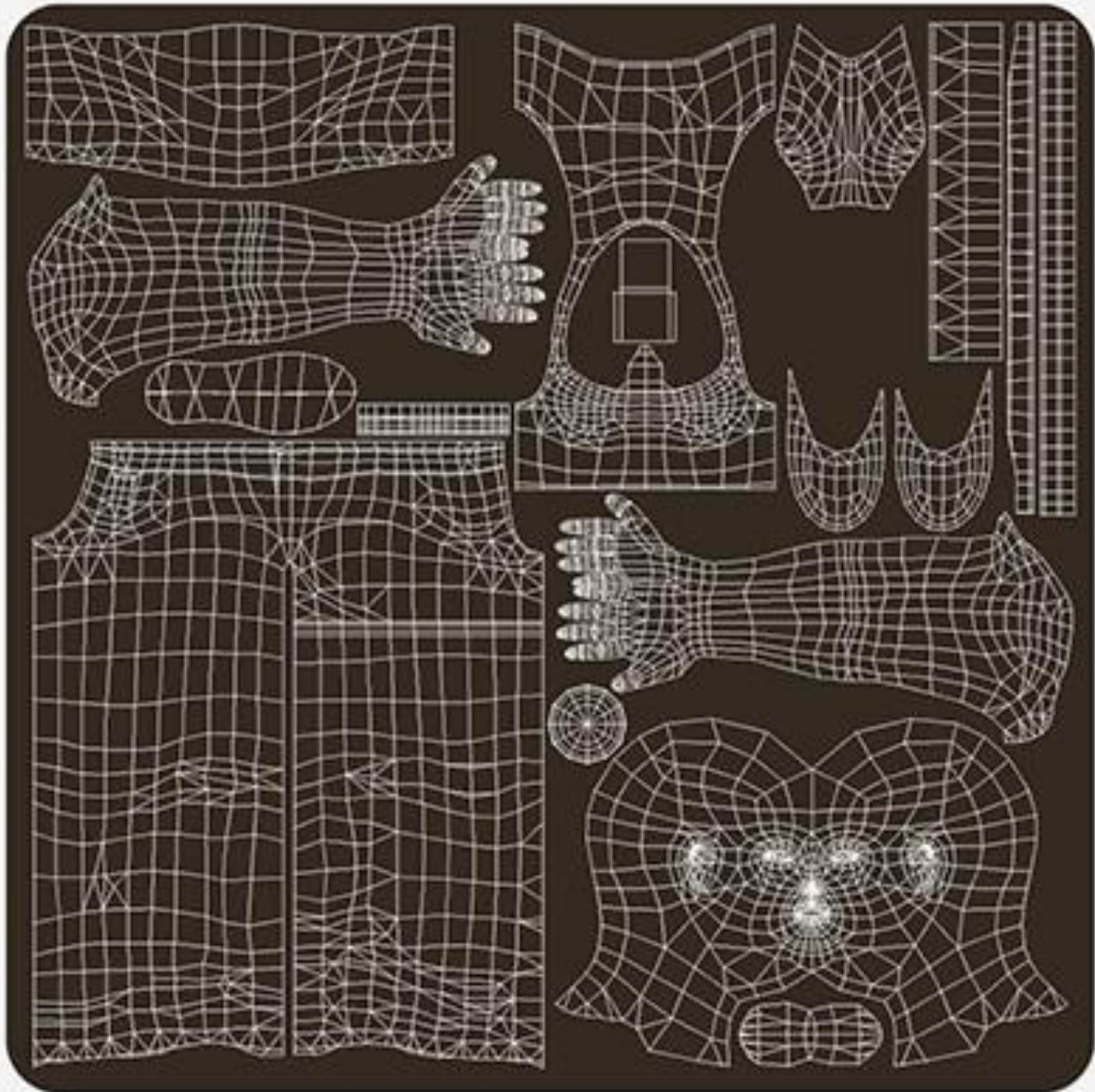






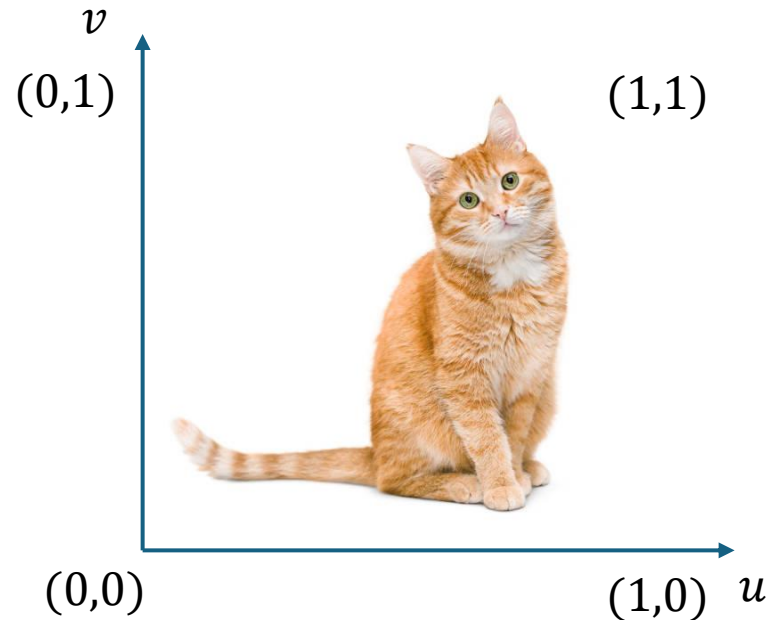






# Texture space

- Textures are in 2D images that specify the color of a 3D object
- Texture coordinates are in  $[0,1]^2$

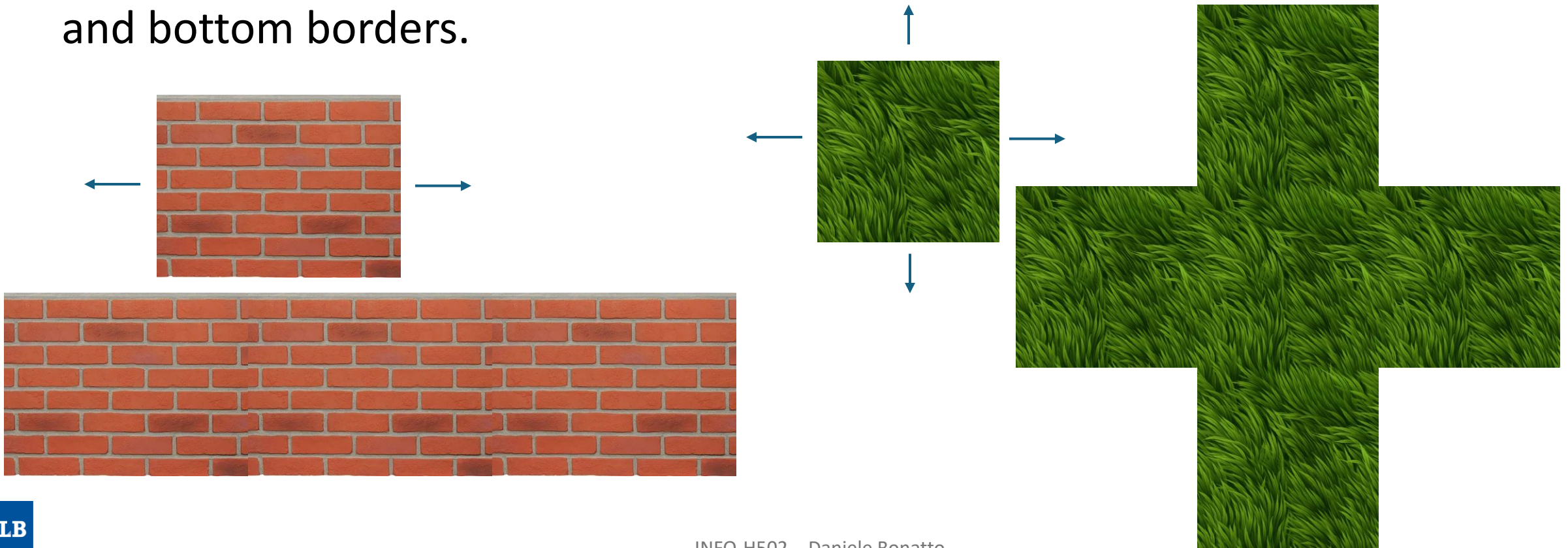


- The coordinates system often uses  $(u, v)$  or  $(s, t)$  as names for the axis
- **Sampling** a texture means retrieving the color at a particular  $(u, v)$ .
- A **texel** refer to the color sampled in the texture.

We'll see later why it is called *sampling*

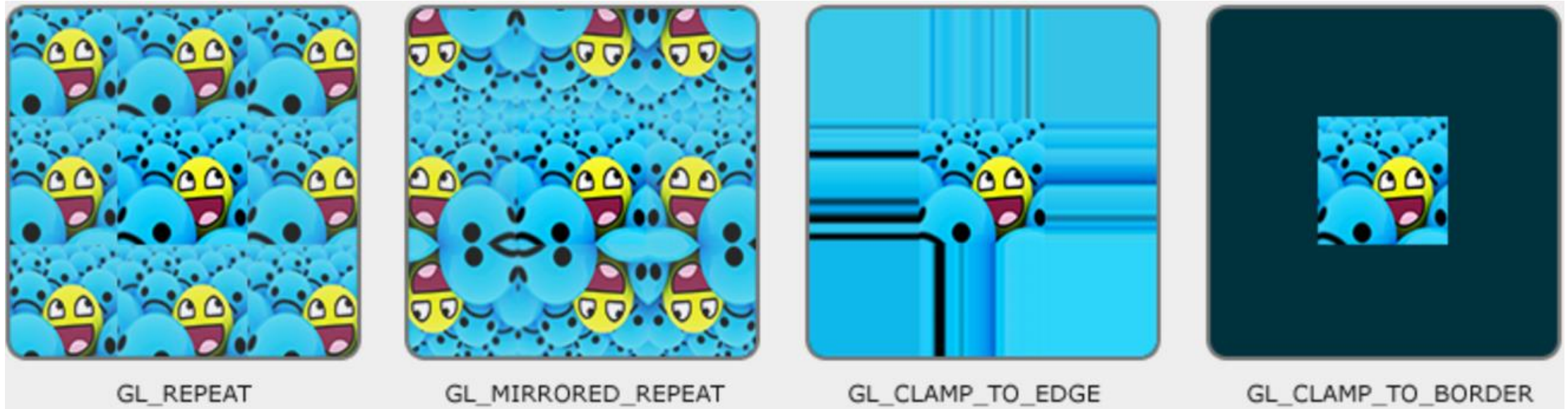
# Textures warping

- While  $(u, v) \in [0,1]^2$
- We can sample outside of this square, resulting in texture warping
- Useful when textures are repeated in space. For example a brick wall
- Allow to use textures with corresponding left and right borders and/or top and bottom borders.



# Textures Warping

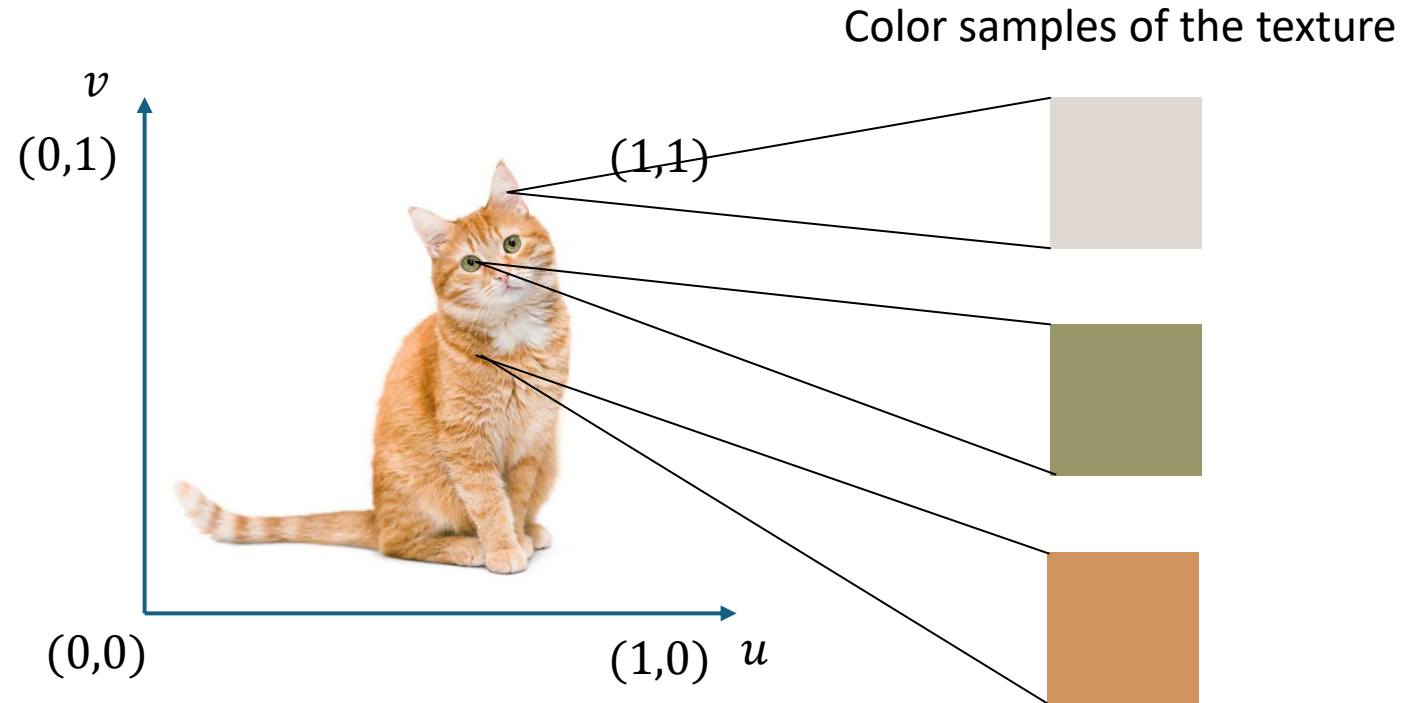
- In OpenGL you can define several other warping.



- In compute shader, we can for example solve differential problems.
- Mirror and Clamp to edge are useful to make border conditions.

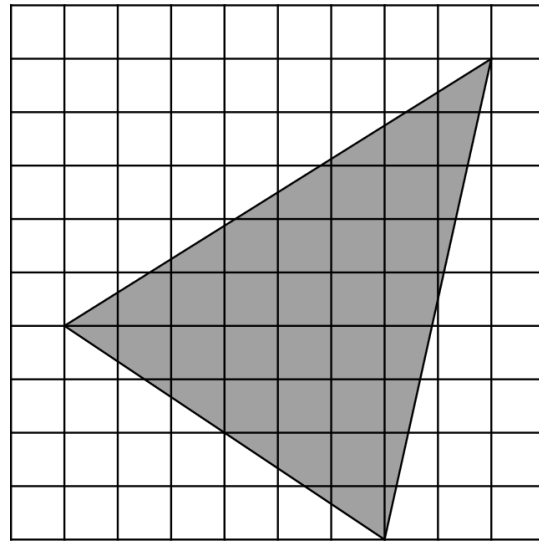
# Texture Sampling

- Let see why we say we sample the texture and not just fetch a color from it

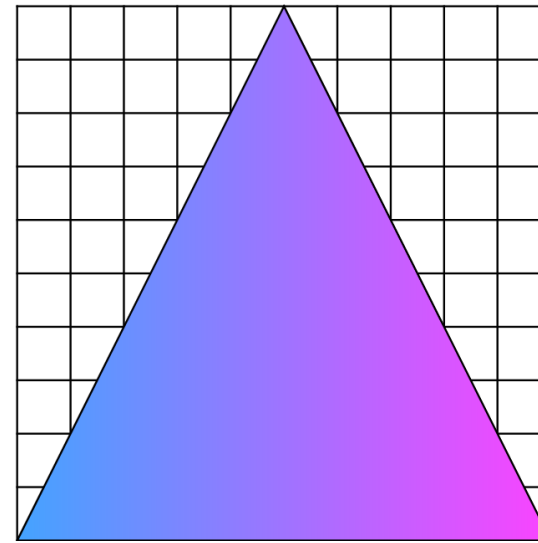


# Texture Sampling

- Suppose you projected a 3D triangle to the screen
- And you want to colorize it using this texture



Screen

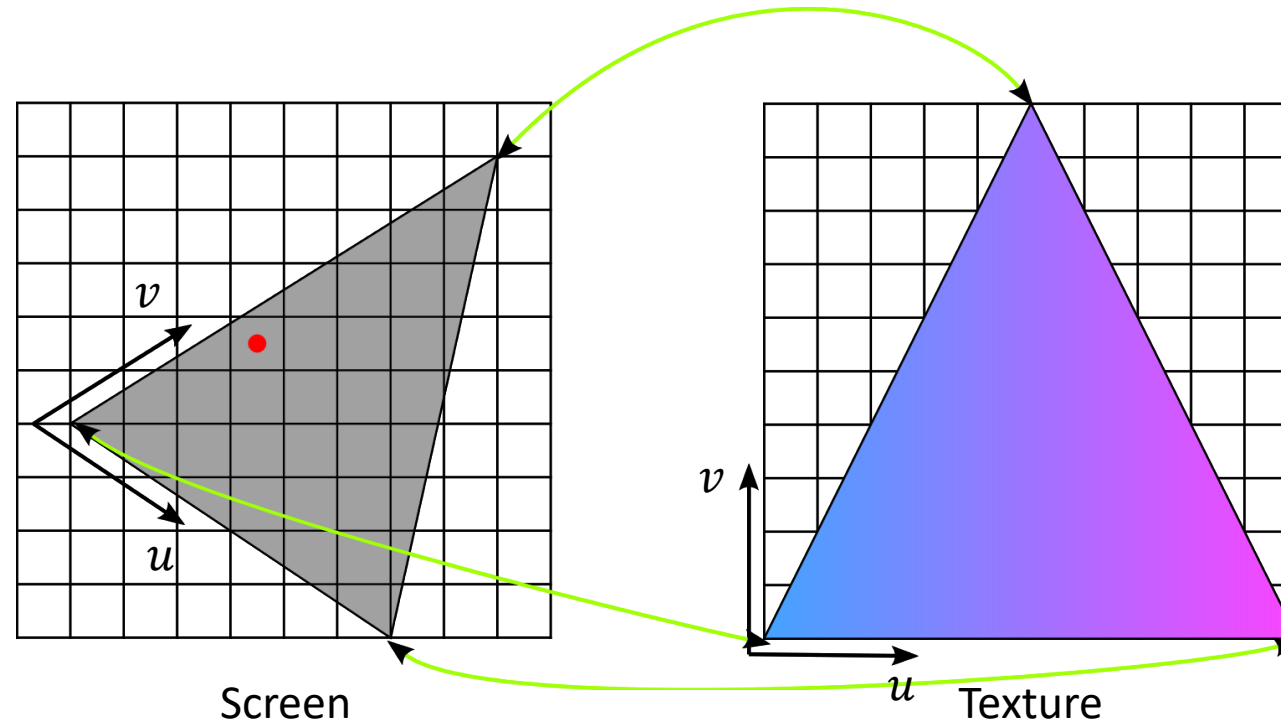


Texture

- What is the first step ?

# Texture Sampling

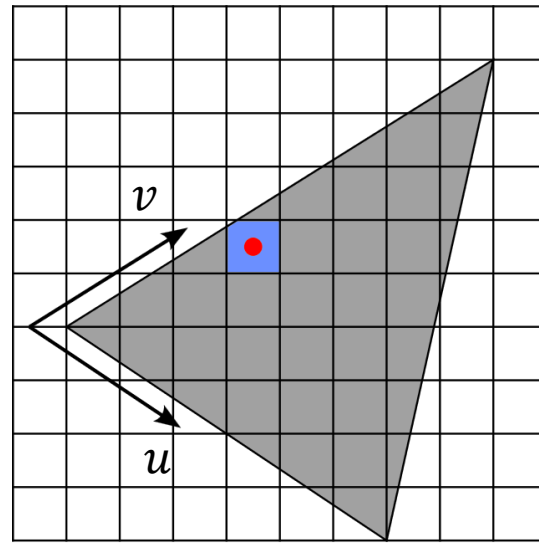
- First of all, we need a mapping between the vertices and the texture



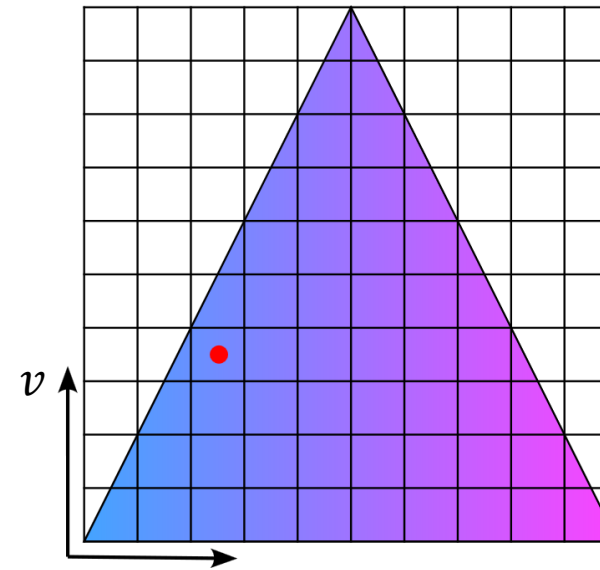
- What is the color where the red dot is ?

# Texture Sampling

- What is the color where the red dot is ?



Screen

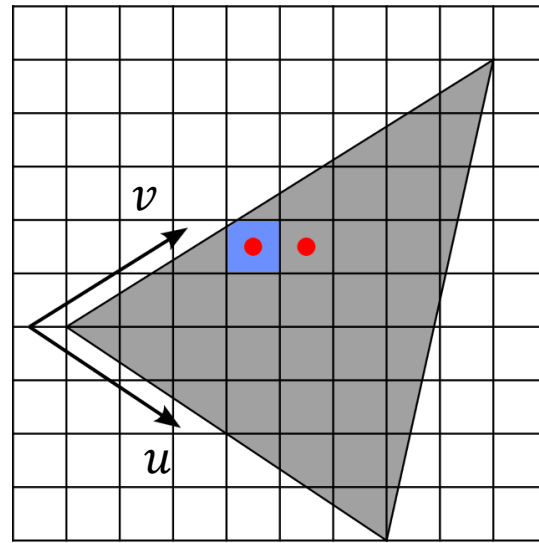


Texture

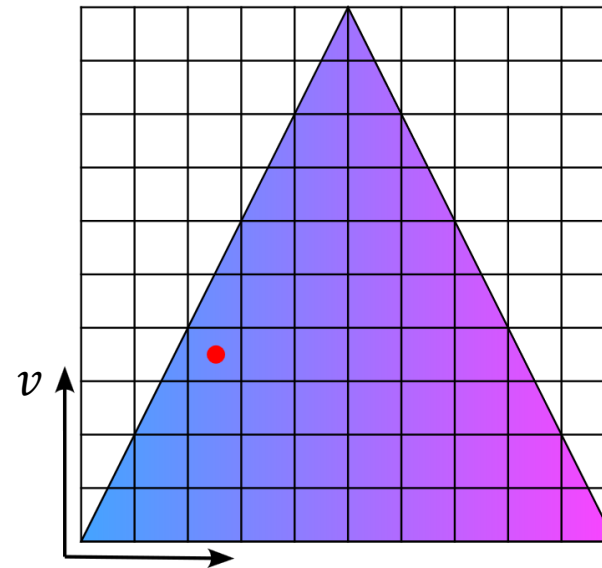
- Okay, it was easy !

# Texture Sampling

- Okay, it was easy !



Screen

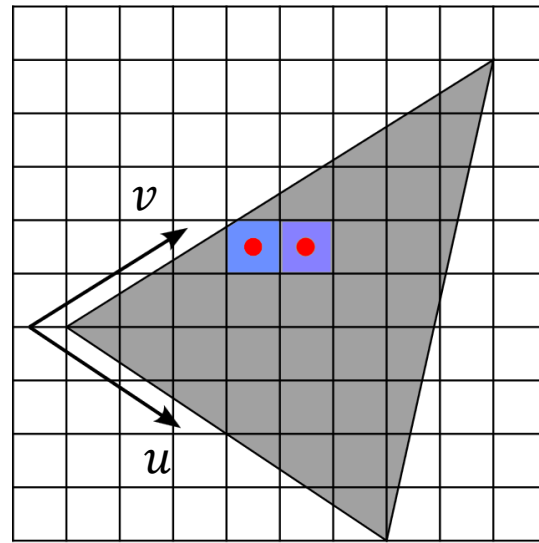


Texture

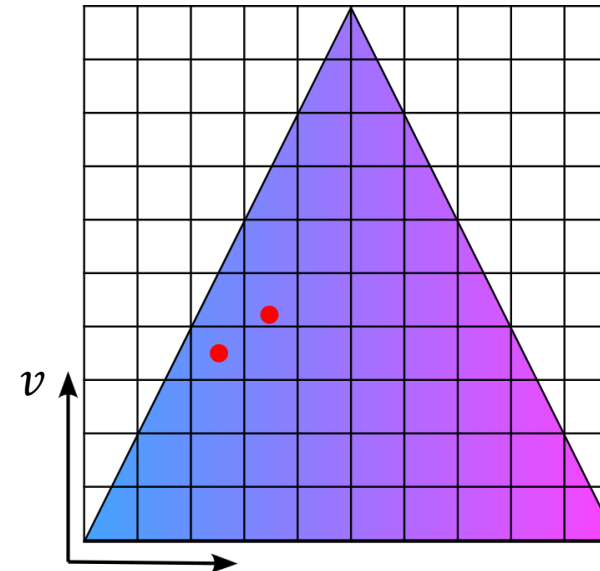
- What about the next color horizontally in the screen ?

# Texture Sampling

- What about the next color horizontally ?



Screen

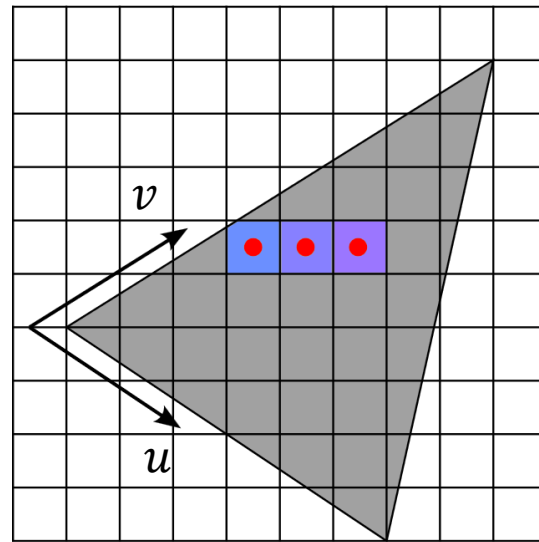


Texture

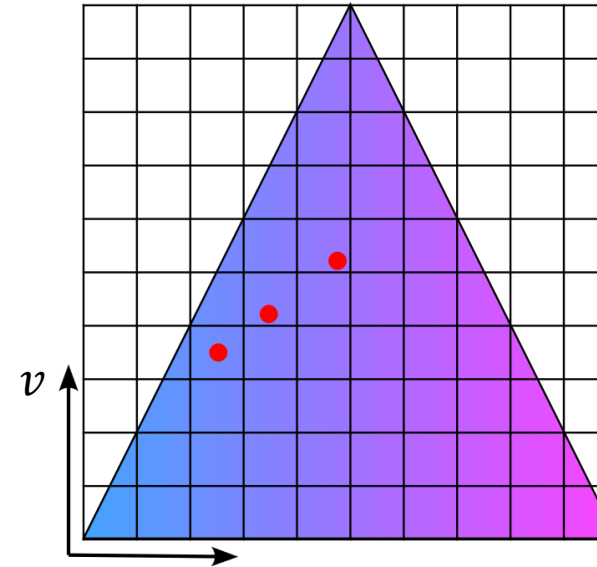
- And the next ?

# Texture Sampling

- And the next ?



Screen

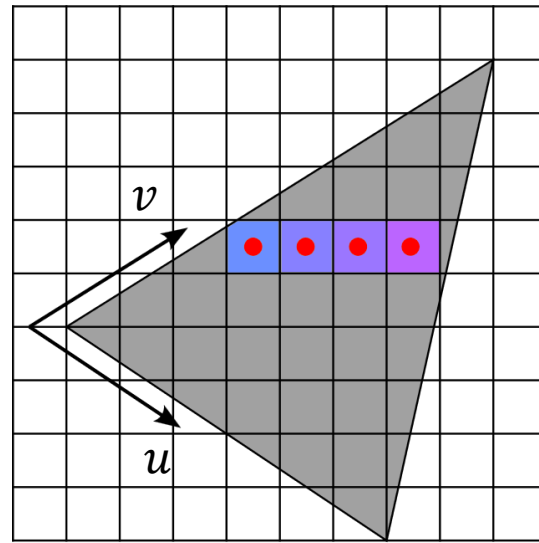


Texture

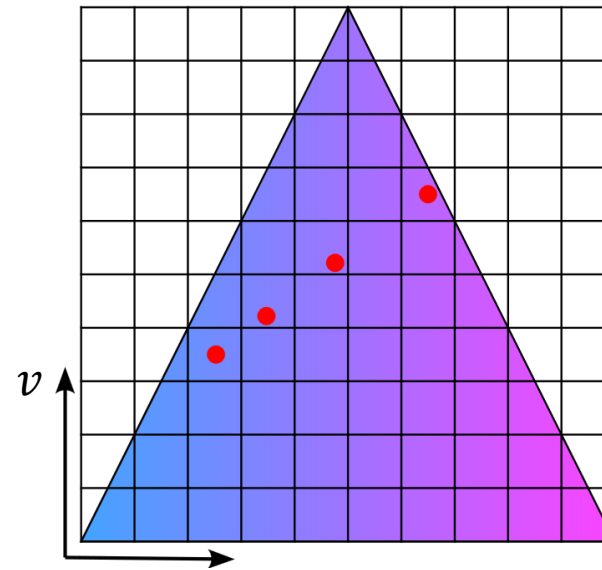
- And the next ?

# Texture Sampling

- And the next ?



Screen

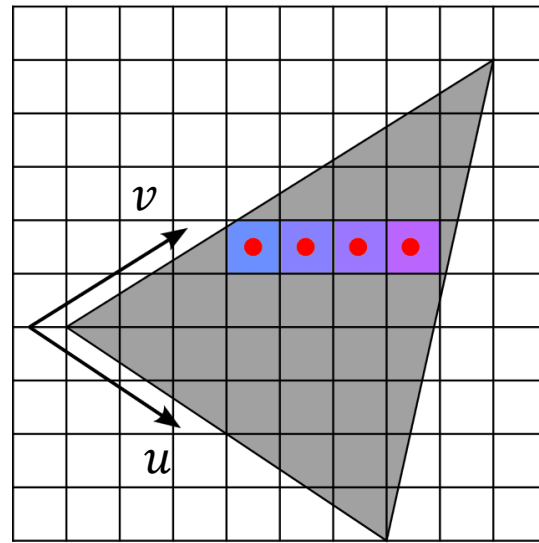


Texture

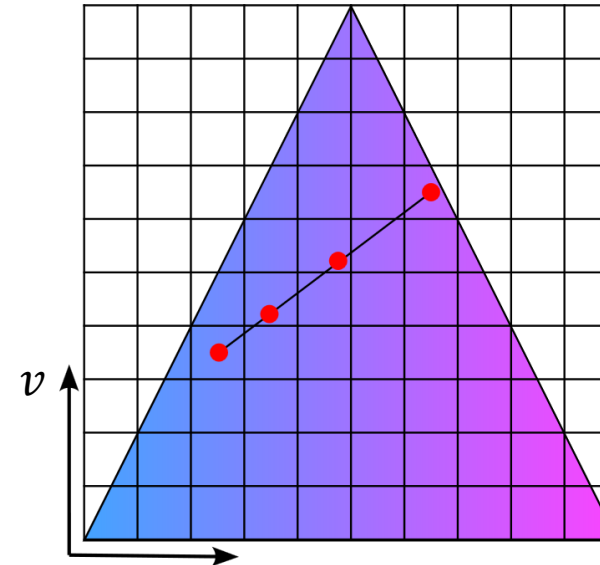
- Notice something ?

# Texture Sampling

- Notice something ?



Screen

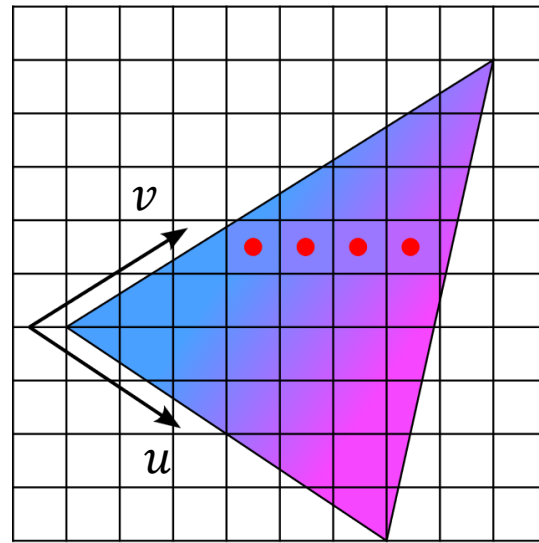


Texture

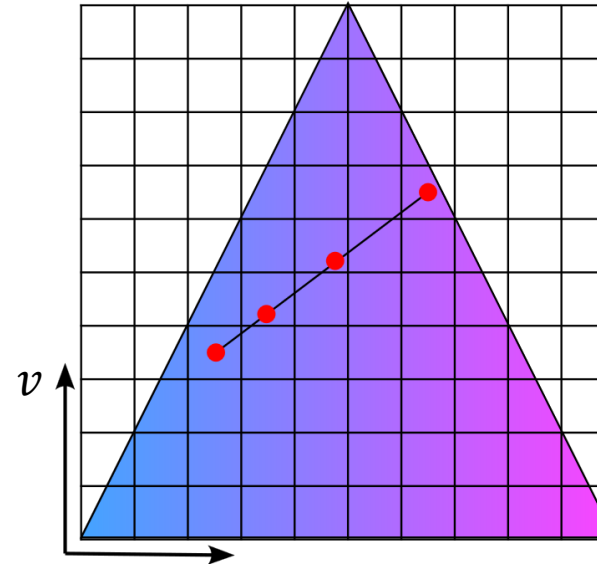
- We **sample** the colors following the depth of the object ! **Not linearly**
- Let's colorize all the rest.

# Texture Sampling

- We do not sample the colors linearly on the texture.. Is that a problem?



Screen

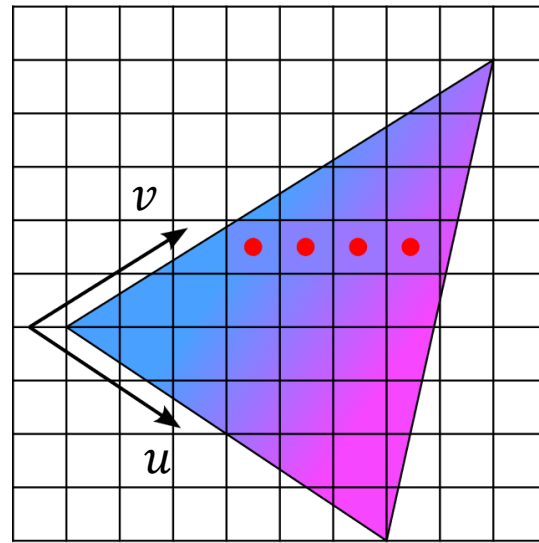


Texture

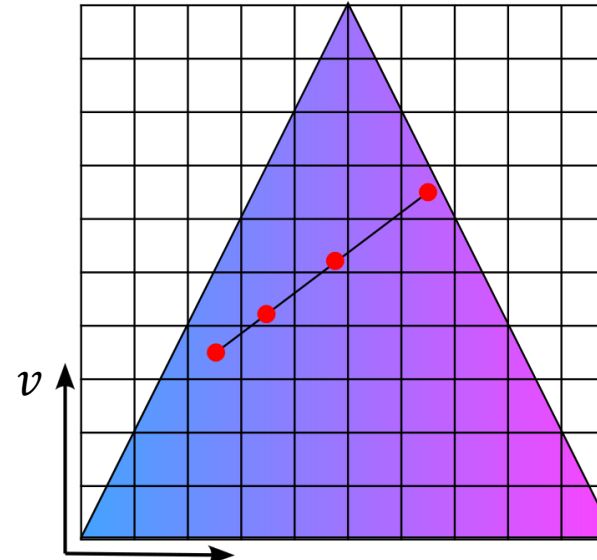
- The triangle was projected close to the screen, there is not really an issue
- We just need to find the color on the line between the dots !

# Texture Sampling

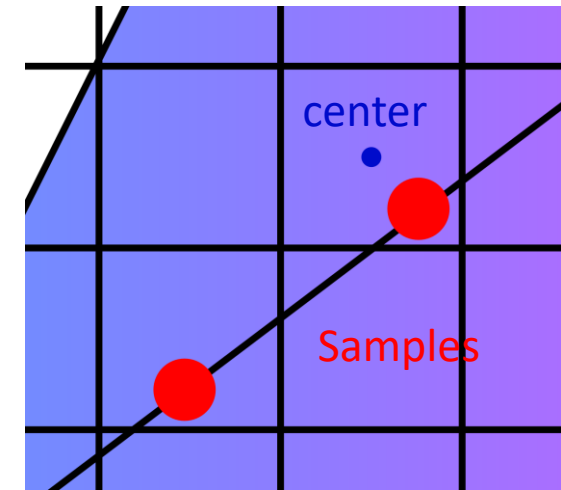
- Here, the triangle was projected close to the screen, in this case there is not really an issue.



Screen



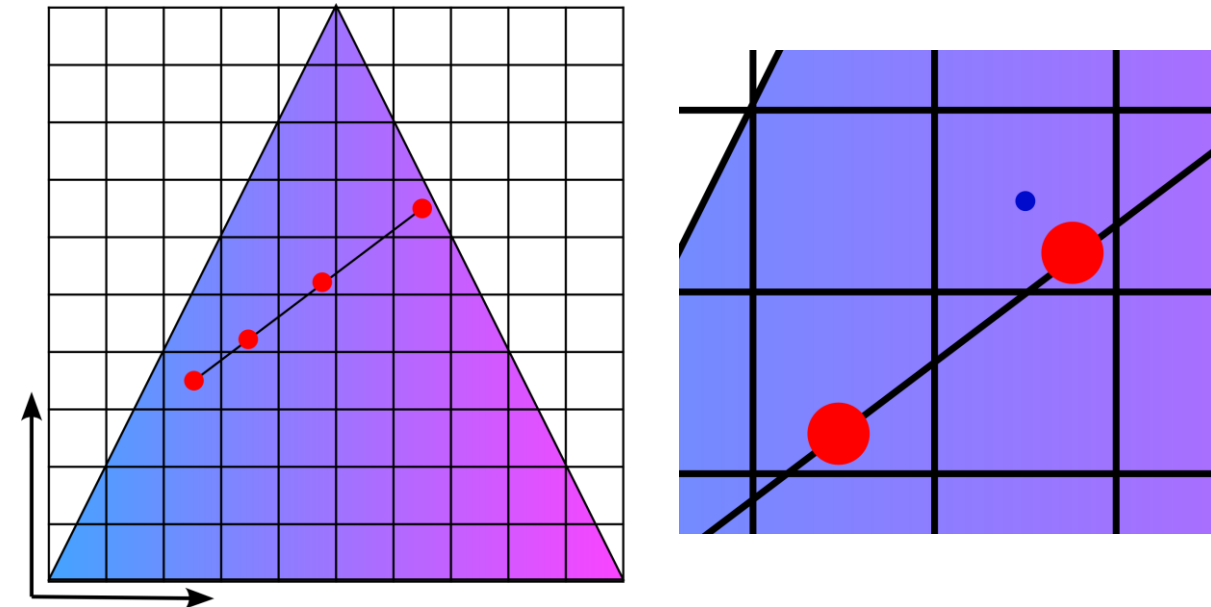
Texture



- But the samples do not correspond to the center of a pixel in the texture.
- Is this a problem?

# Texture Sampling – Nearest Neighbor Interpolation

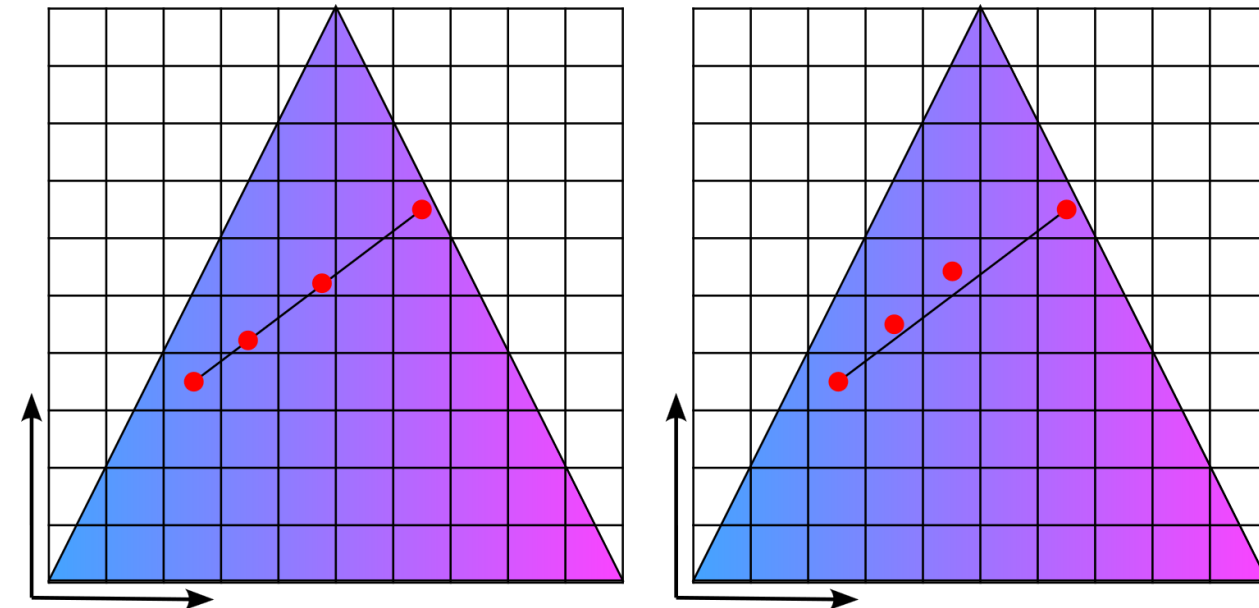
- The sampling point is not at the center of the pixels in the texture
- So what color should we take ?



# Texture Sampling – Nearest Neighbor Interpolation

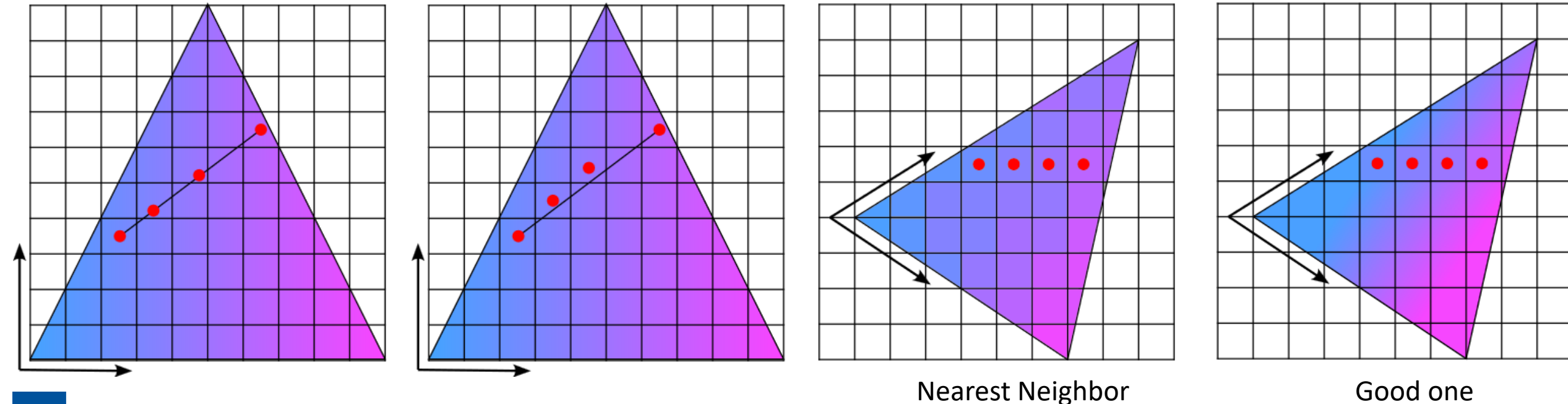
- But the sampling point is not at the center of the pixels in the texture
- So what color should we take ?
- A first approach is **nearest-neighbor interpolation**

Does it create good results?



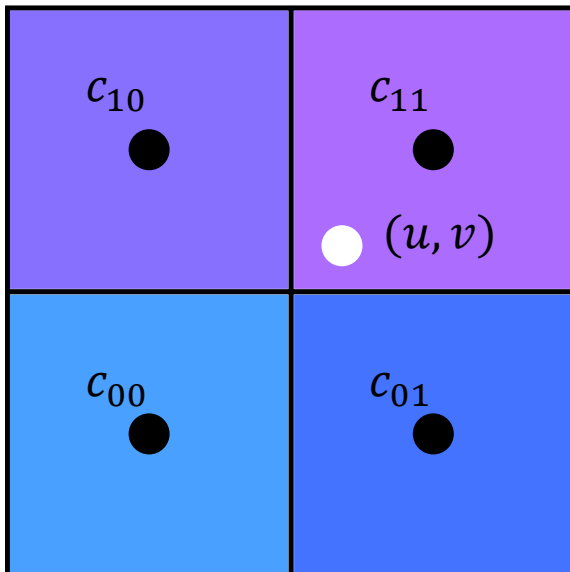
# Texture Sampling – Nearest Neighbor Interpolation

- But the sampling point is not at the center of the pixels in the texture
- So what color should we take ?
- A first approach is **nearest-neighbor interpolation, no good results**
- What should we do ?



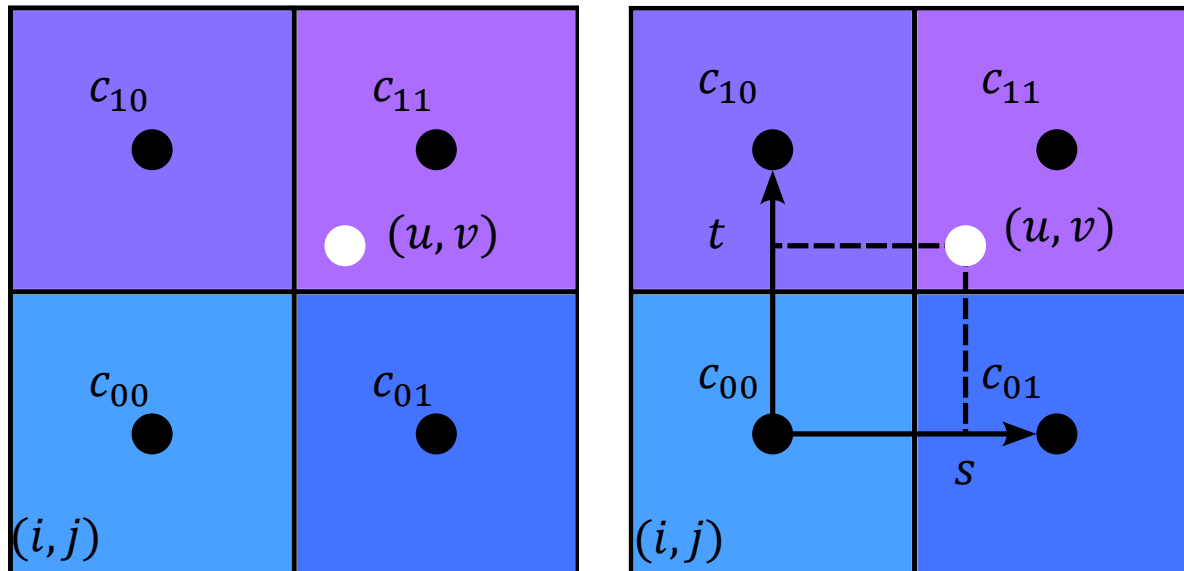
# Texture Sampling – Bilinear Interpolation

- Instead of using the nearest neighbor, we can *interpolate* the color between texels.
- Let's find the color of the white dot.
- We expect it to have a mixture of  $c_{00}$ ,  $c_{10}$ ,  $c_{01}$  and  $c_{11}$  colors



# Texture Sampling – Bilinear Interpolation

- Instead of using the nearest neighbor, we can interpolate the color between texels.
- Let's find the color of the white dot.
- We can define a coordinates system taking any center as reference.
- How do we interpolate ?



$$i = \left\lfloor u - \frac{1}{2} \right\rfloor$$
$$j = \left\lfloor v - \frac{1}{2} \right\rfloor$$

Whole part coordinates

$$s = u - \left( i + \frac{1}{2} \right) \in [0,1]$$
$$t = v - \left( j + \frac{1}{2} \right) \in [0,1]$$

White dot coordinates

Check it !

$(i, j)$  refer to the texel coordinates, not its center !

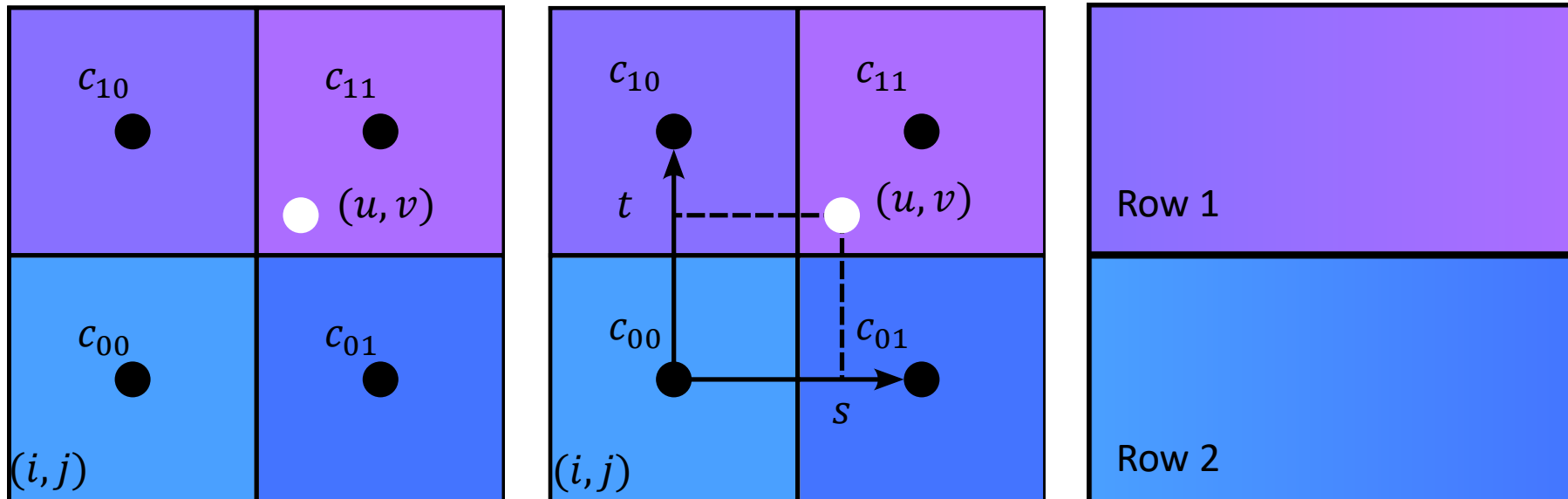
# Texture Sampling – Bilinear Interpolation

- We can define a coordinates system taking any center as reference.
- We can interpolate linearly horizontally between two centers

$$\text{Row 1} = (1 - s)c_{10} + sc_{11}$$

$$\text{Row 2} = (1 - s)c_{00} + sc_{01}$$

- What to do next?

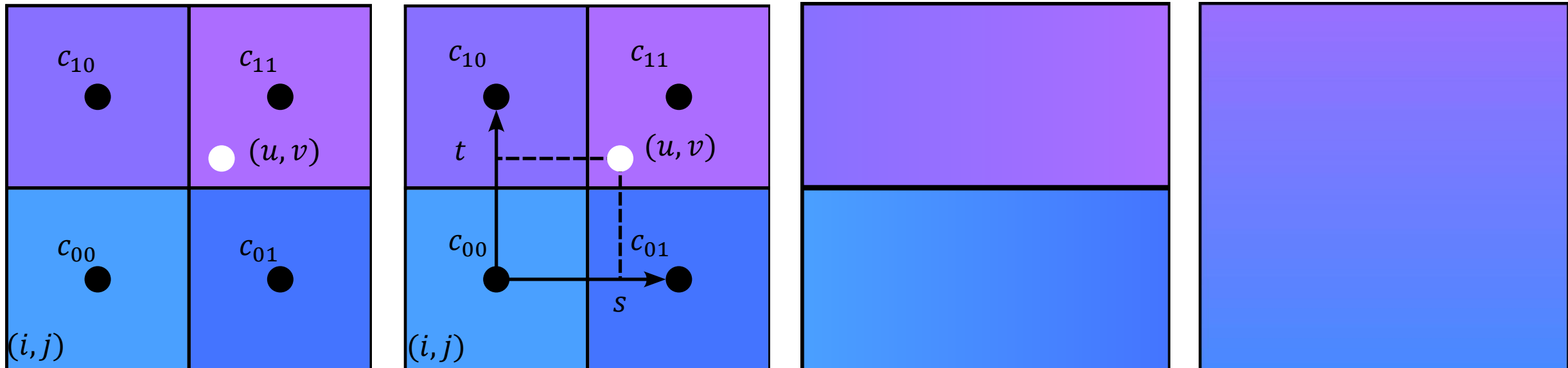


# Texture Sampling – Bilinear Interpolation

- We can interpolate linearly horizontally between two centers
- And linearly vertically

$$(1 - t) \text{ row1} + t \text{ row2}$$
$$(1 - t)[(1 - s)c_{10} + sc_{11}] + t[(1 - s)c_{00} + sc_{01}]$$

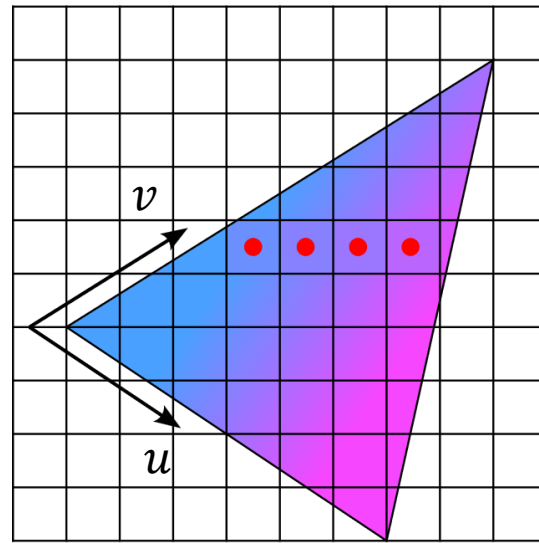
- This is **bilinear interpolation**, our white dot has coordinates  $(s, t)$



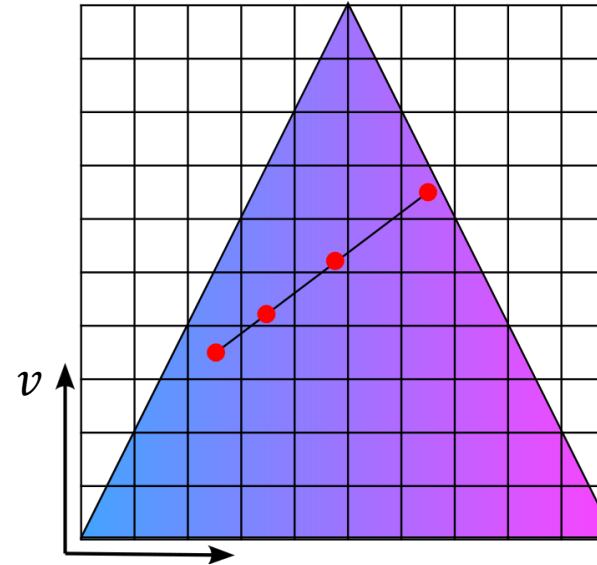
What if we did the columns first and row after ?

# Texture Sampling

- The triangle we sampled was projected close to the screen.
- Bilinear interpolation is enough to find its color



Screen



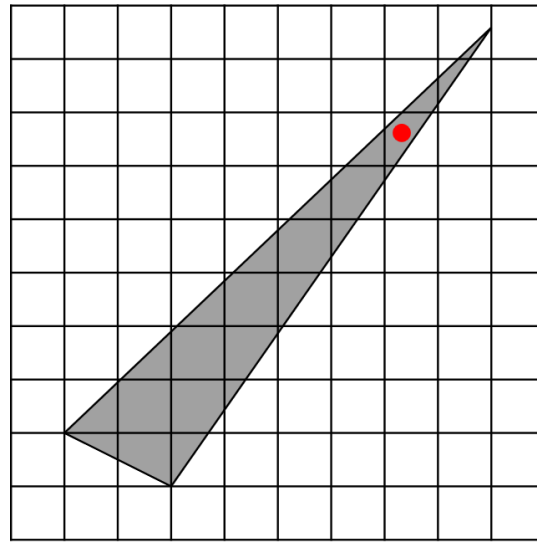
Texture

- But, what about..

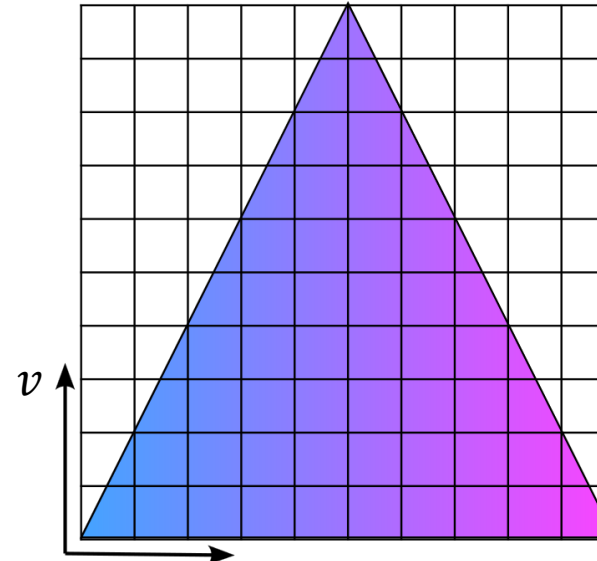
# Texture Sampling

- The triangle we sampled was projected close to the screen.
- We can use bilinear interpolation to find its color

Triangle stretched



Screen

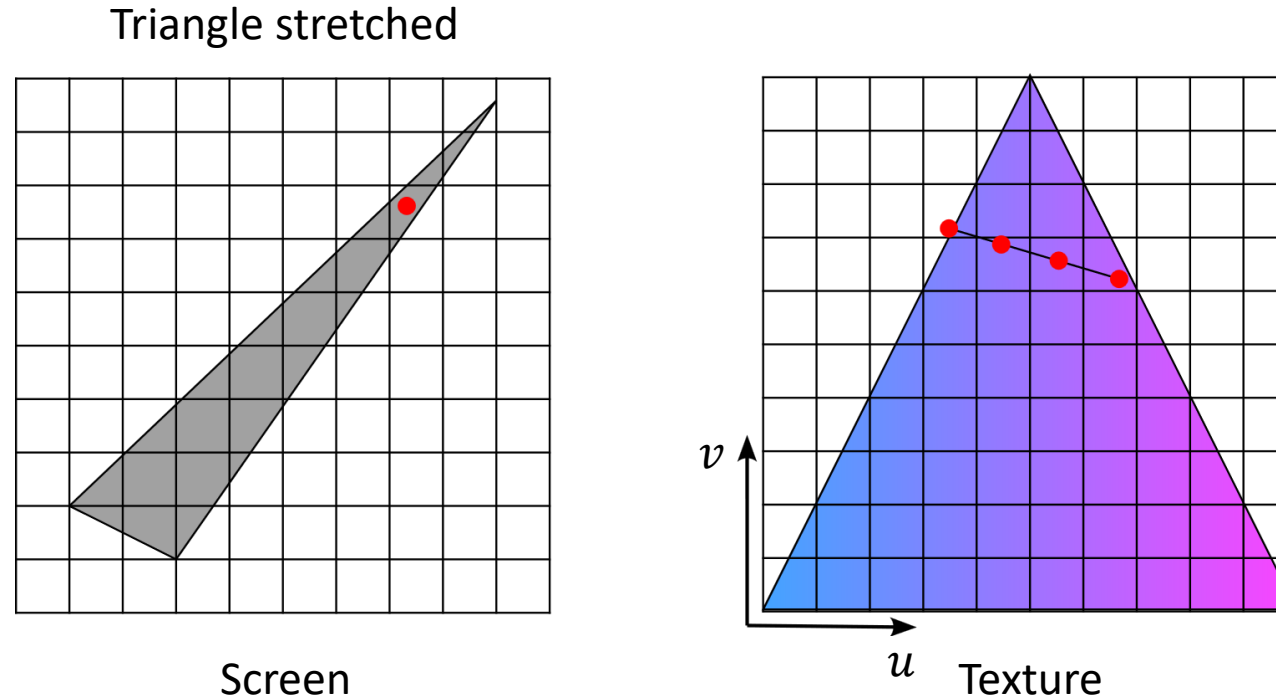


Texture

- What color should we use for the red dot ?

# Texture Sampling

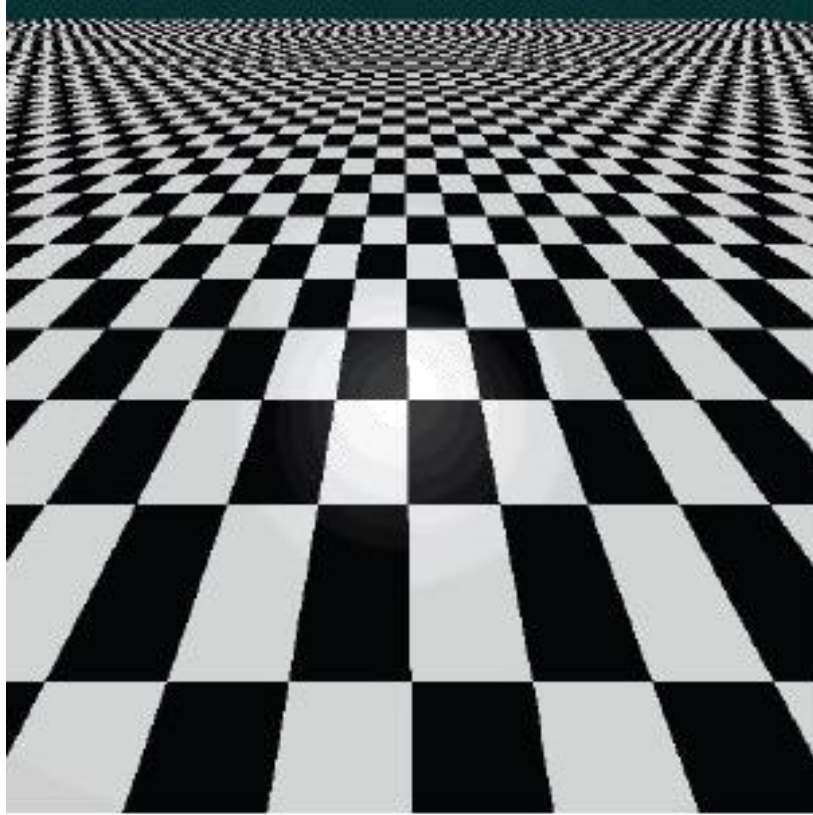
- What color should we use for the red dot ?



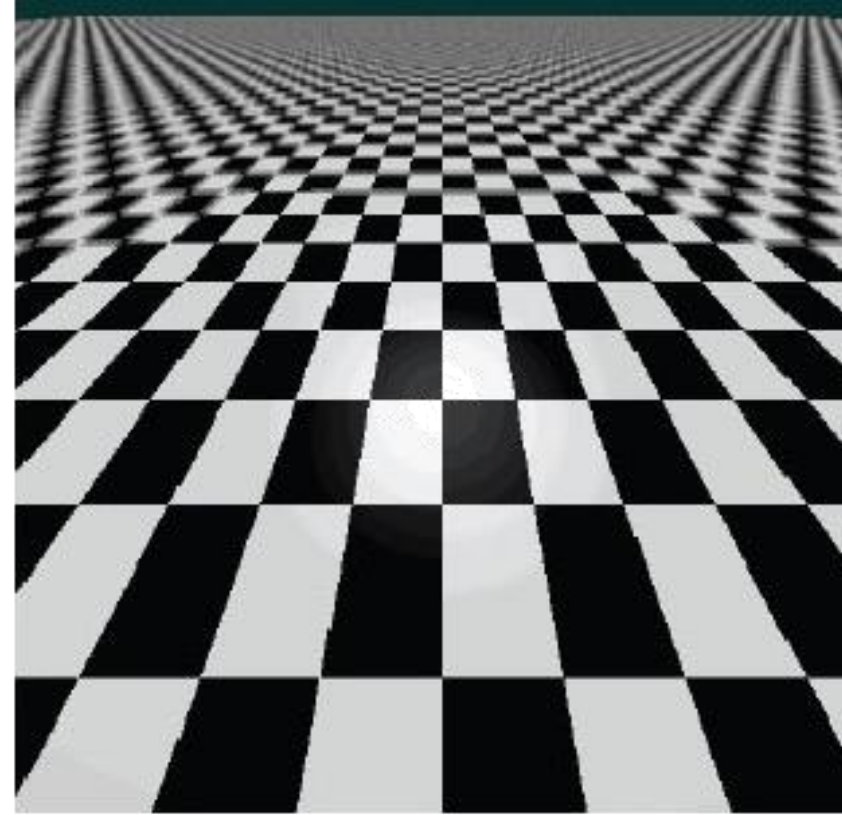
- Which color should we use that is consistent with the neighbors of the pixel on the screen ?

# Texture Sampling

- Example of what happens if we do not take the correct color



Bad sampling

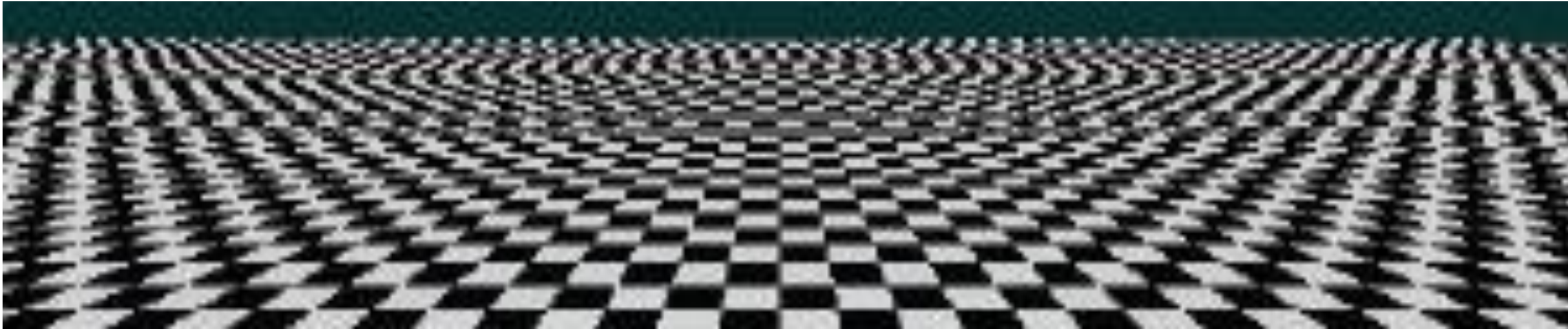


Good sampling

- What is this effect called ?

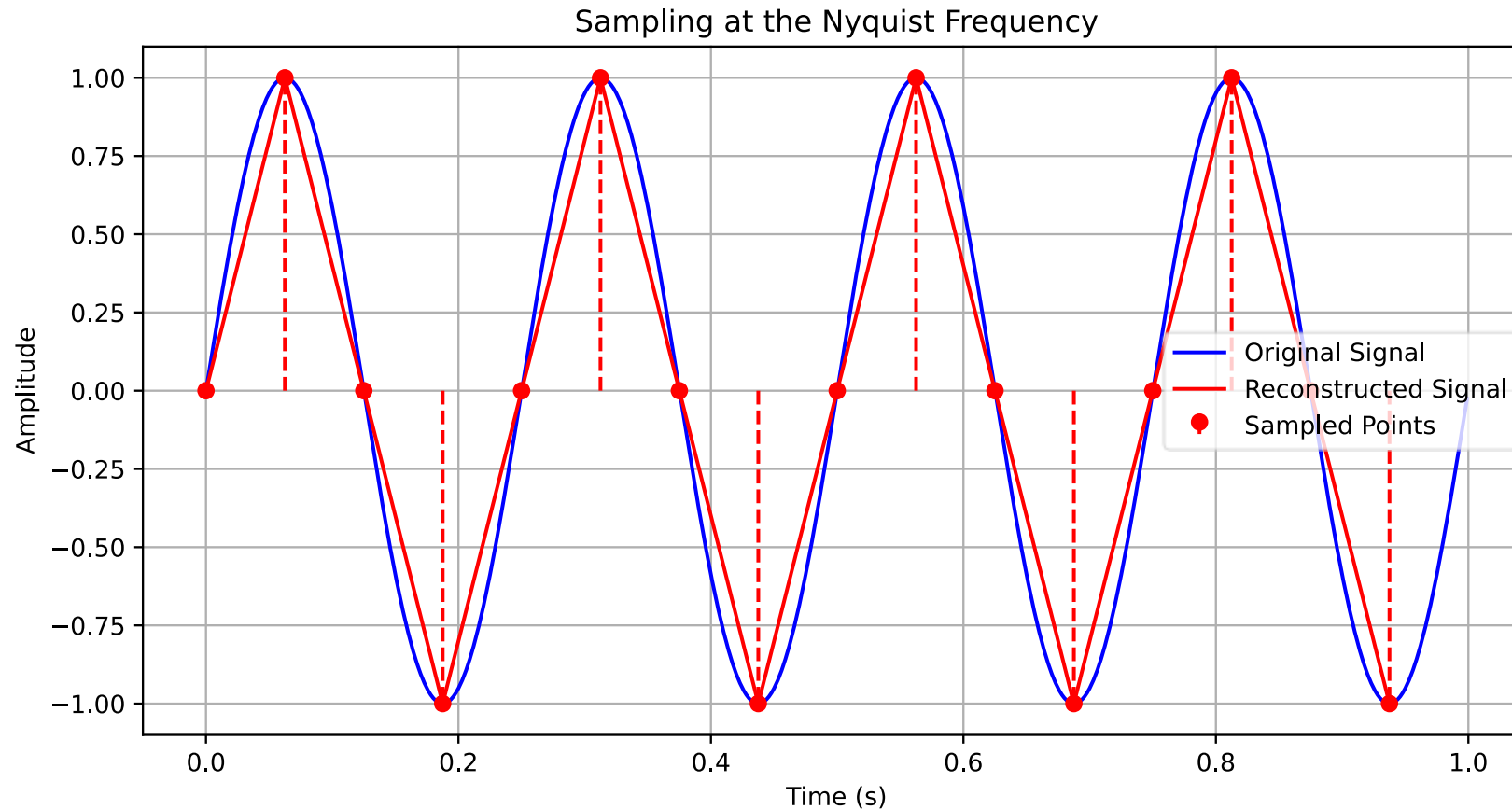
# Aliasing

- What is this effect called ?
- **Aliasing**
- Texture aliasing occurs because a single pixel on the screen covers many pixels of the texture
- Let's review what aliasing of a signal means.



# Aliasing

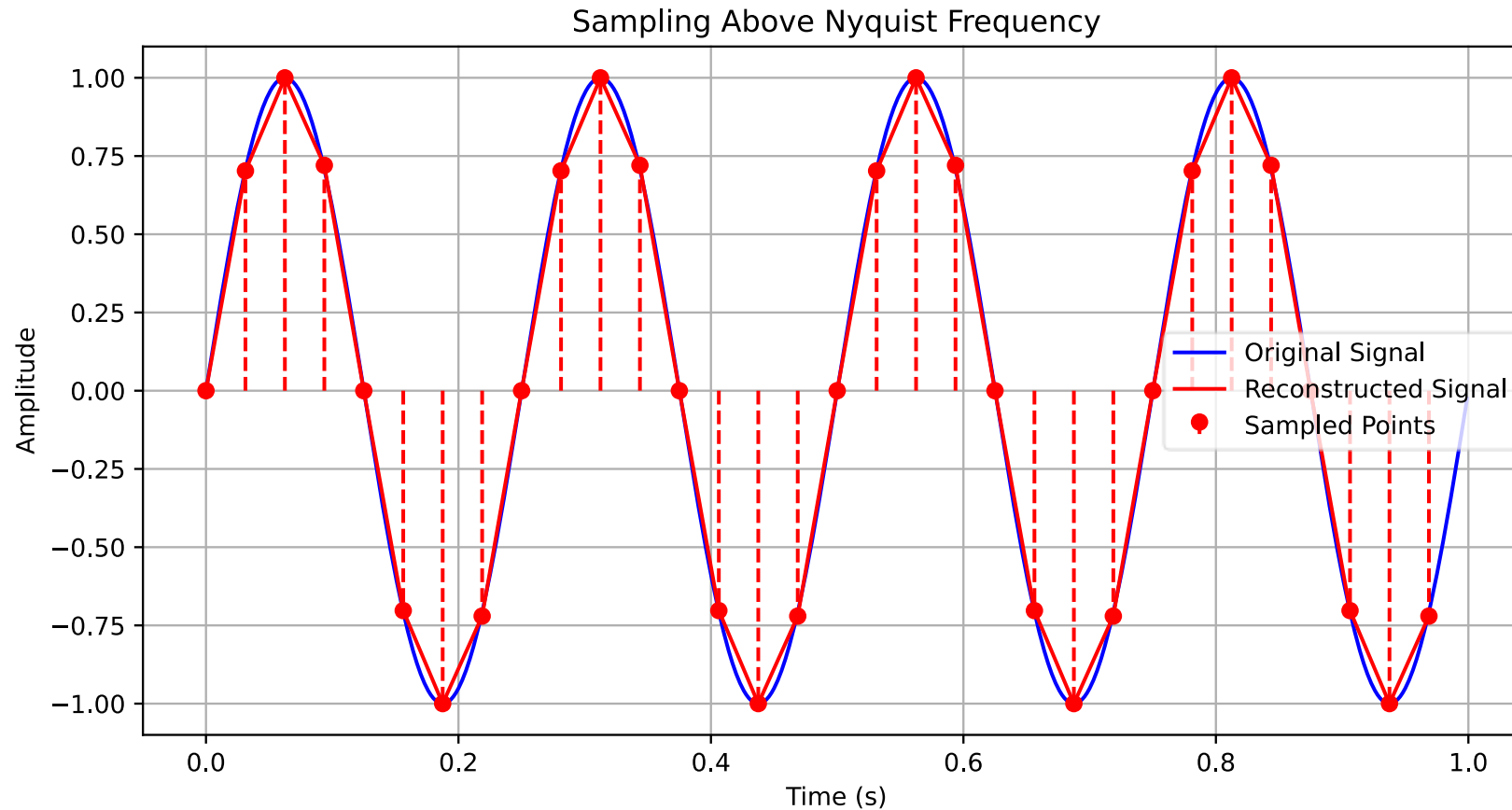
- Sampling a signal **at** the Nyquist frequency



- We can easily reconstruct our signal given the sampling points

# Aliasing

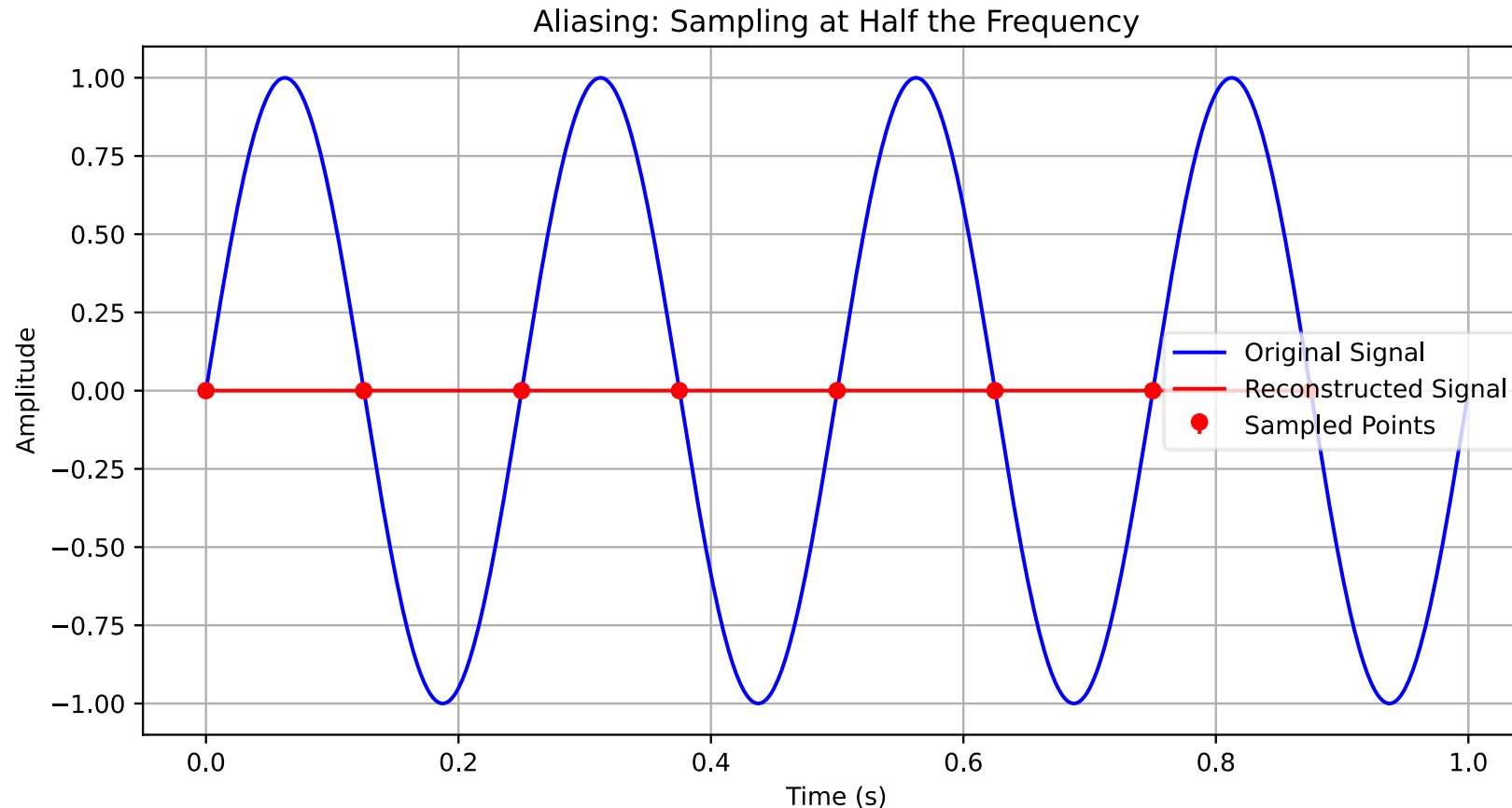
- Sampling a signal **above** the Nyquist frequency.



- The signal is overdetermined! We can reconstruct it easily.

# Aliasing

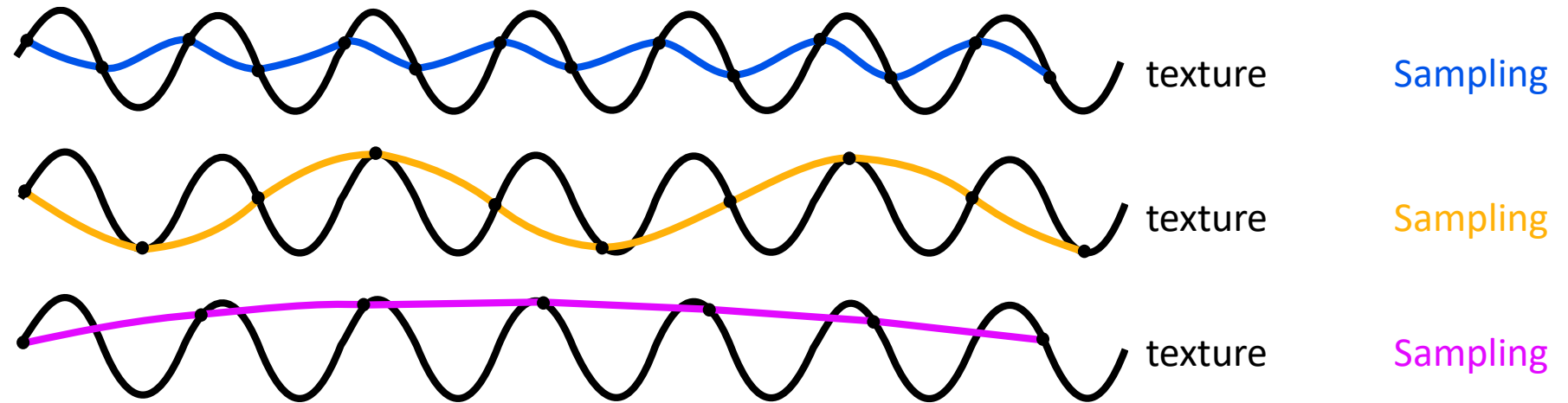
- Sampling a signal **below** the Nyquist frequency



- The signal has not enough samples to be reconstructed !
- This is what happens if we do not take averages

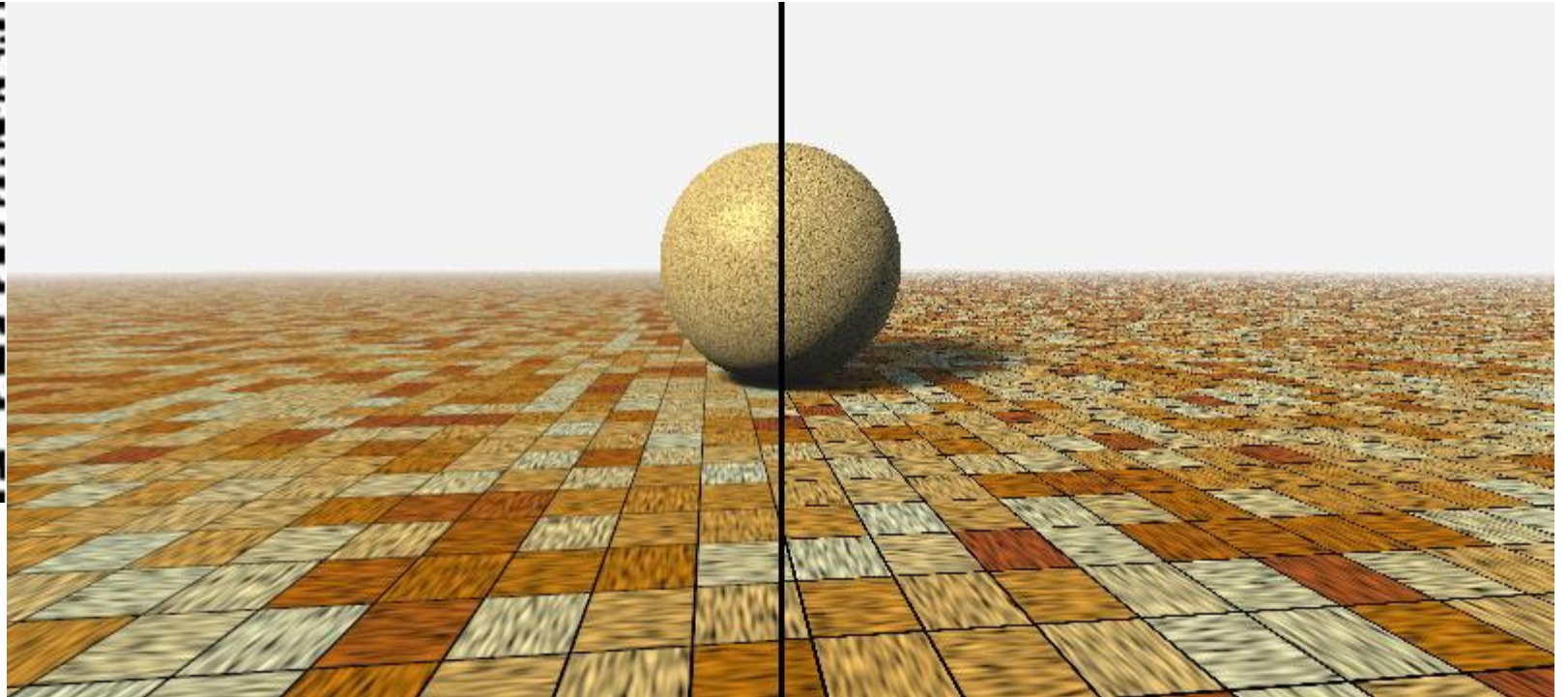
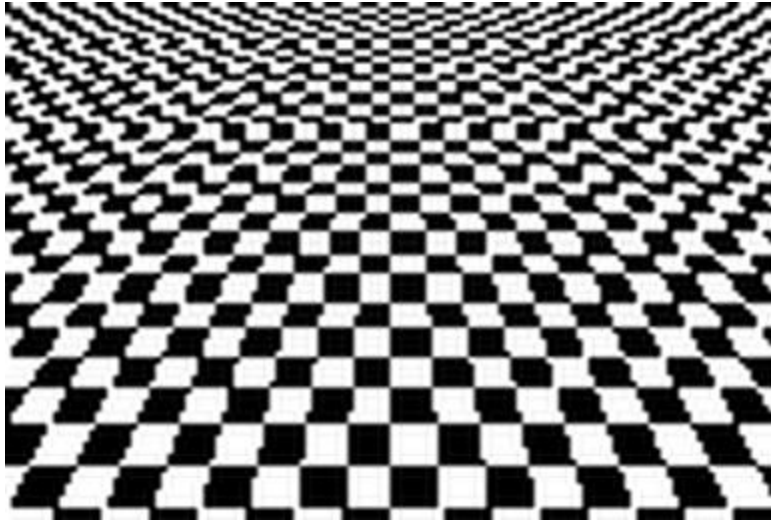
# Minification – Aliasing

- Texture aliasing occurs because a single pixel on the screen covers many pixels of the texture
- Two near pixels in screen coordinates correspond to sampling the texture at locations under the Nyquist frequency in the texture.

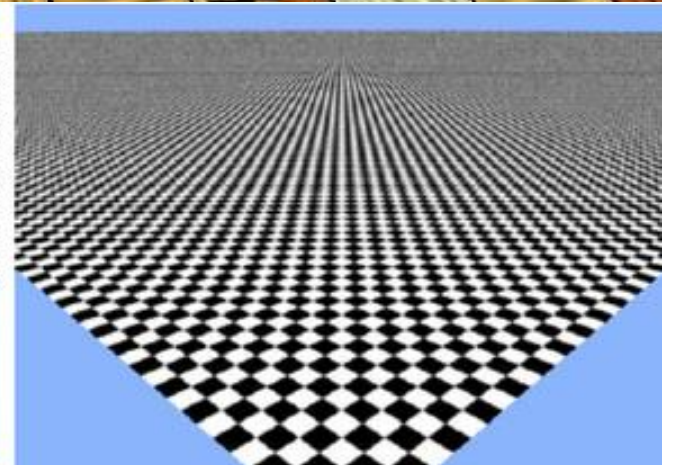
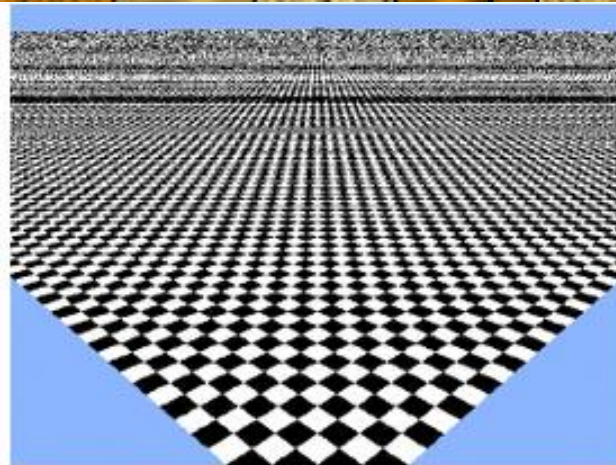
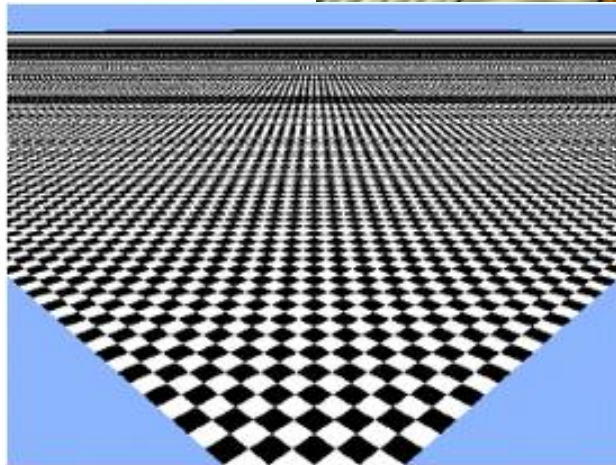


- Bilinear interpolation do not solve the problem, our samples get smashed together, creating patterns of incorrect adjacent colors.

# Aliasing effects



- Bilinear not enough
- What can we do?

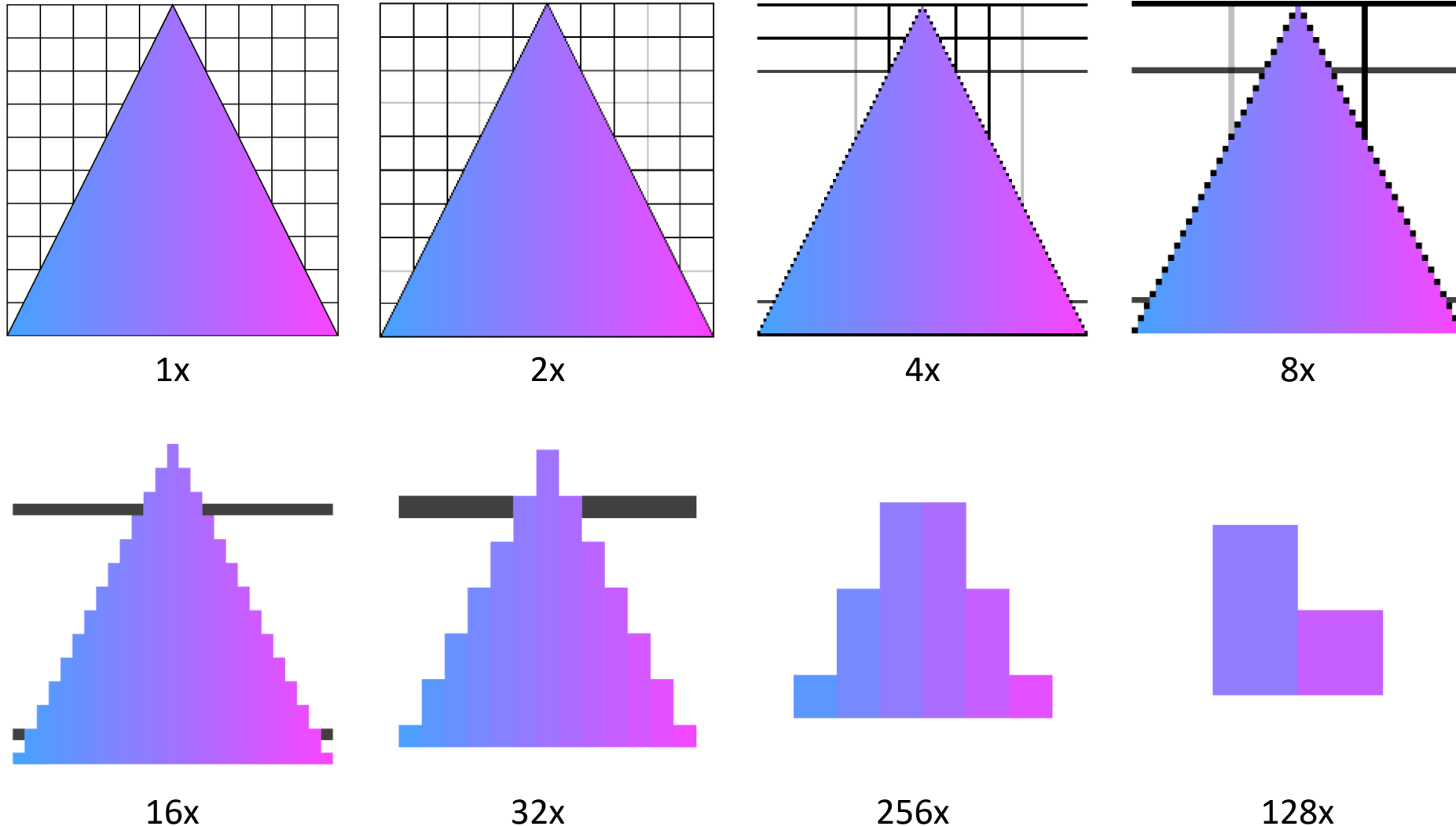


# Minification – Texture prefiltering

- Texture aliasing occurs because a single pixel on the screen covers many pixels of the texture
- Two near pixels in screen coordinates correspond to sampling the texture at locations under the Nyquist frequency in the texture.
- Bilinear interpolation do not solve the problem, our samples get smashed together, creating patterns of incorrect adjacent colors.
- What can we do instead ?
- As in signal theory, we can prefilter the image (=downsample it) once and look up the averages (many times) at run-time.

# Minification – Prefiltering

- We create at least 6 prefiltered textures, usually dividing by 2 the resolution

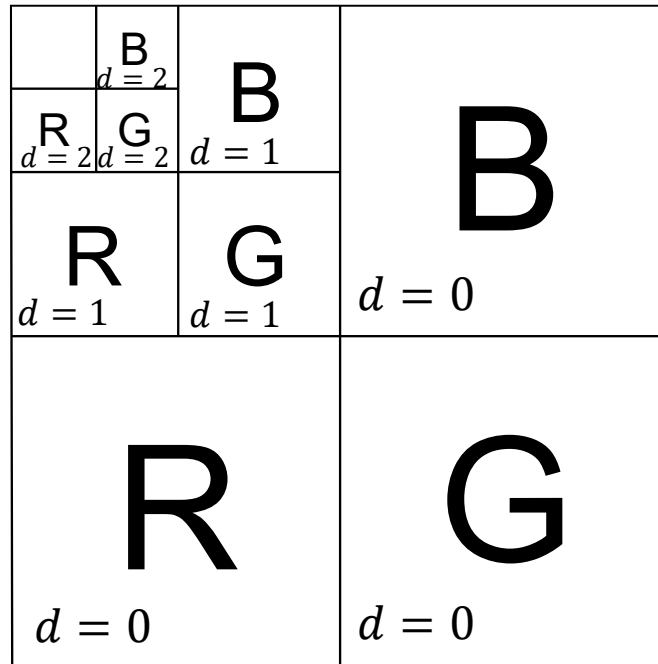


# Mipmapping

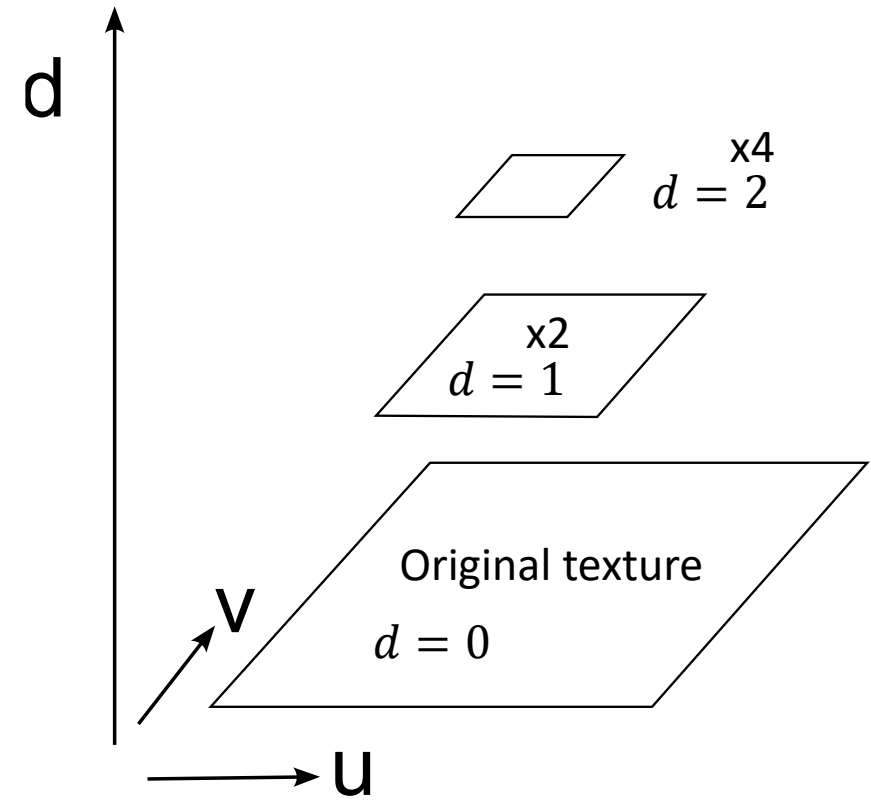
- This process of generating downscaled versions of our texture is called **mipmapping**
- We want downscaled versions of our image at *every possible lower scale*
- Texels at higher levels store average of texture over a region of texture space (downsampled)
- When we sample the texture, based on the distance, we look up a single pixel from the mipmap of appropriate size.

# Mipmapping

- When you load a texture in OpenGL
- It automatically generates mipmaps
- They are stored in a special layout such as this:



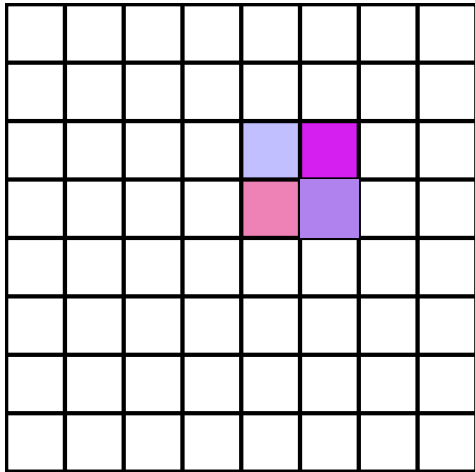
- $d$  represents the mipmap level.



# Mipmapping

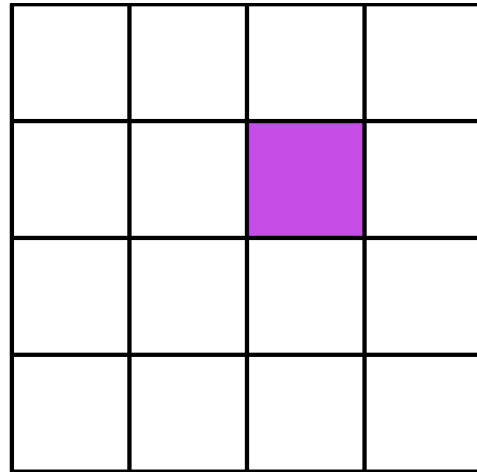
- Another view of mipmapping
- The texture is downsampled by 2 at every step, there are 4 levels.

$d = 0$



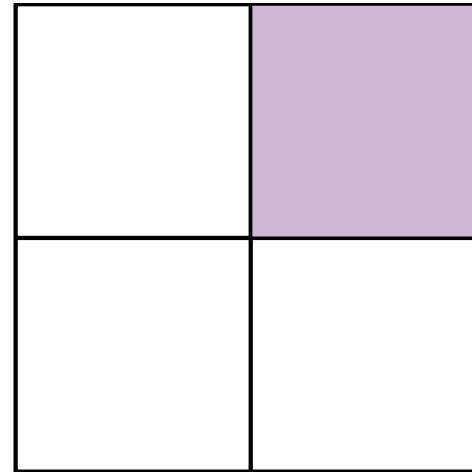
$2^3 \times 2^3$

$d = 1$



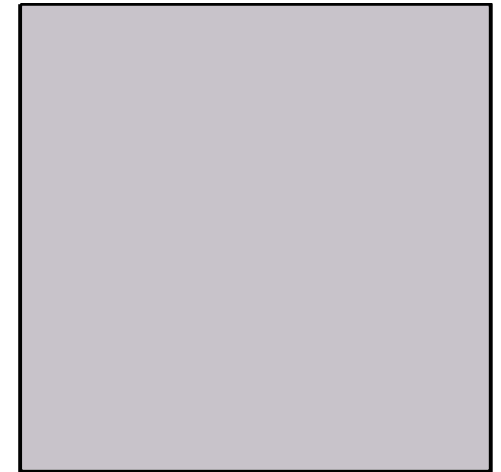
$2^2 \times 2^2$

$d = 2$



$2 \times 2$

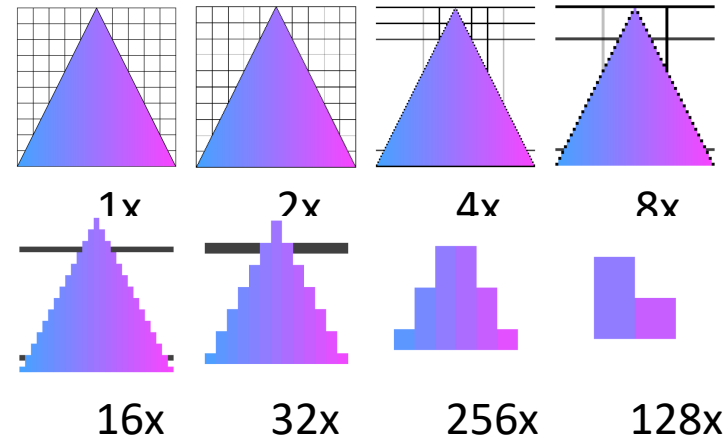
$d = 3$



$1 \times 1$

# Mipmapping

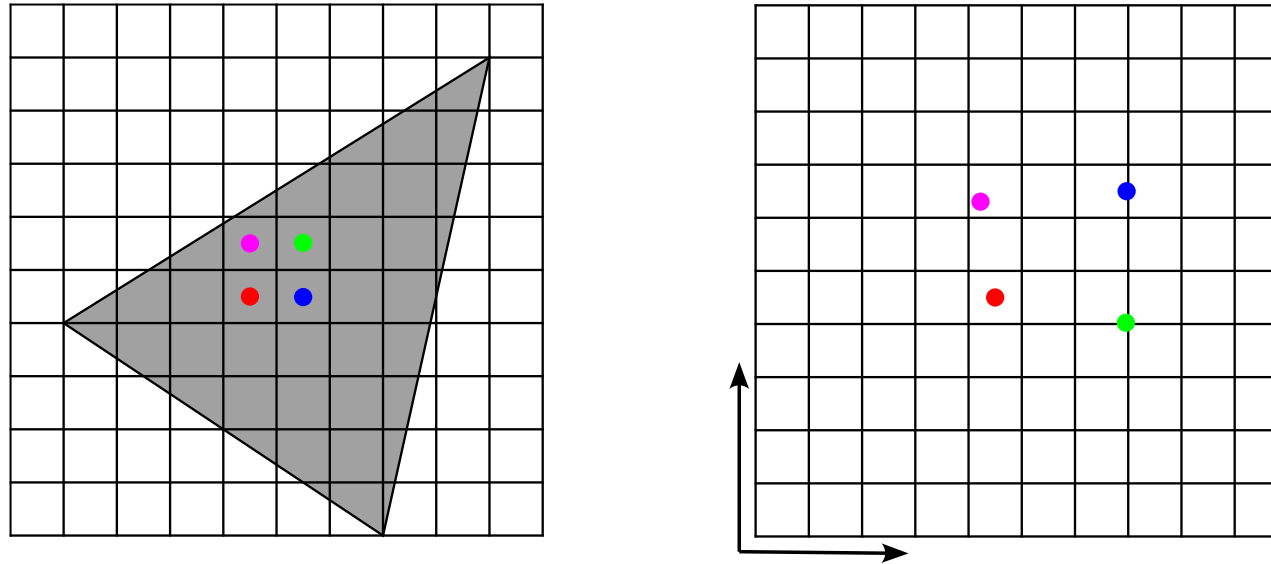
- We have our texture and several subsampled versions of it.



- We know that if the object is far away, we should use a downscaled version to avoid aliasing. (increasing  $d$ )
- But, how can we choose the level of the mipmap ?

# Choosing Mipmap Level

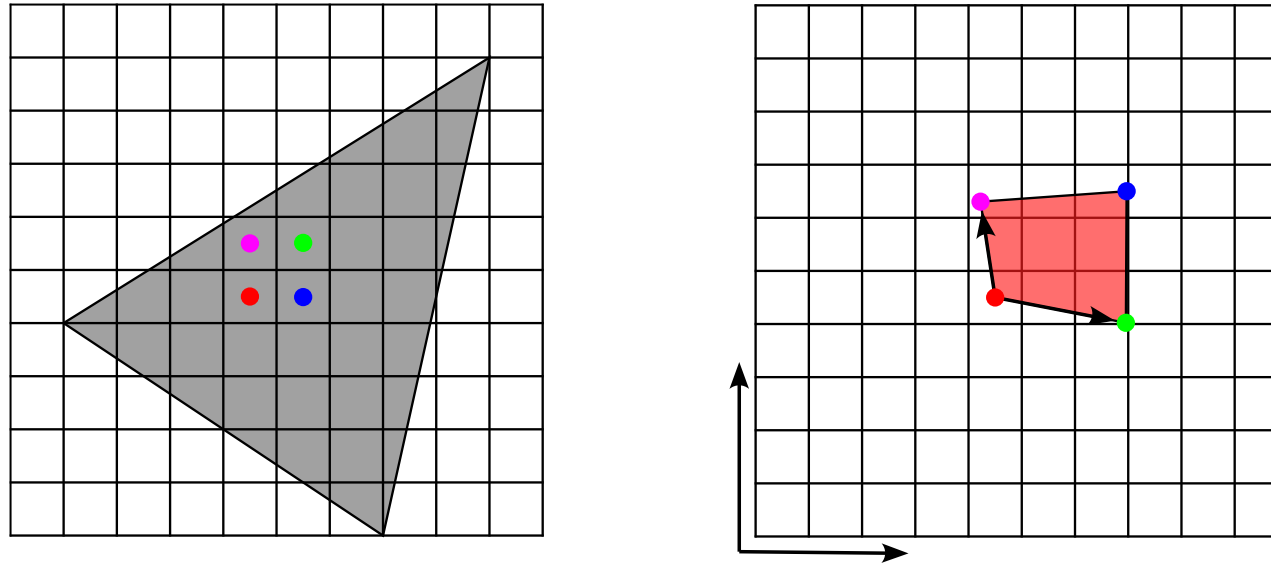
- Suppose we look at 4 neighboring pixels in screen coordinates
- And we find the corresponding texels like this:



- How their spread inform us on the mipmap level?

# Choosing Mipmap Level

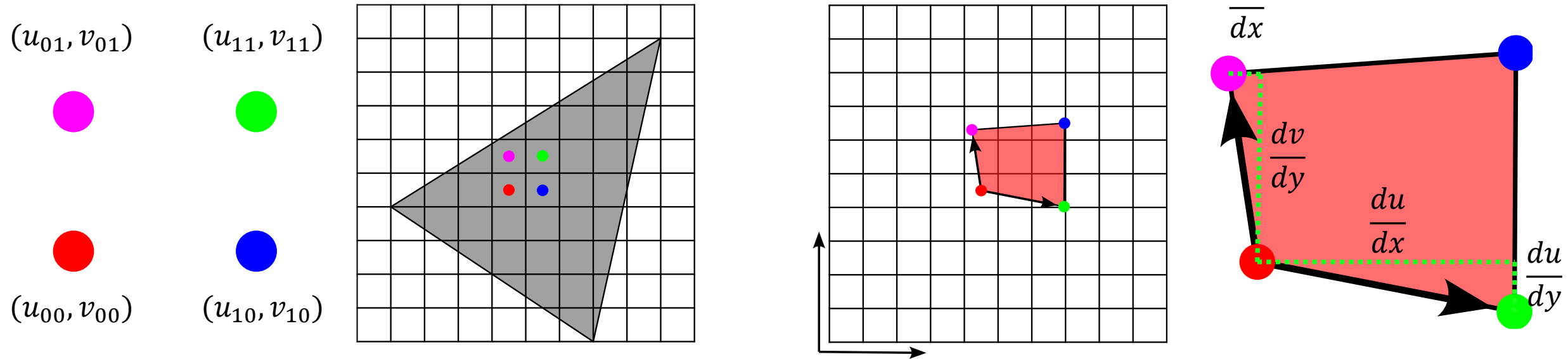
- How their spread inform us on the mipmap level?:



- Yes, the more they are spread apart, the higher the level  $d$
- We want to find the area of that box, how ?

# Choosing Mipmap Level

- Yes, the more they are spread apart, the higher the level  $d$
- We want to find the area of that box, how ?

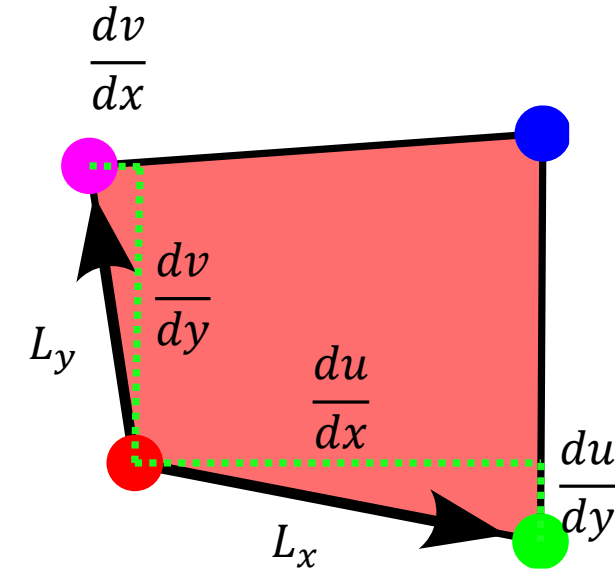
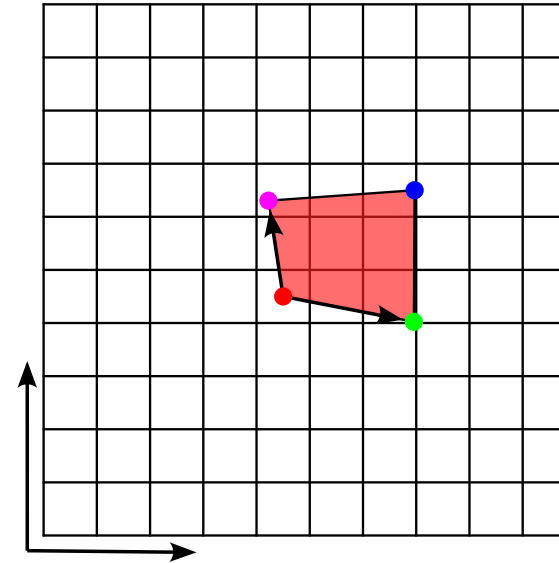
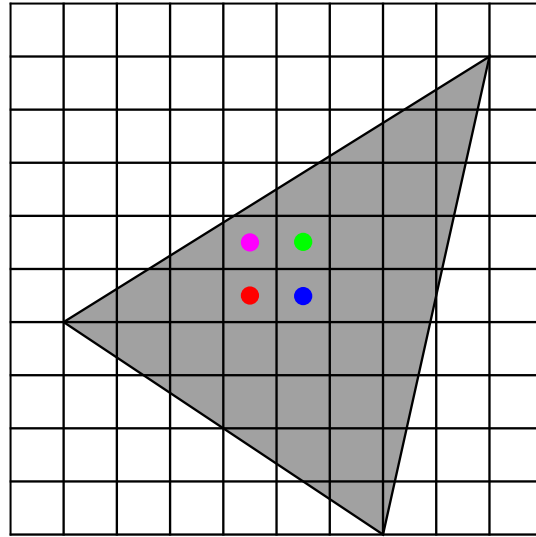


- We can at least measure the length of the vectors in texture space

$$\begin{aligned} \frac{du}{dx} &= u_{10} - u_{00}, & \frac{du}{dy} &= u_{01} - u_{00} \\ \frac{dv}{dx} &= v_{10} - v_{00}, & \frac{dv}{dy} &= v_{01} - v_{00} \end{aligned}$$

# Choosing Mipmap Level

- Yes, the more they are spread apart, the higher the level  $d$
- We want to find the area of that box, how ?



- The length of the vector is given by:

$$L_x^2 = \left(\frac{du}{dx}\right)^2 + \left(\frac{dv}{dx}\right)^2,$$

$$L_y^2 = \left(\frac{du}{dy}\right)^2 + \left(\frac{dv}{dy}\right)^2$$

Why ?

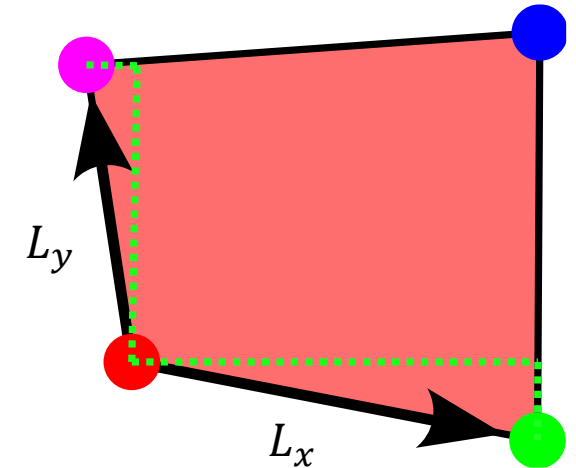
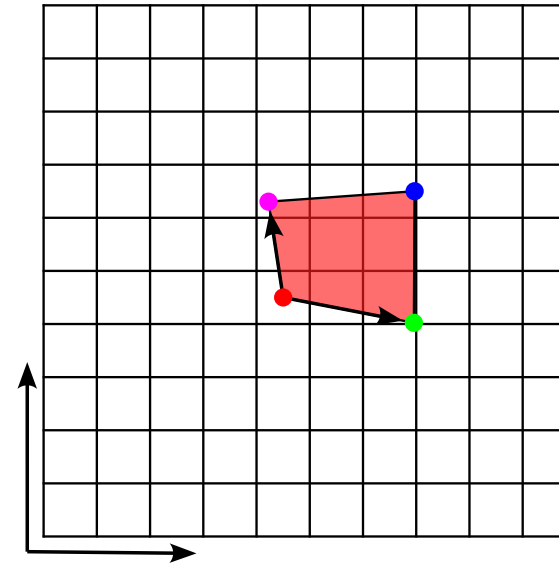
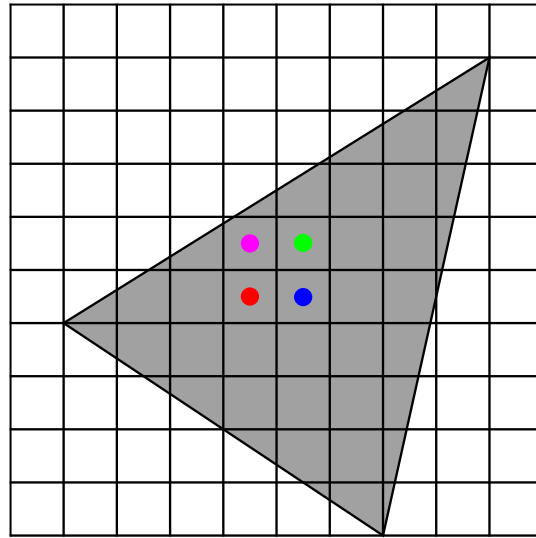
# Choosing Mipmap Level

- Yes, the more they are spread apart, the higher the level  $d$
- We want to find the area of that box, how ?

$$\frac{du}{dx} = u_{10} - u_{00}, \quad \frac{du}{dy} = u_{01} - u_{00}$$

$$\frac{dv}{dx} = v_{10} - v_{00}, \quad \frac{dv}{dy} = v_{01} - v_{00}$$

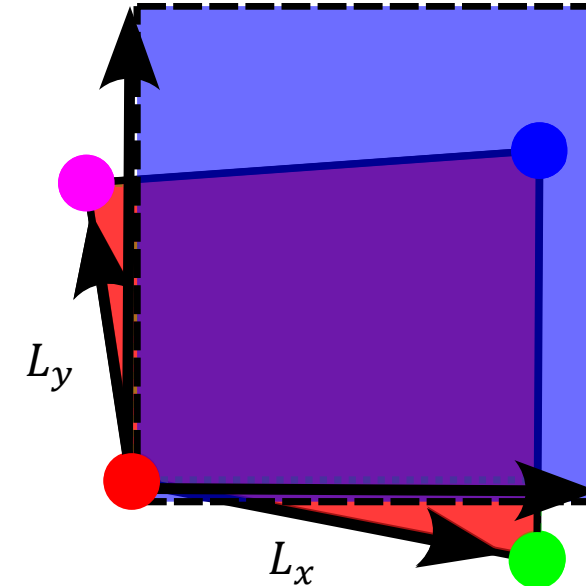
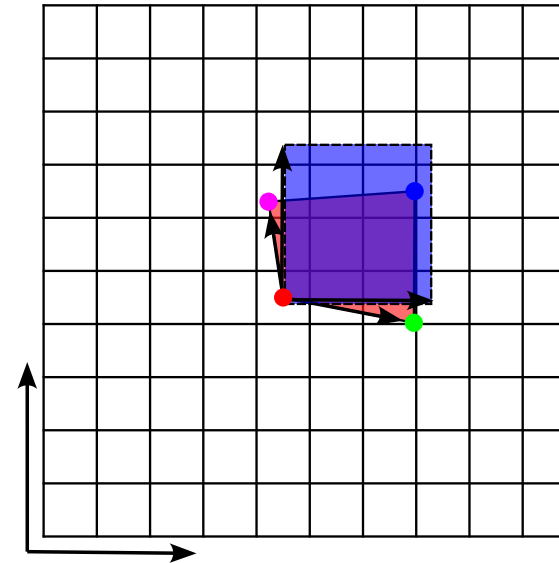
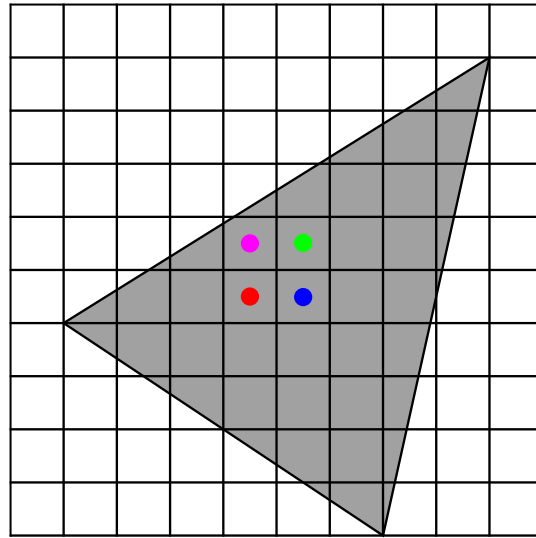
$$L_x^2 = \left(\frac{du}{dx}\right)^2 + \left(\frac{dv}{dx}\right)^2, \quad L_y^2 = \left(\frac{du}{dy}\right)^2 + \left(\frac{dv}{dy}\right)^2$$



- But, even with the vectors, how to compute the area of that box ?

# Choosing Mipmap Level

- Yes, the more they are spread apart, the higher the level  $d$
- We want to find the area of that box, how ?

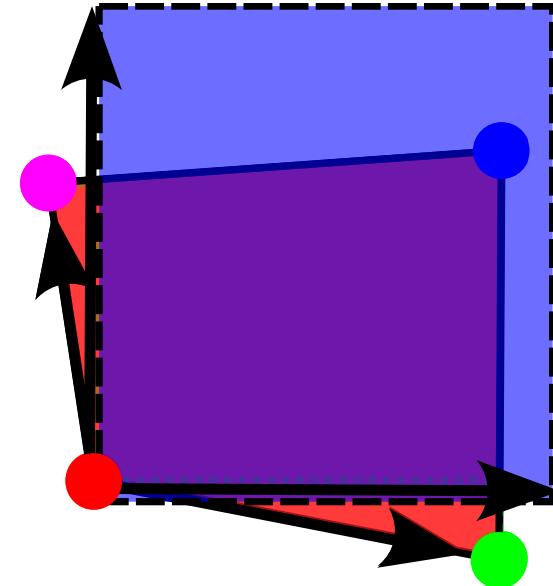
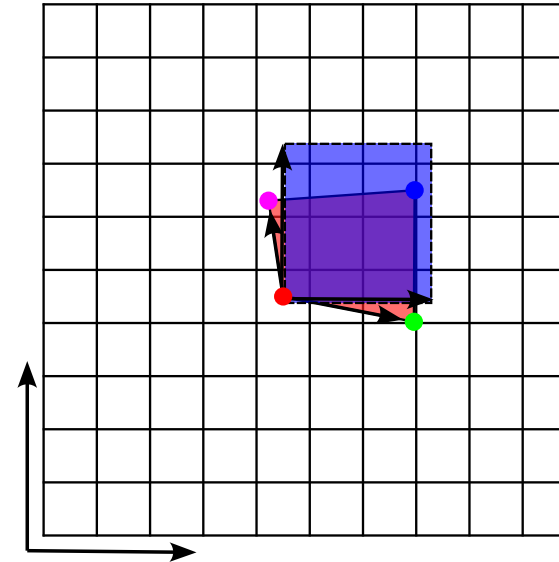
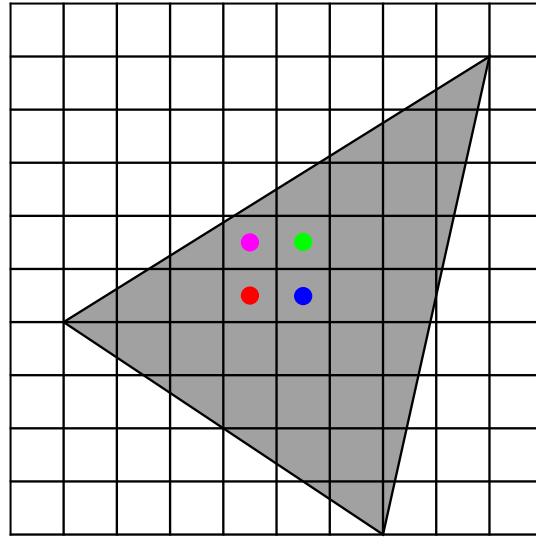


- We don't care about the exact area !
- We just want an order of magnitude of the largest side !

$$L = \sqrt{\max(L_x^2, L_y^2)}$$

# Choosing Mipmap Level

- Yes, the more they are spread apart, the higher the level  $d$
- We want to find the area of that box, how ?

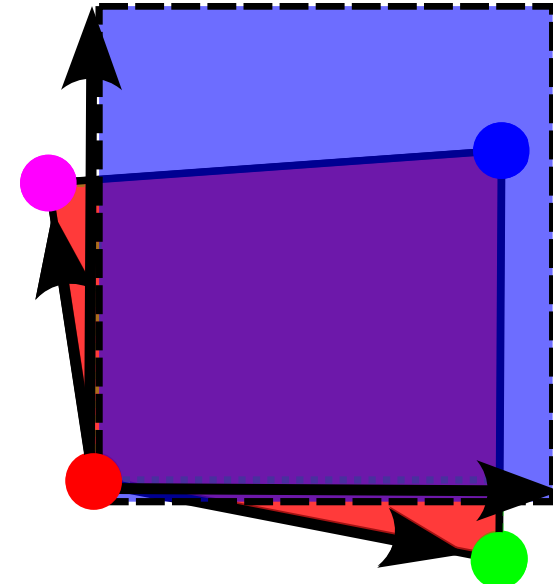
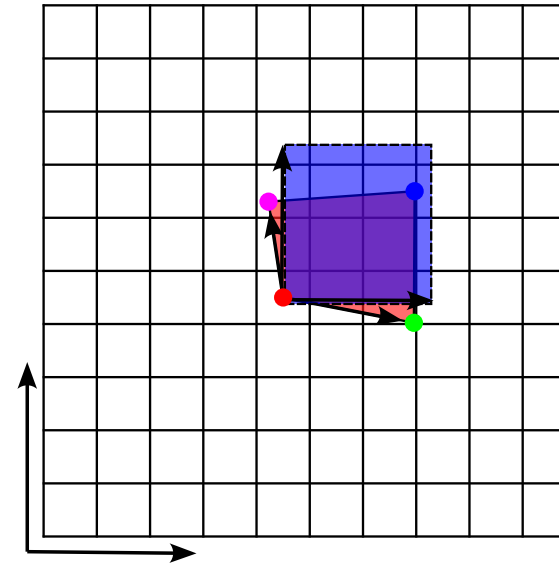
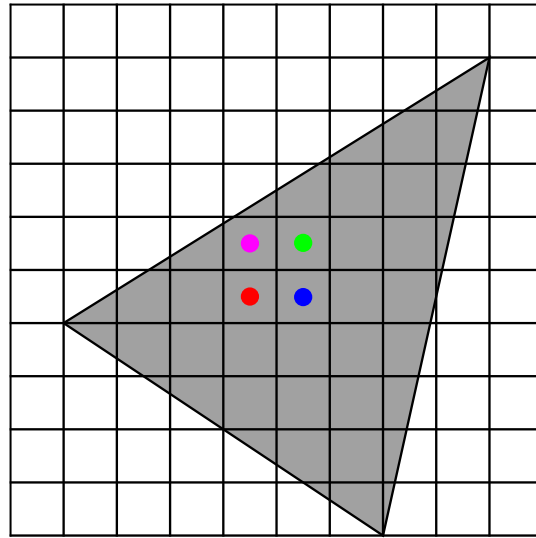


$$L = \sqrt{\max(L_x^2, L_y^2)}$$

- Now that we have the area of this square
- How to find the level  $d$  of the mipmap ?

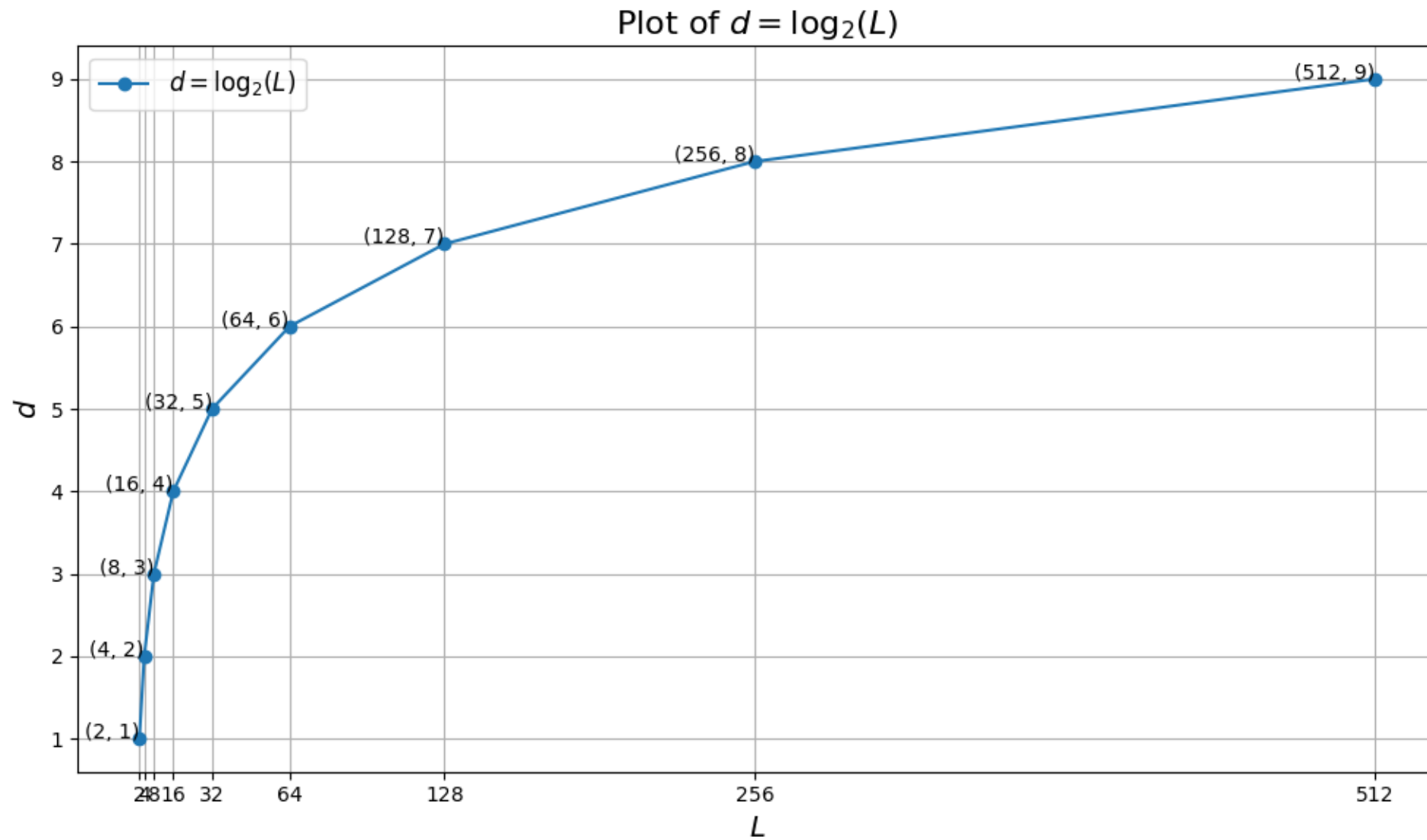
# Choosing Mipmap Level

- Now that we have the area of this square
- How to find the level  $d$  of the mipmap ?



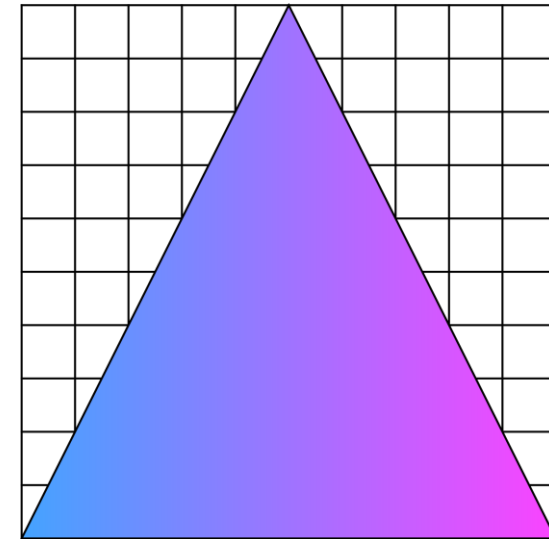
$$L = \sqrt{\max(L_x^2, L_y^2)}$$

- We always divided the texture by 2
- It seems natural to have  $L = 2^d$
- i.e., for every increase in  $d$ , the length is  $d = \log_2 L$



# Problems With Mipmapping

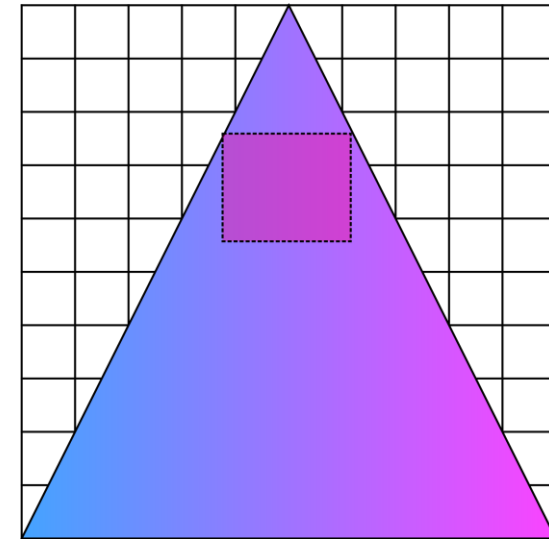
- Now suppose you have a very tangential view of the floor
- This triangle is extremely stretched in the far distance



- How does our box approximation look like ?

# Problems With Mipmapping

- How does our box approximation look like ?

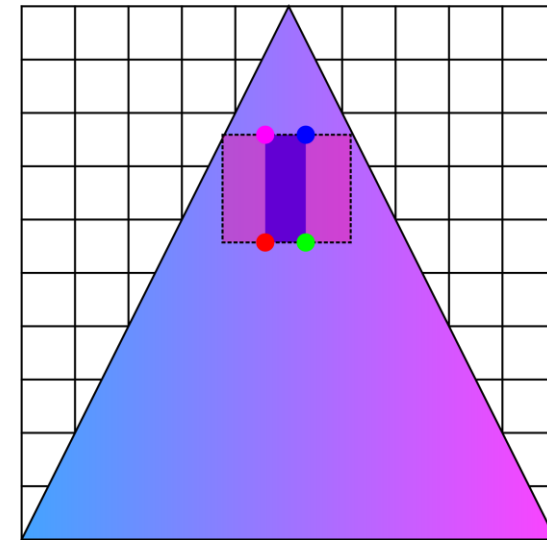


- In far distance we average over almost the whole tip of the triangle
- But how was the true deformation ?

# Problems With Mipmapping

- So, in far distance we average over almost the whole tip of the triangle
- But how was the true deformation ?

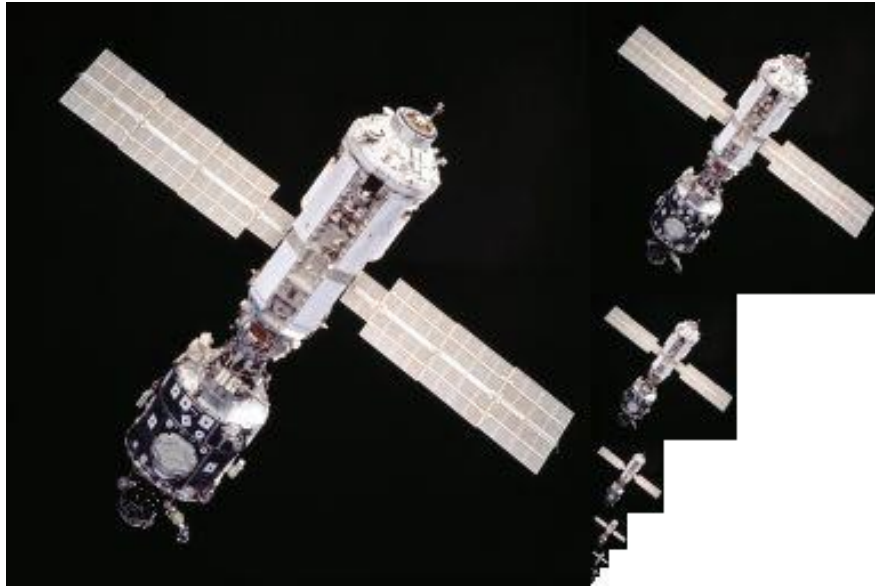
$$L = \sqrt{\max(L_x^2, L_y^2)}$$
$$d = \log_2 L$$



- Oh, the  $v$  deformation was way higher than the  $u$  deformation..
- Our box approximation averages over  $u$  way too much, what to do?

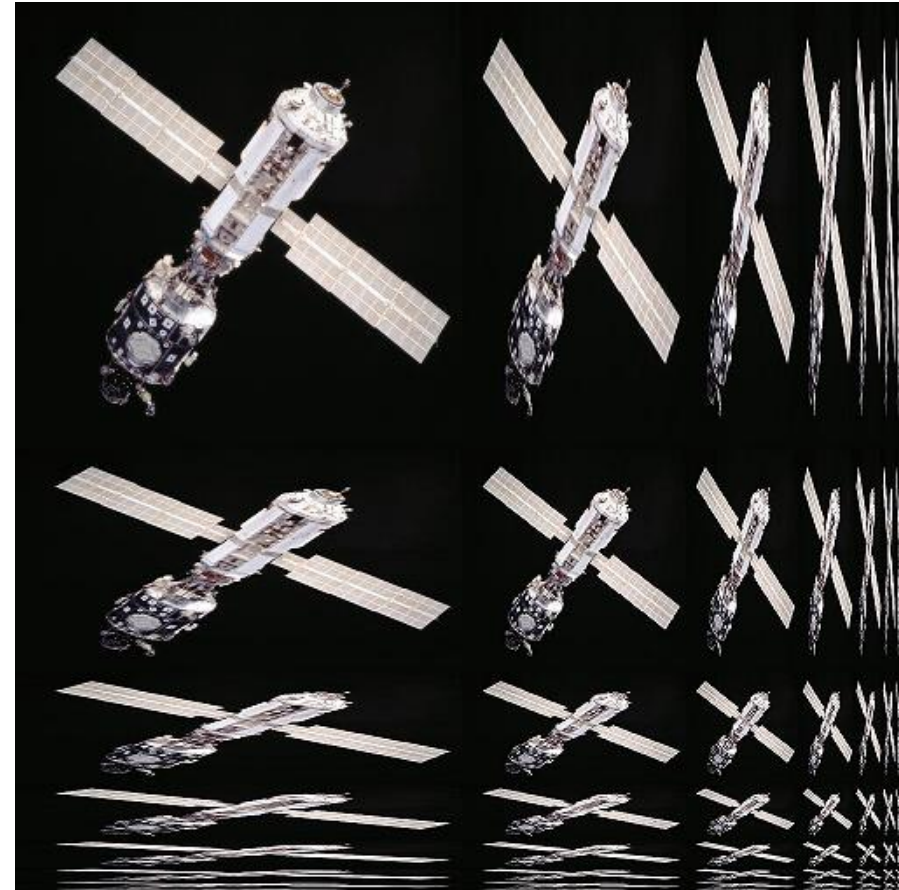
# Problems With Mipmapping – Anisotropic

- Oh, the  $v$  deformation was way higher than the  $u$  deformation..
- Our box approximation averages over  $u$  way too much, what to do?
- We can store anisotropic mipmaps !



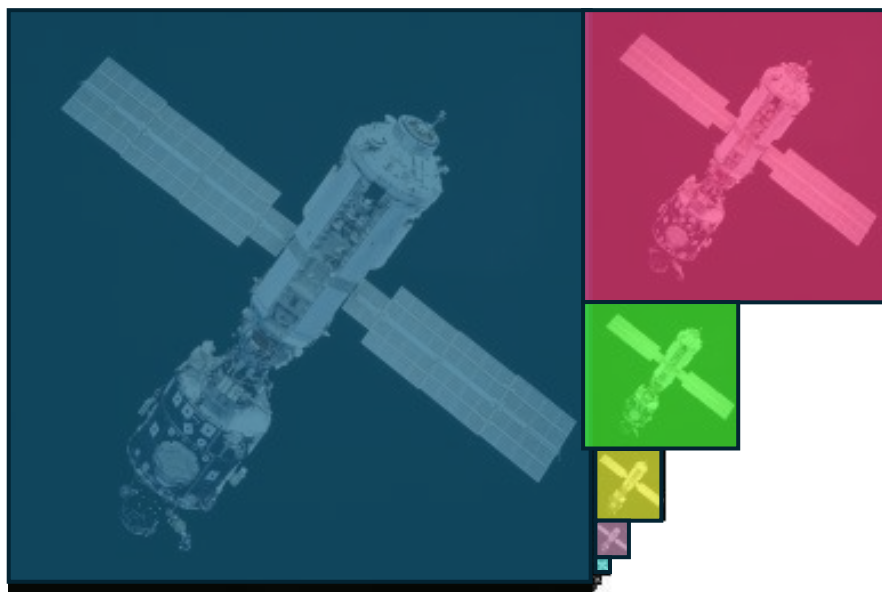
Classic Mipmap

Anisotropic Mipmap



# Problems With Mipmapping – Anisotropic

- Oh, the  $v$  deformation was way higher than the  $u$  deformation..
- Our box approximation averages over  $u$  way too much, what to do?
- We can store anisotropic mipmaps !



Classic Mipmap

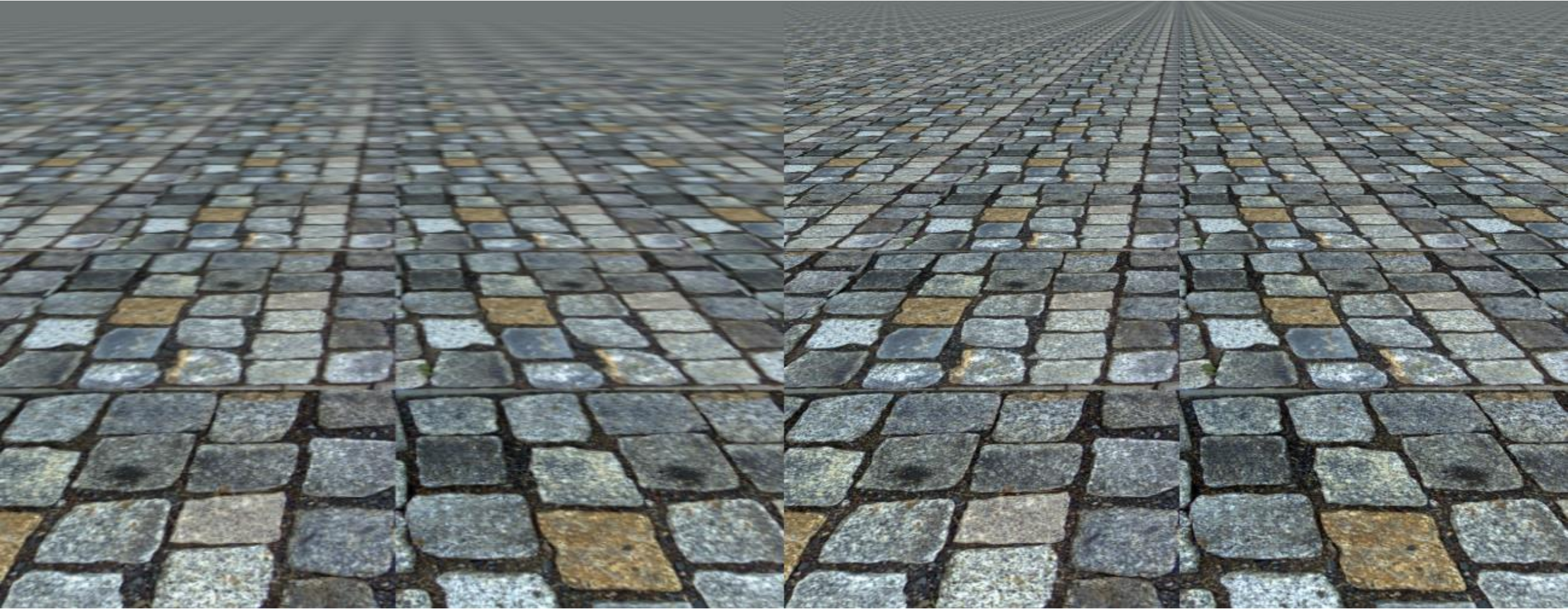
- On the diagonal, classic mipmap

Anisotropic Mipmap





# Effects Of Anisotropic Mip Mapping

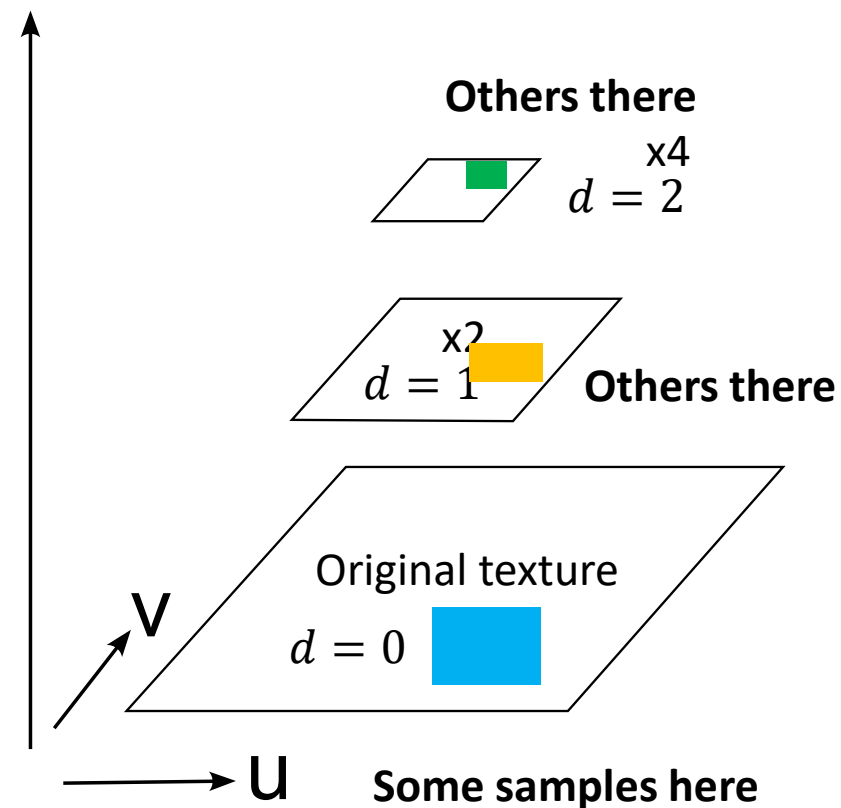
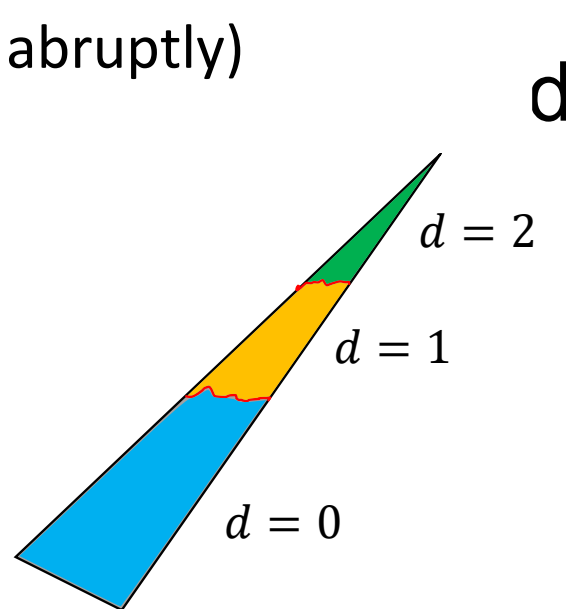


Classic Mip Map

Anisotropic Mip Map

# Problems With Mipmapping

- Now we have several mip maps that we can sample to render the texture
- But..
- If we use the nearest  $d$  level, we get artifacts where the level jumps
  - (textures goes from sharp to blurry abruptly)



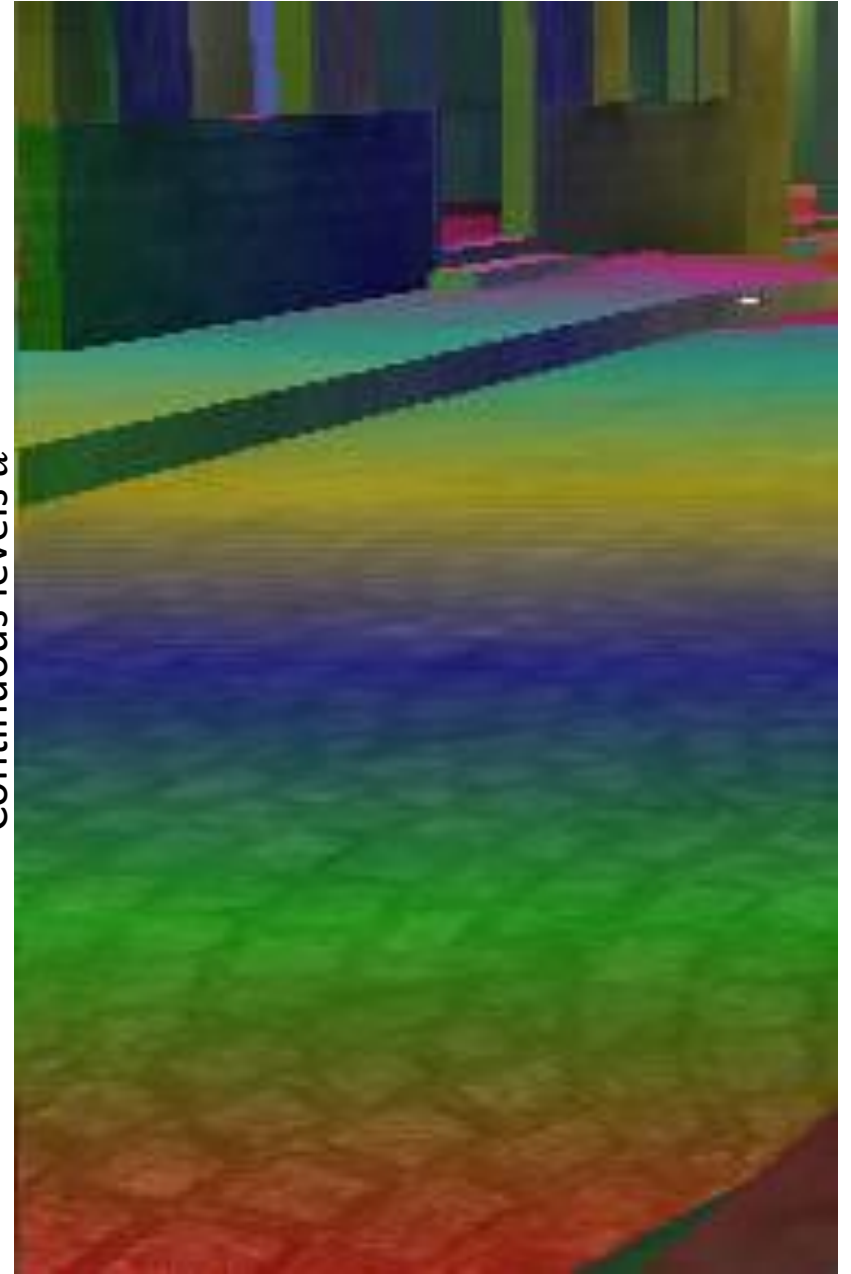
- What to do instead ?

# Discrete vs continuous mipmap levels

Discrete levels  $d$

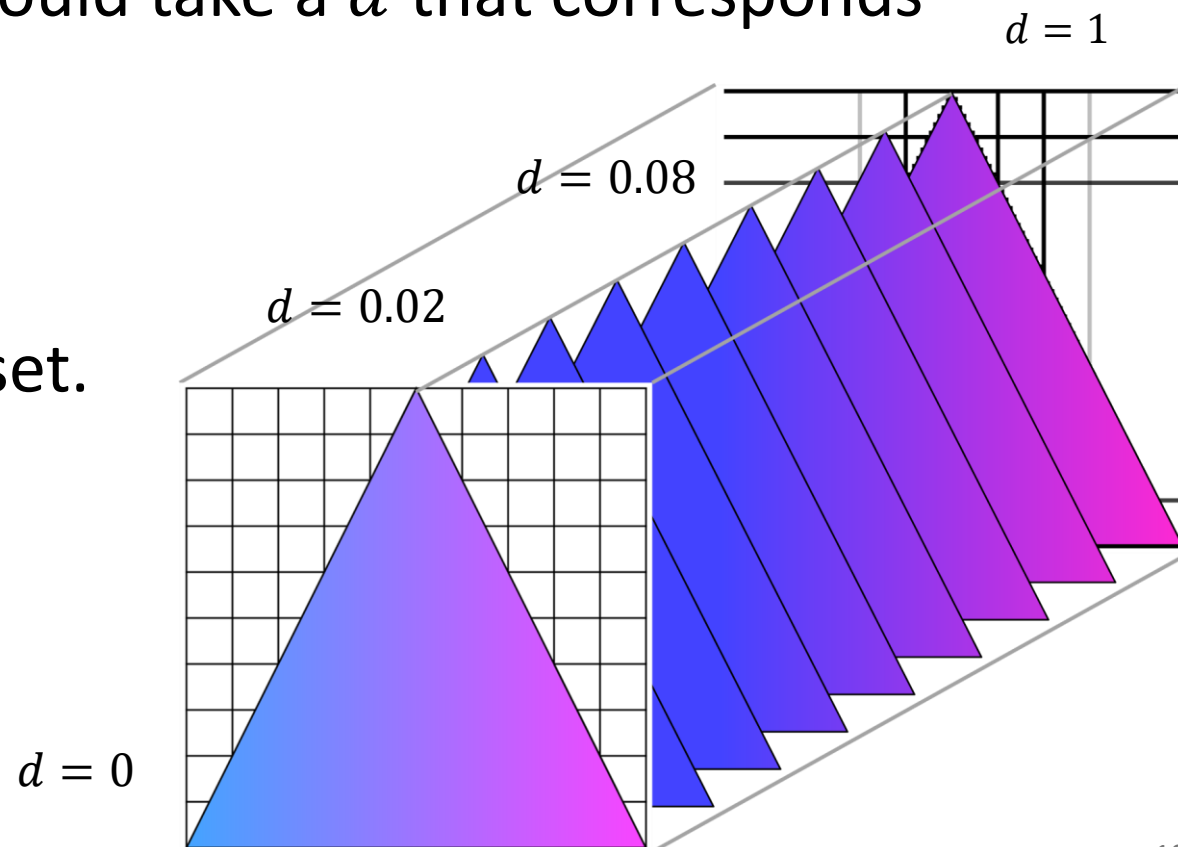


Continuous levels  $d$



# Problems with MIP mapping

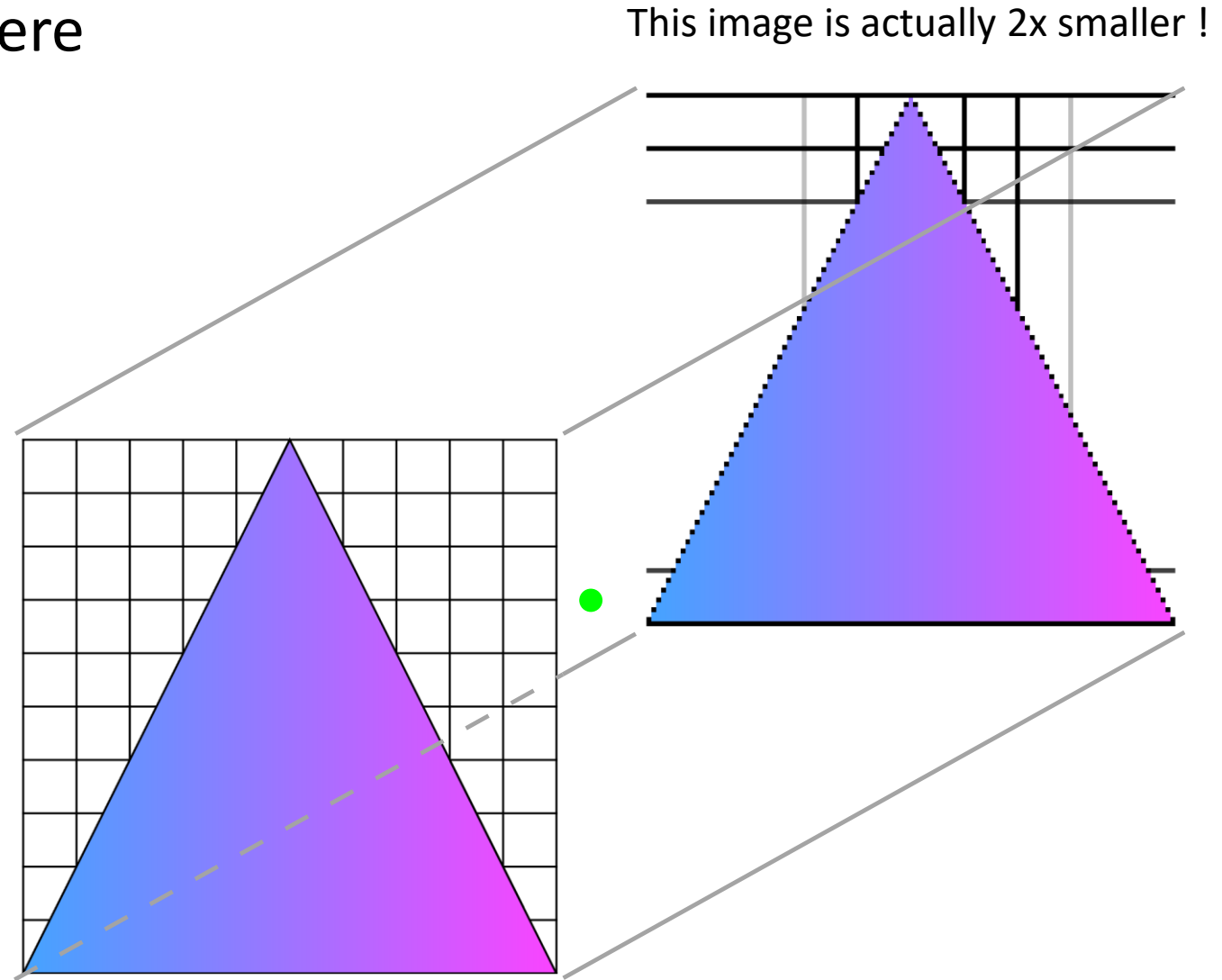
- What to do instead ?
- Suppose we had not downsampled by 2 every mip map.
- But by a continuous downscaling  $d \in (0, 0.001, 0.002, \dots, 0.999, 1, 1.001, \dots)$
- Then instead of taking  $d = \log_2 L$  we could take a  $d$  that corresponds exactly to the correct downscale.
- But.. We do not have this continuous mip map level  $d$ , we only have a finite set.
- What to do ?



# Continuous Mipmapping

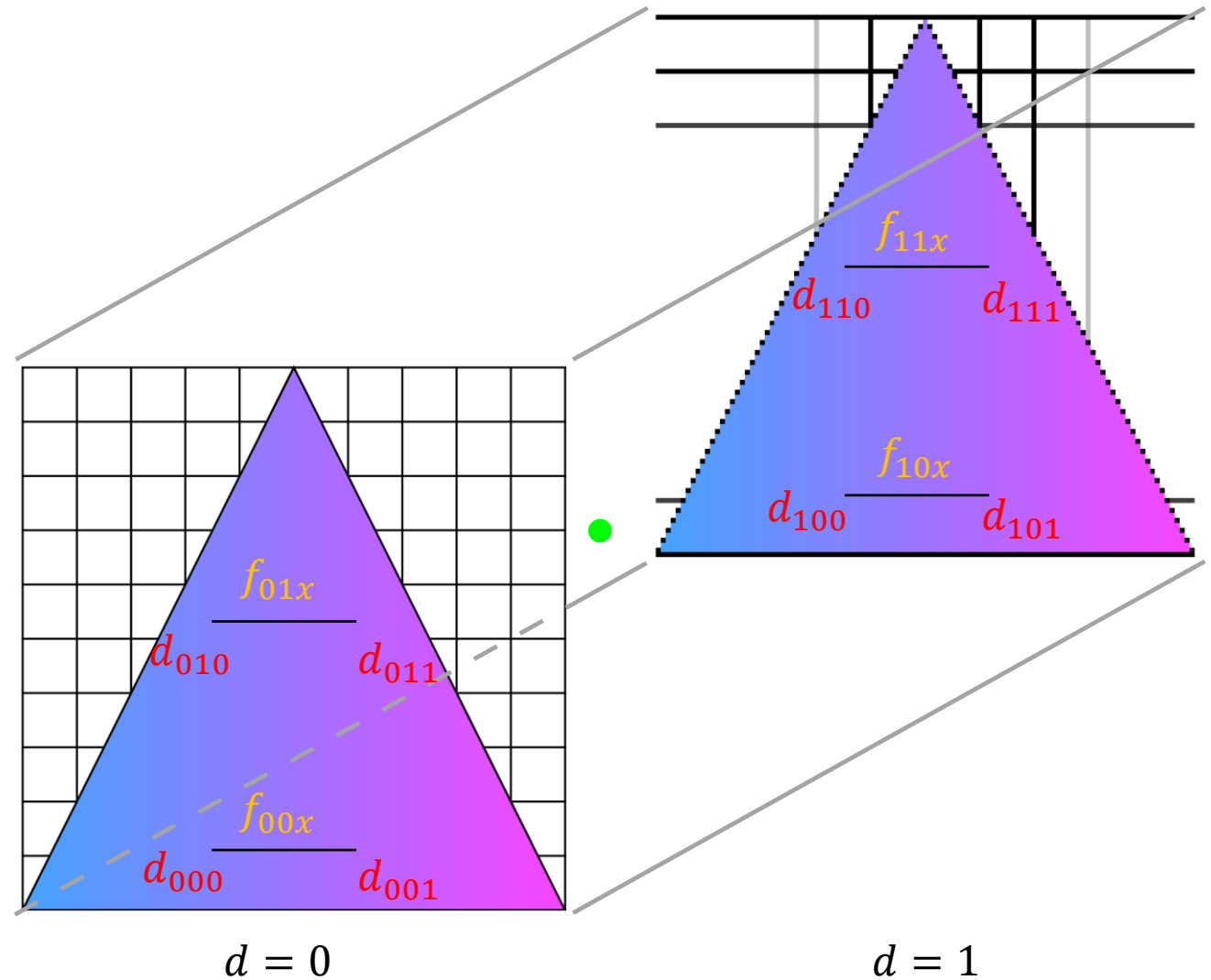
- Suppose we want the green dot here

- Can we compute its color from the mip maps we have ?



- Suppose we want the green dot here
- Can we compute its color from the mip maps we have ?

- We can interpolate in each mip map, taking 8 closest neighbors
- $f_{11x} = (1 - u)d_{110} + ud_{111}$
- $f_{10x} = (1 - u)d_{100} + ud_{101}$
- $f_{01x} = (1 - u)d_{010} + ud_{011}$
- $f_{00x} = (1 - u)d_{000} + ud_{001}$
- 4 interpolations around green dot

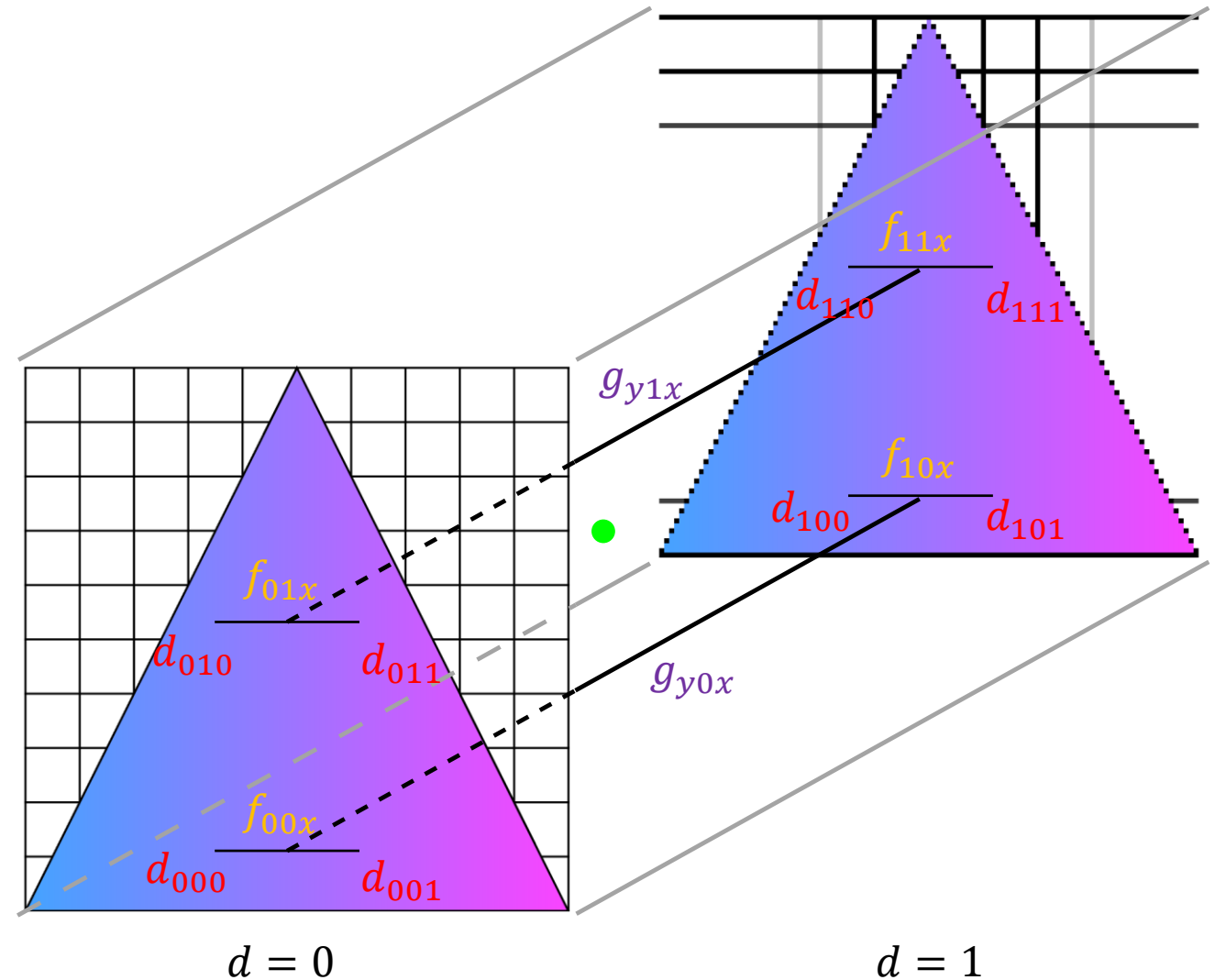


- Suppose we want the green dot here
- Can we compute its color from the mip maps we have ?

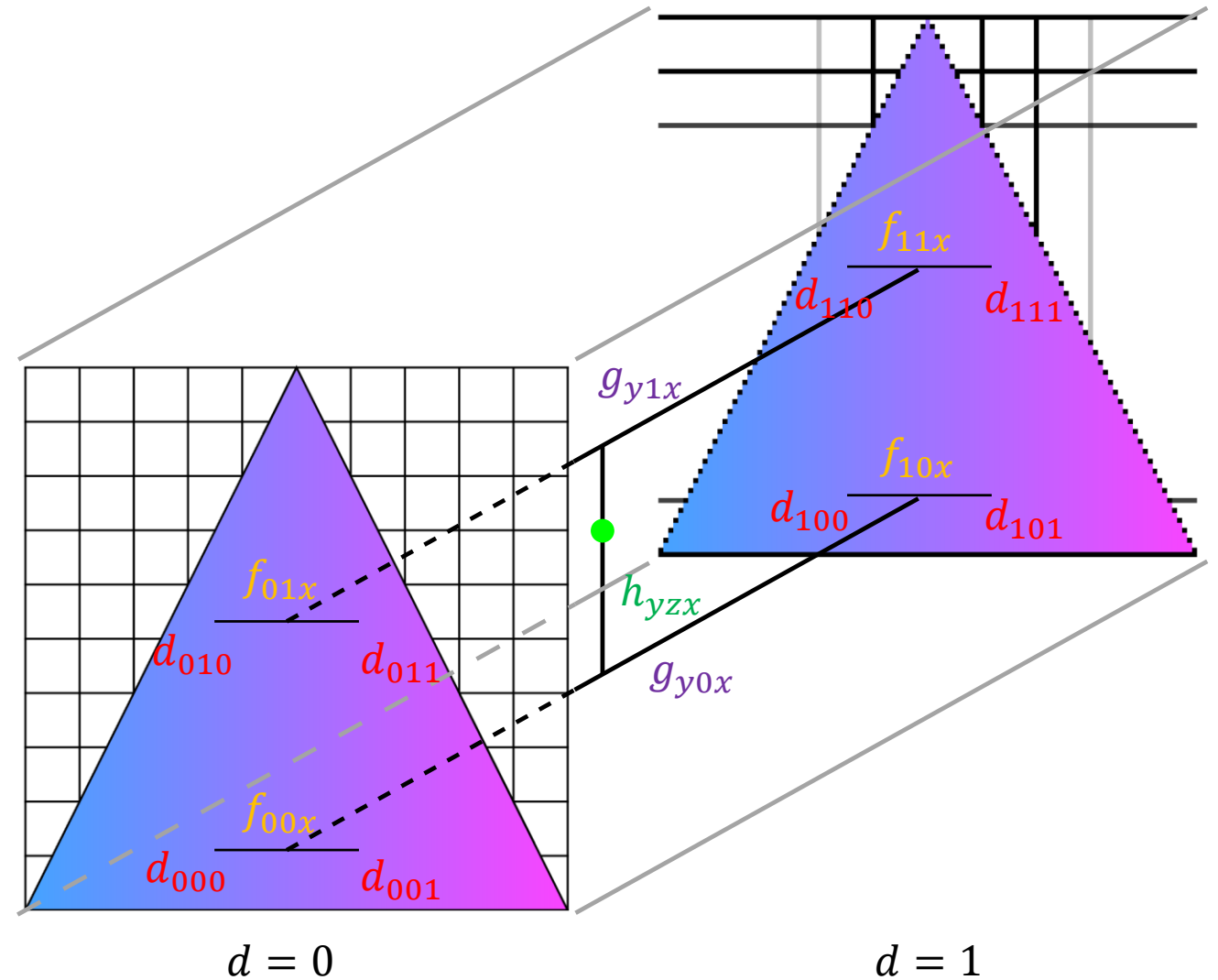
- We can interpolate  $f$  between mip maps

- $g_{y1x} = (1 - v)f_{11x} + vf_{01x}$
- $g_{y0x} = (1 - v)f_{10x} + vf_{00x}$

- 2 interpolations between  $f$ 's



- Suppose we want the green dot here
- Can we compute its color from the mip maps we have ?
- Finally, we can interpolate  $g$ 's
- $h_{yzx} = (1 - w)g_{y0x} + wg_{y1x}$
- 1 interpolations between  $g$ 's



- This is called **trilinear interpolation**

$$f_{11x} = (1 - u)d_{110} + ud_{111}$$

$$f_{10x} = (1 - u)d_{100} + ud_{101}$$

$$f_{01x} = (1 - u)d_{010} + ud_{011}$$

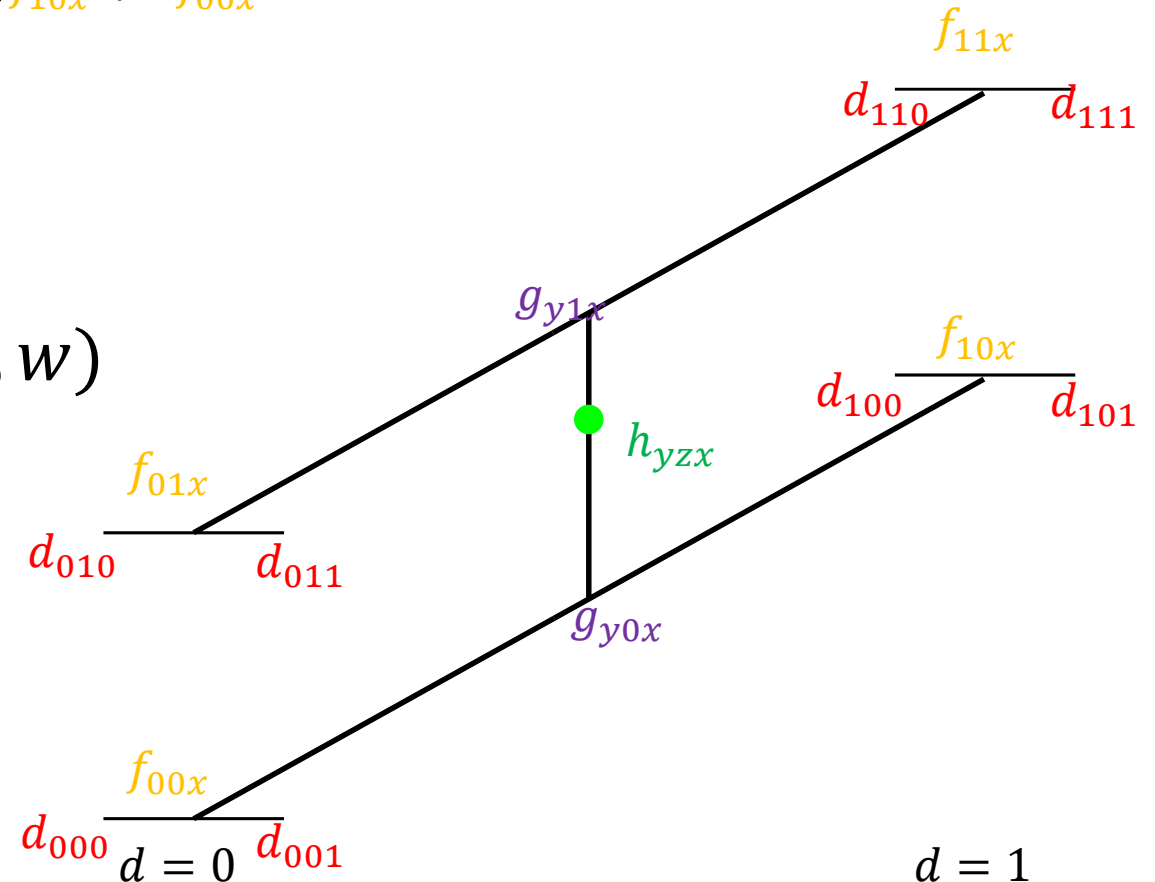
$$f_{00x} = (1 - u)d_{000} + ud_{001}$$

$$g_{y1x} = (1 - v)f_{11x} + vf_{01x}$$

$$g_{y0x} = (1 - v)f_{10x} + vf_{00x}$$

$$h_{yzx} = (1 - w)g_{y0x} + wg_{y1x}$$

- Our green dot has 3 coordinates  $(u, v, w)$
- We can use the interpolation to find its color !



# Mipmapping – Trilinear Interpolation

- Recap:
  - We interpolate between two mipmaps closest to  $d$
  - Perform a 2 linear interpolation independently in each mipmap
  - Perform 2 linear interpolations between the interpolated values
  - Perform 1 linear interpolation between the two interpolated interpolated values
- Its expensive !
  - Bilinear interpolation
    - 4 texel reads
    - 3 linear interpolations
  - Trilinear interpolation
    - 8 texel reads
    - 7 linear interpolations
- We use specialized hardware in the GPU

# Mipmapping – Complete Recap

- Complete recap of texture sampling:
  - Compute  $u$  and  $v$  from screen sample  $(x, y)$
  - Approximate  $\frac{du}{dx}, \frac{du}{dy}, \frac{dv}{dx}, \frac{dv}{dy}$  by taking differences
  - Compute mip map level  $d$
  - Convert normalized  $[0,1]$  texture coordinate  $(u, v)$  to pixel locations  $(U, V) \in [W, H]$  in texture image
  - Determine texels needed for filter
    - 8 neighbors in trilinear interpolation
  - Perform trilinear interpolation using  $(U, V, d)$

# Other kind of textures

- We spoke a lot about how to sample textures, but what are they exactly?
  - We assumed they were images
  - But in reality, they are only buffers (memory storages)
  - We can store anything inside !
  - The image you see on the screen is a texture
  - But we can also render an image in an off-screen texture, and use it on objects (Rendering the viewpoint of a mirror for example)
- 
- Let's see what else we can do with those buffers.

# Rendering complex models

- Real meshes have a polycount of millions to billions of triangles
- OpenGL cannot compute light on that many elements in real-time!



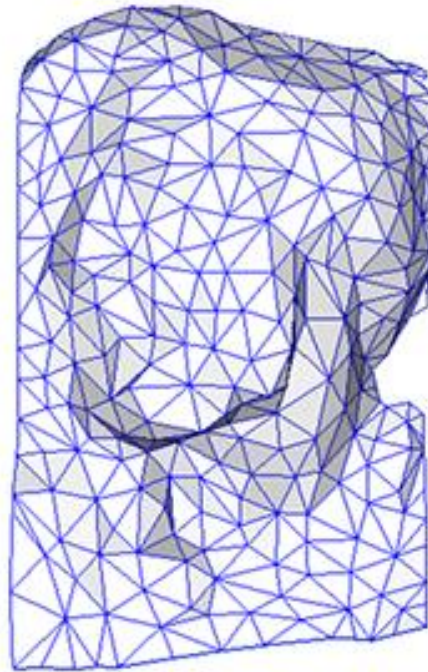
original mesh  
4M triangles

# Rendering complex models

- Real meshes have a polycount of millions to billions of triangles
- OpenGL cannot compute light on that many elements in real-time!
- We merely have enough power to compute simplified meshes



original mesh  
4M triangles



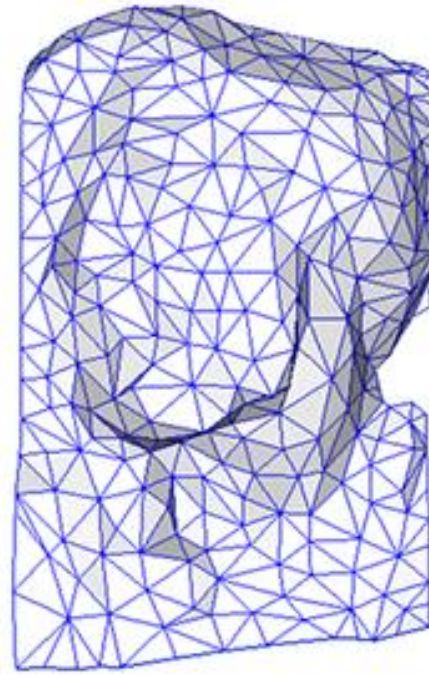
simplified mesh  
500 triangles

# Rendering complex models

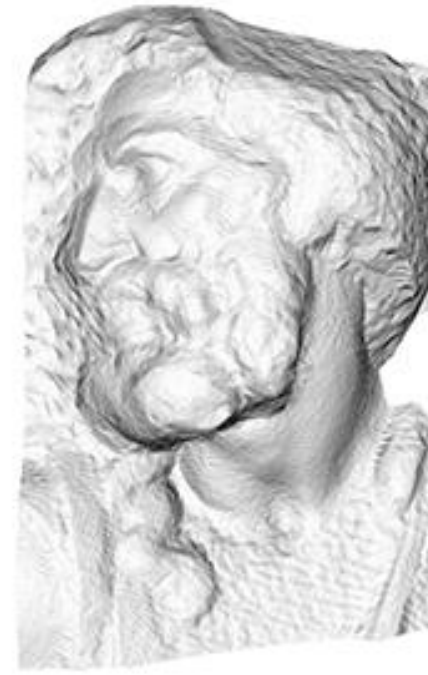
- Real meshes have a polycount of millions to billions of triangles
- OpenGL cannot compute light on that many elements in real-time!
- We merely have enough power to compute simplified meshes
- How are real-games rendered then?



original mesh  
4M triangles



simplified mesh  
500 triangles



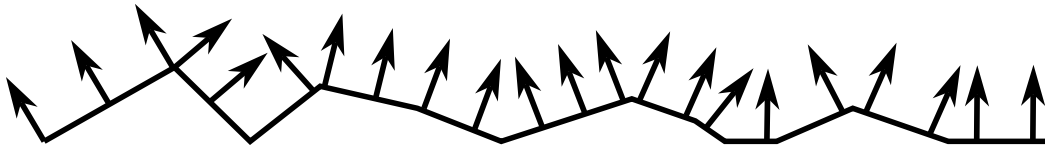
simplified mesh  
500 triangles

# Normal Mapping

- We use a trick with the light to avoid that many computations!



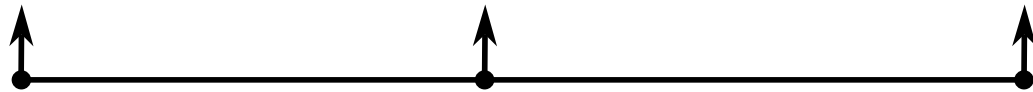
Suppose we have this surface



It has lots of normal vectors



We would like a flat surface with less vertices



This surface too has normal vectors



What if we keep the normals of the real surface



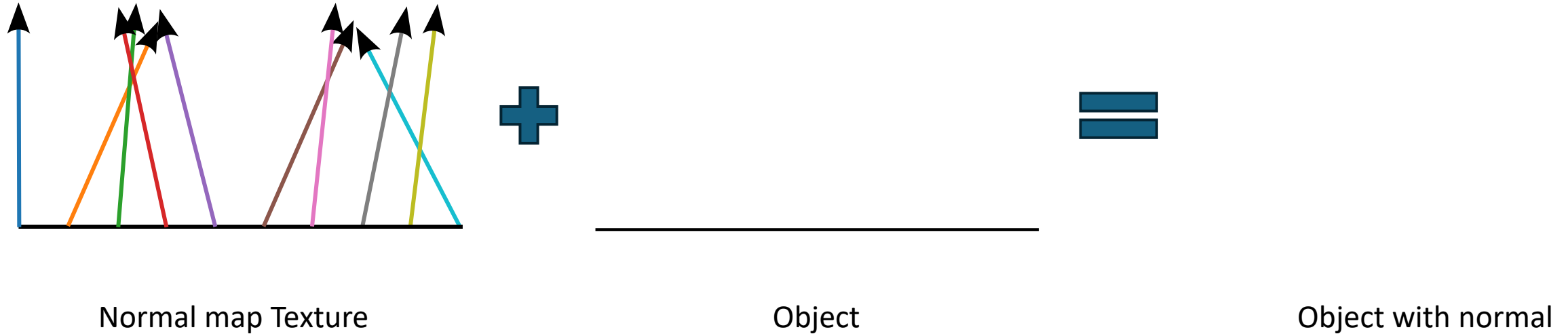
And apply them on the flat surface..  
What happens with the light equations ?

# Normal Mapping

- This is **normal mapping** !
- We start with a complex model
- We extract the normal vectors and store them into a texture
  - (We will describe how shortly)
- We put a flat rectangle instead of the model (2 triangles = 4 vertices)
- When rendering, we only have the 4 vertices to render
- BUT we can use the normals from the texture to compute the lights !

# Normal Mapping

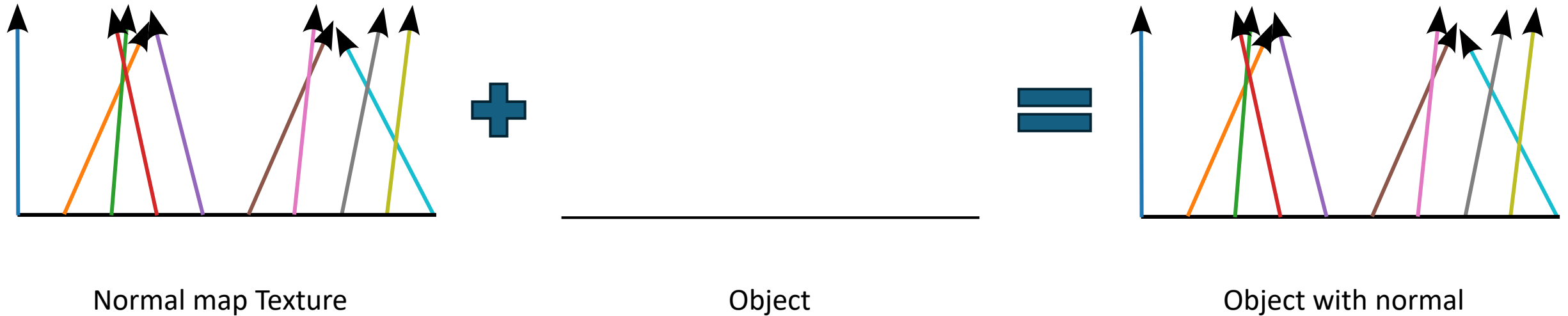
- Ok.. We stored normals in a texture..
- Now, we would like to apply it to a model



- What this gives ?

# Normal Mapping

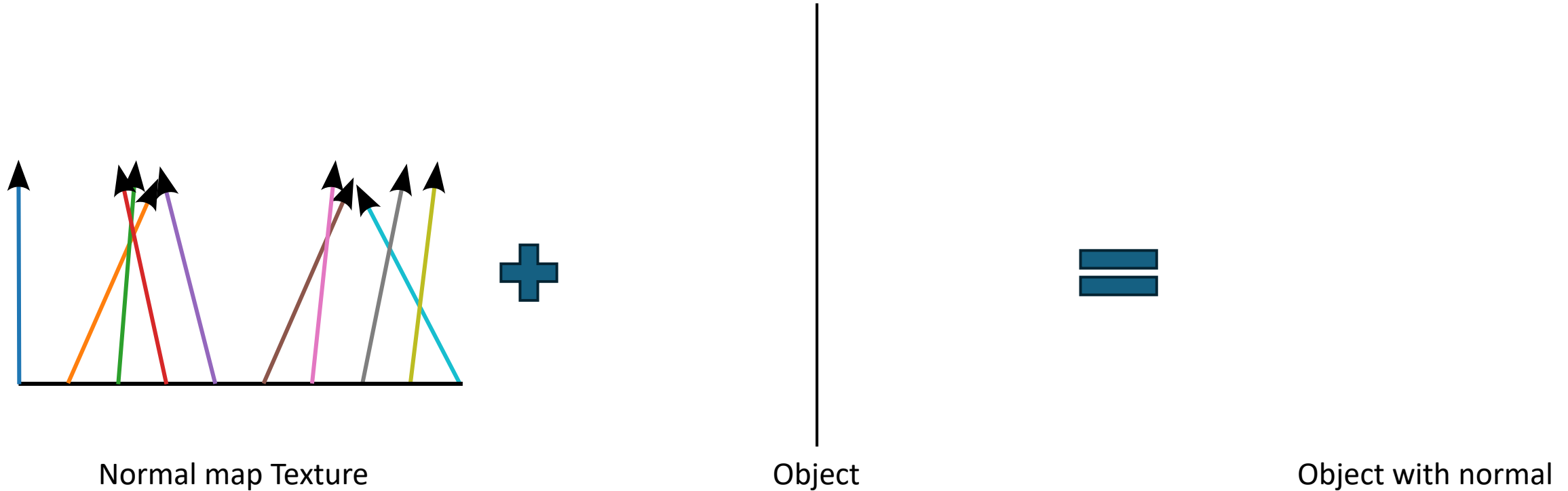
- Ok.. We stored normals in a texture..
- Now, we would like to apply it to a model



- Works perfectly ! What about..

# Normal Mapping

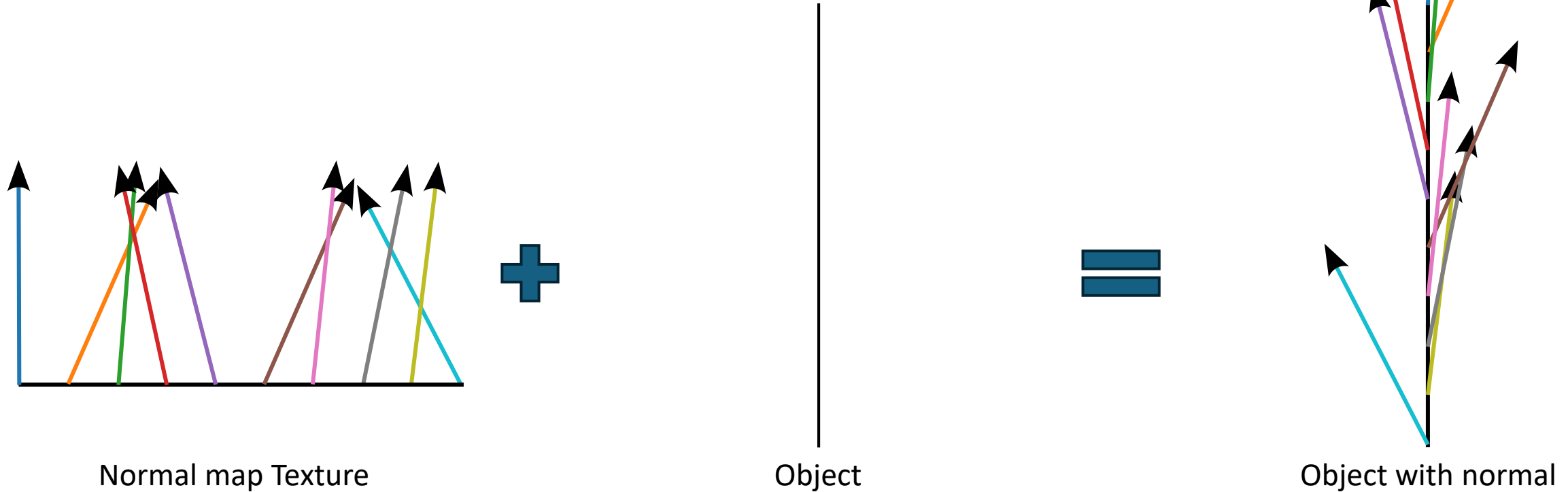
- Ok.. We stored normals in a texture..
- Now, we would like to apply it to a model



- What about this ? The model was rotated 90 degrees

# Normal Mapping

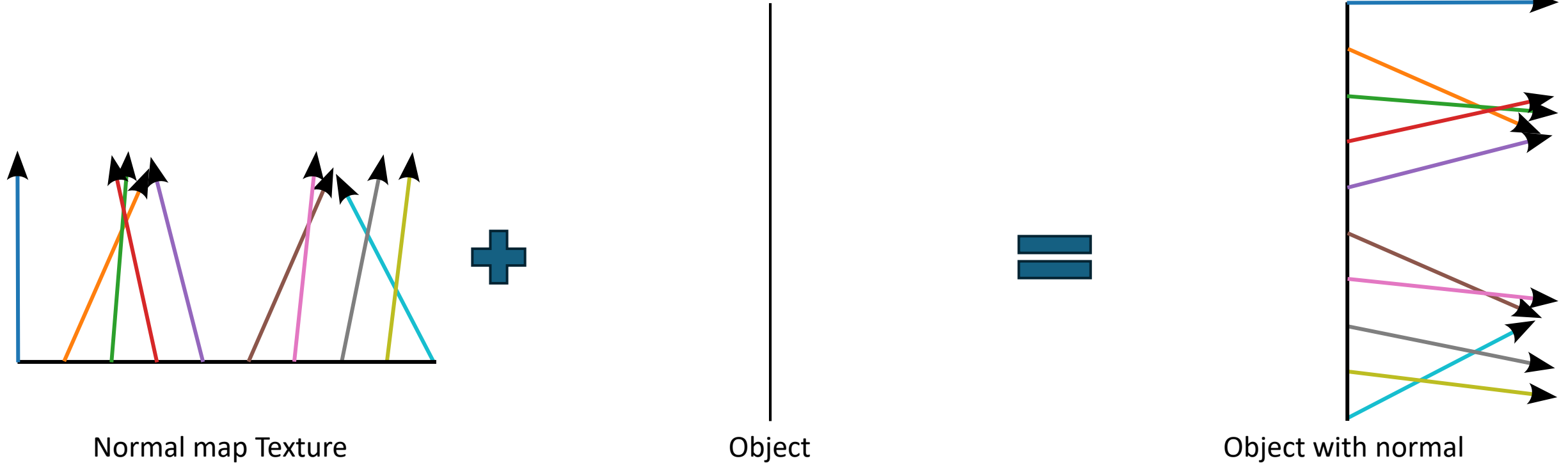
- Ok.. We stored normals in a texture..
- Now, we would like to apply it to a model



- Oups.. We need to rotate the normals as well ! ( $n' = M^T n$ )

# Normal Mapping

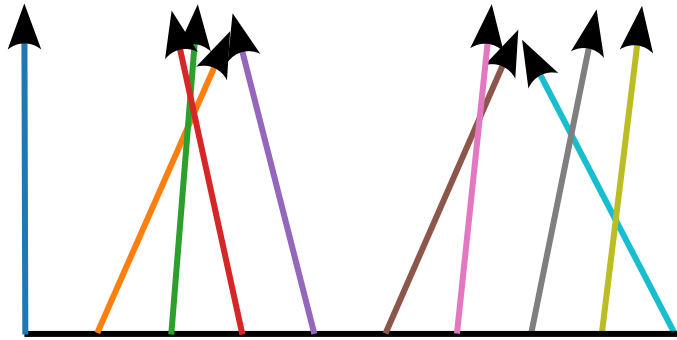
- Ok.. We stored normals in a texture..
- Now, we would like to apply it to a model



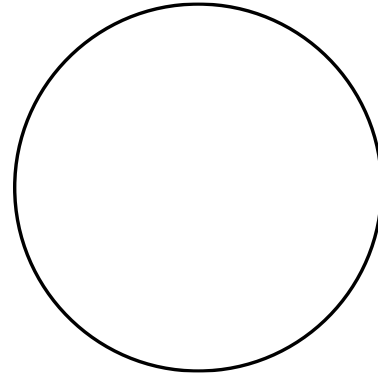
- Ok... Good ! But what about...

# Normal Mapping

- Ok... Good ! But what about...



Normal map Texture



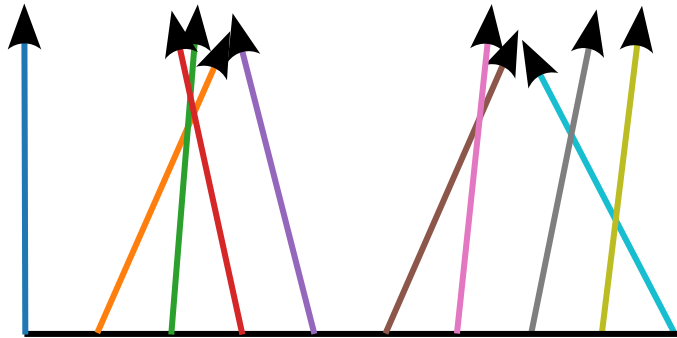
Object



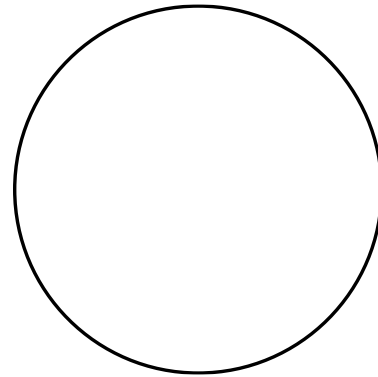
Object with normal

# Normal Mapping

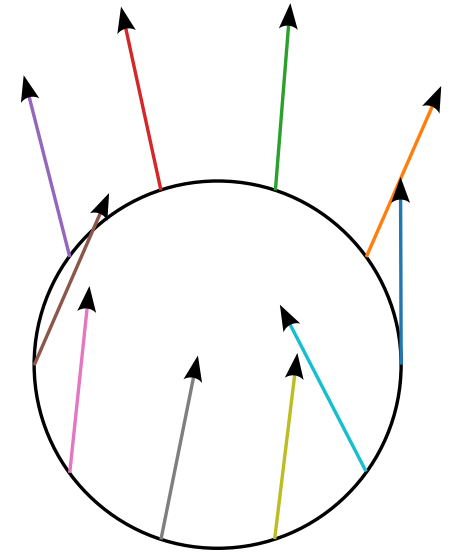
- Ok.. We stored normals in a texture..
- Now, we would like to apply it to a model



Normal map Texture



Object

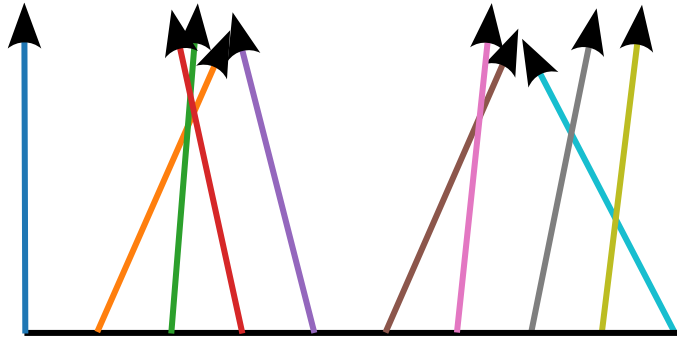


Object with normal

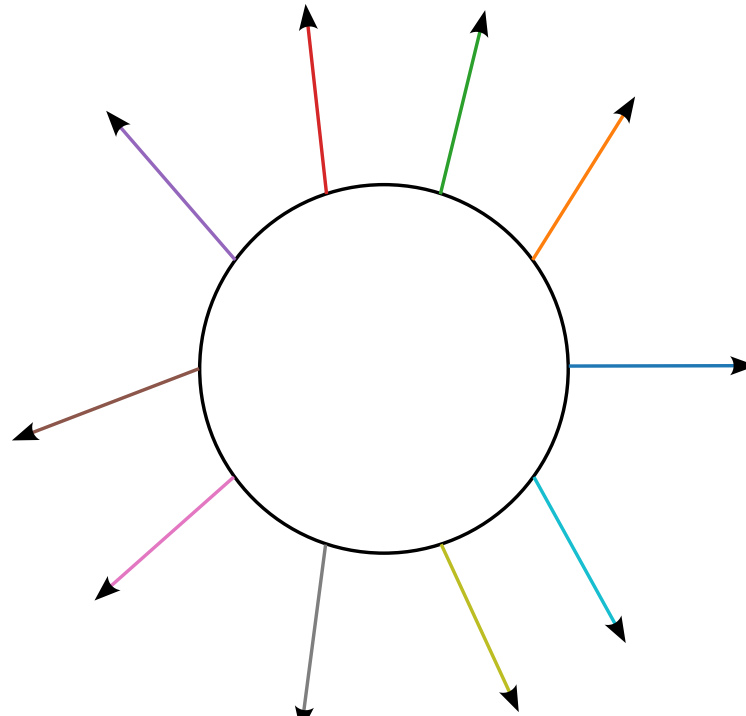
- ... This is not right at all.. Why ? Can we apply a transformation ?

# Normal Mapping

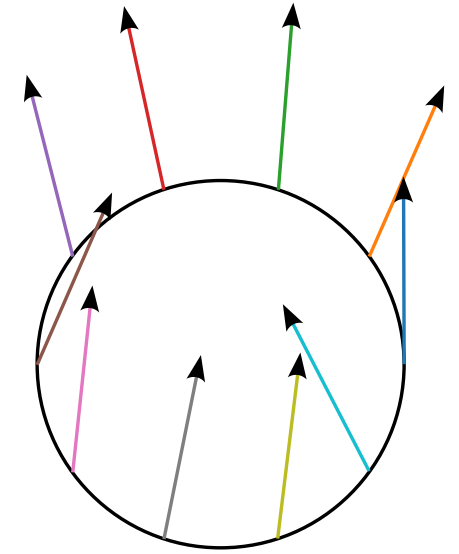
- ... This is not right at all.. Why ? Can we apply a transformation ?



Normal map Texture



Wanted object with normals

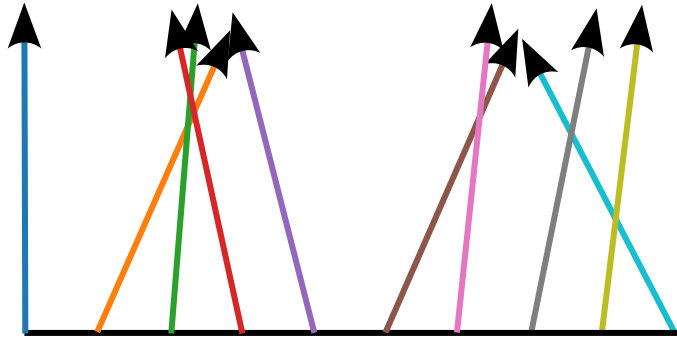


Obtained object with normal

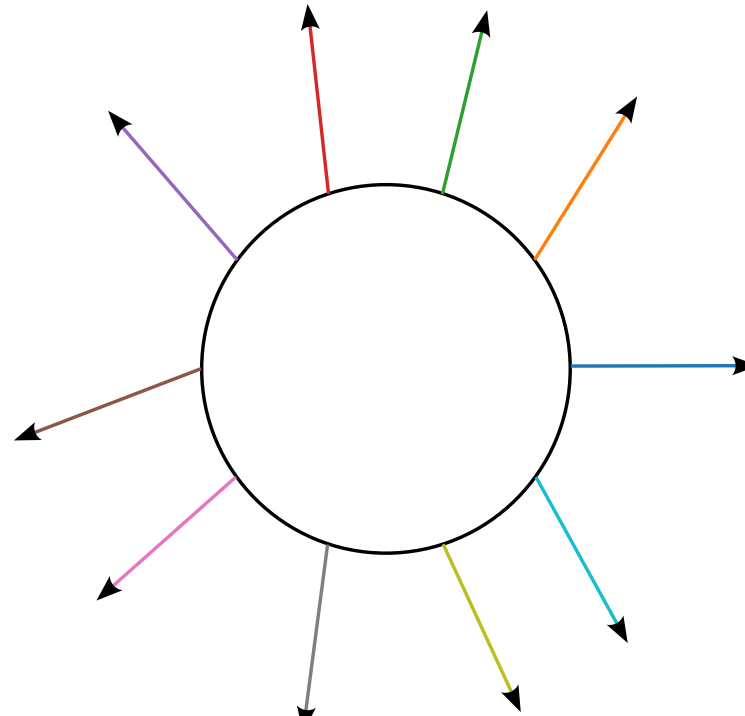
- We can't really find a transformation from our normal map to the object

# Normal Mapping

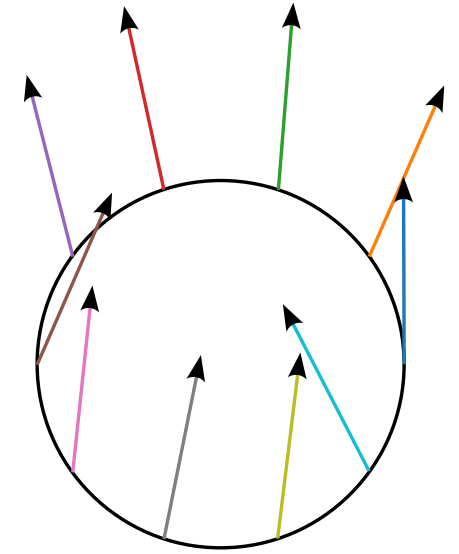
- We can't really find a transformation from our normal map to the object



Normal map Texture



Wanted object with normals

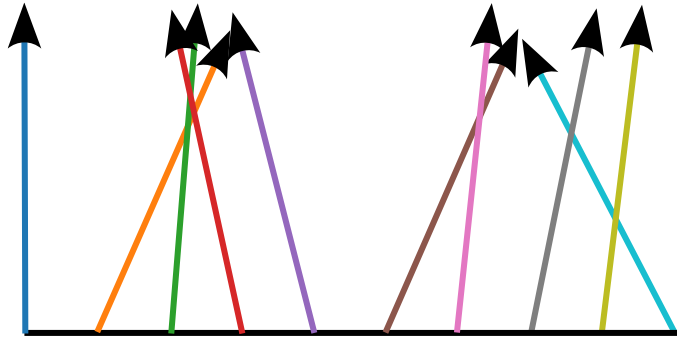


Obtained object with normal

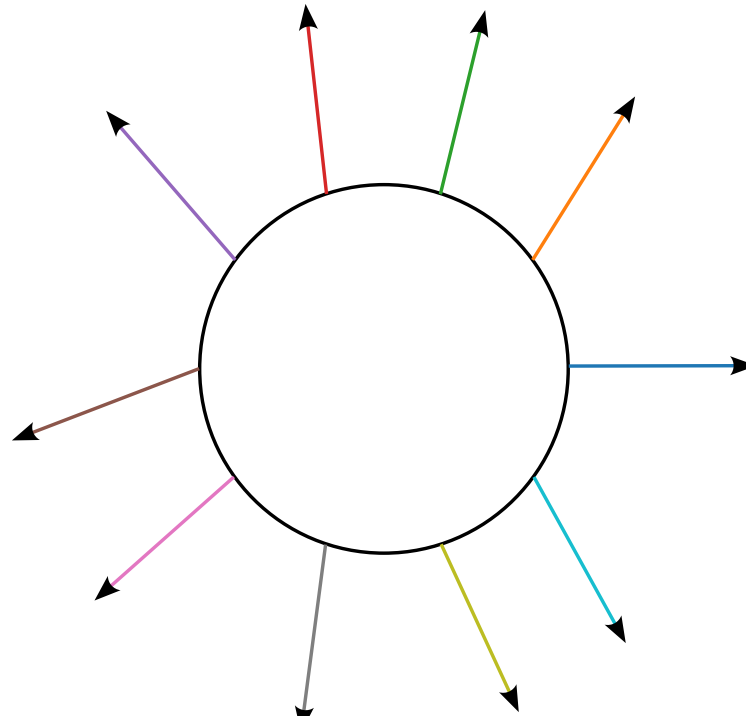
- What can we do ?

# Normal Mapping

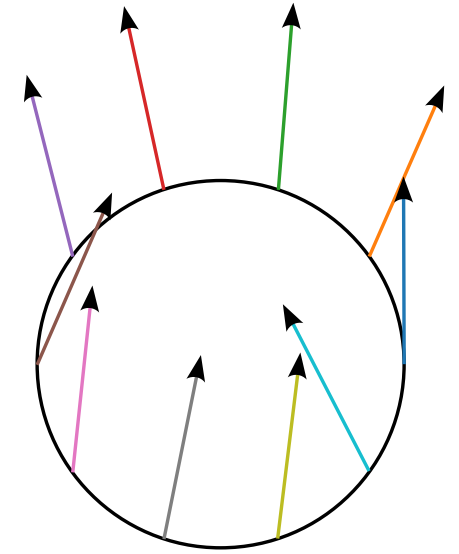
- What can we do ?



Normal map Texture



Wanted object with normals

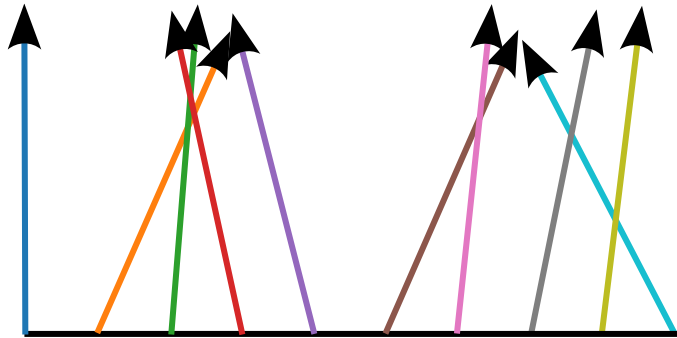


Obtained object with normal

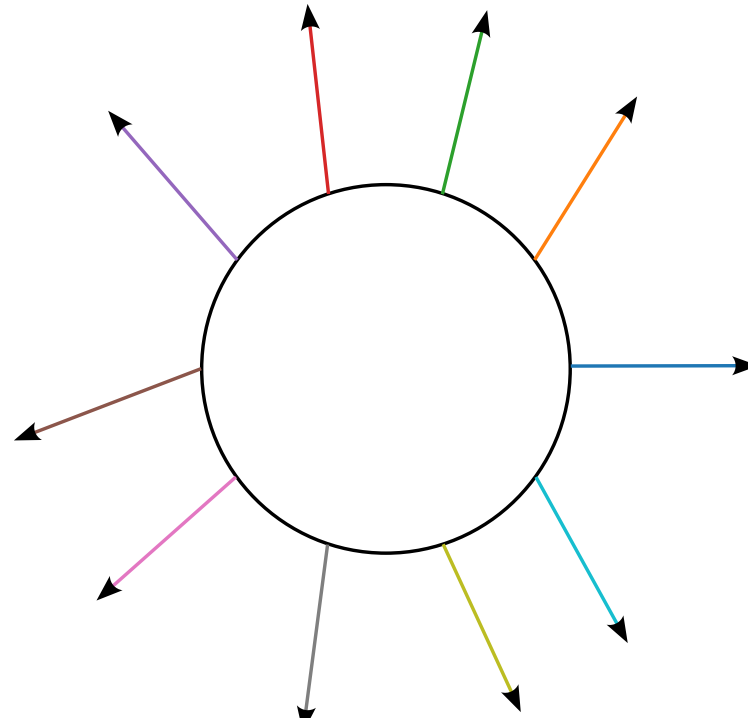
- What if, instead of storing a normal map as this, we stored the normals we want on our object in the texture ?

# Normal Mapping

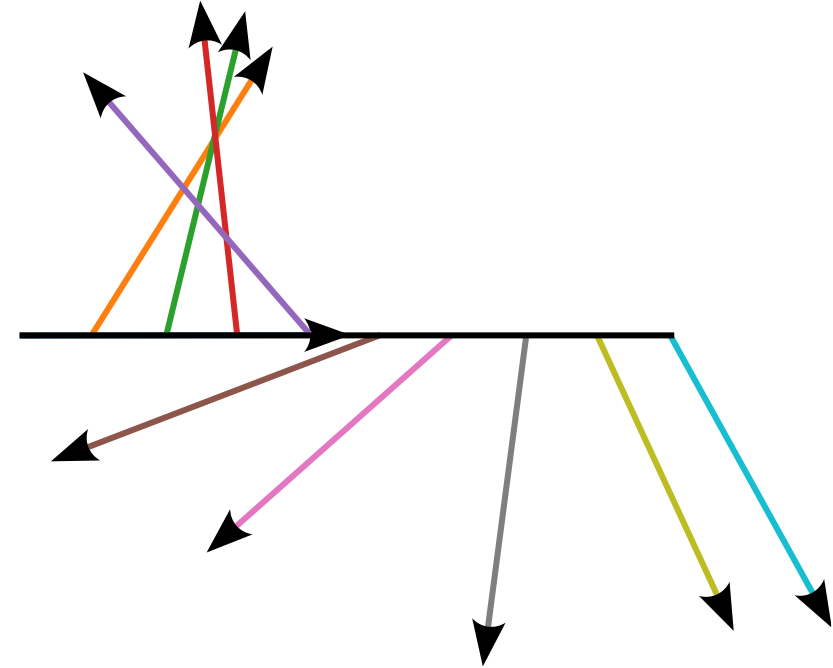
- What if, instead of storing a normal map as this,



Normal map Texture



Wanted object with normals

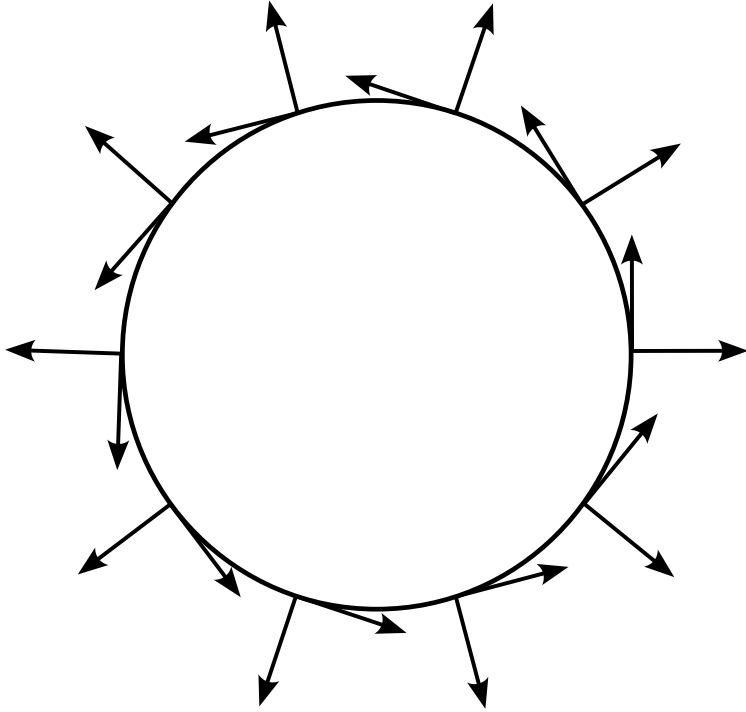


Normal map with correct normals

- Ok, but.. How ?

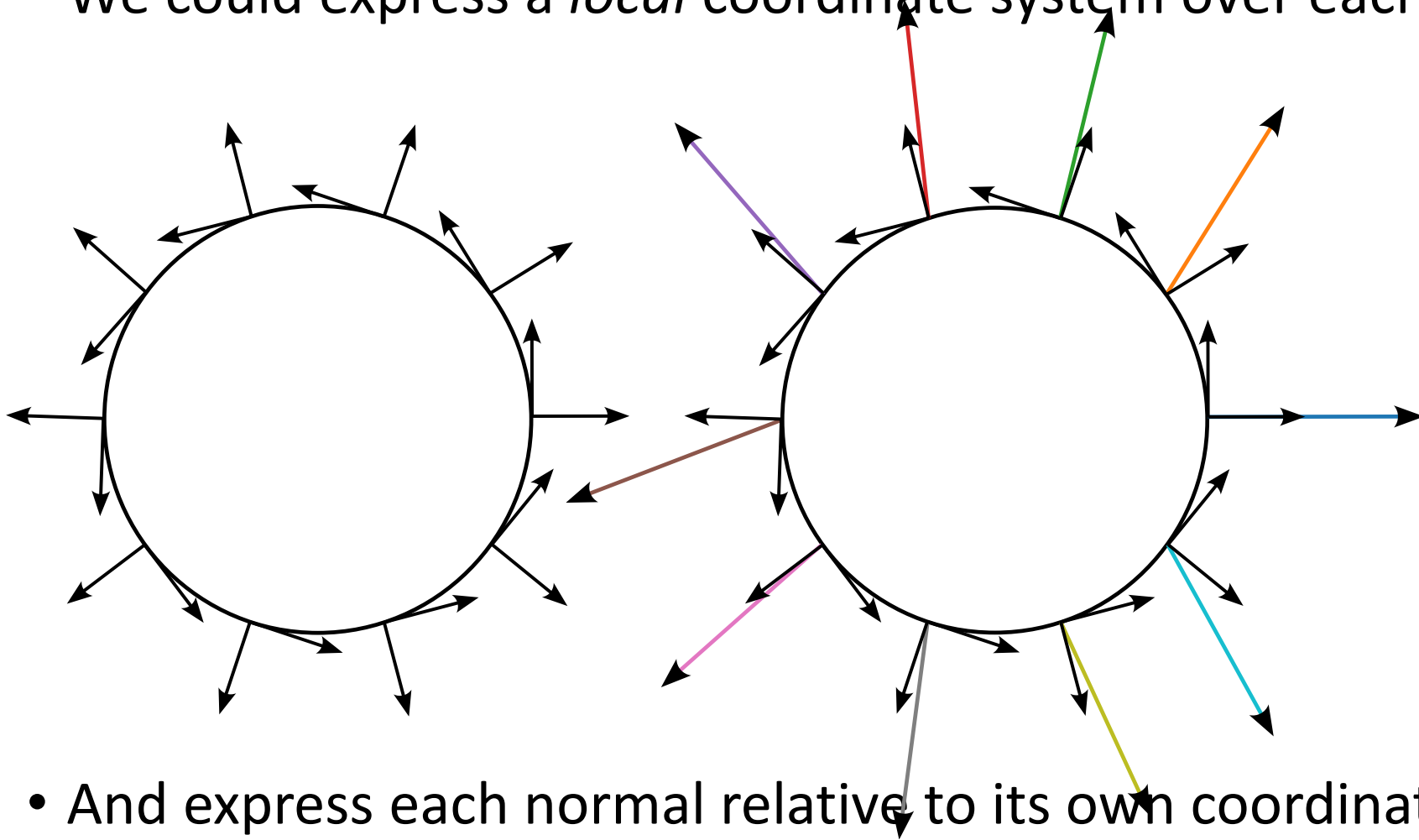
# Normal Mapping

- We could express a *local* coordinate system over each vertex



# Normal Mapping

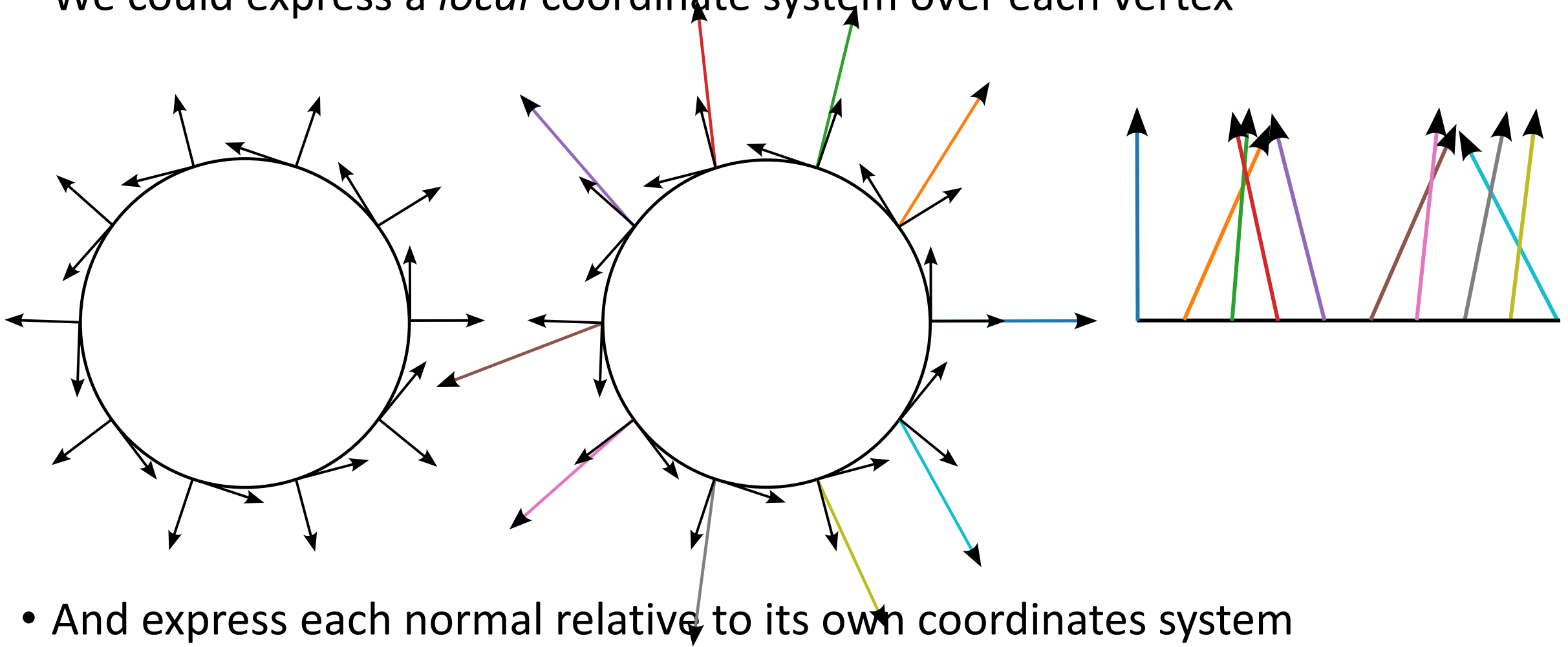
- We could express a *local* coordinate system over each vertex



- And express each normal relative to its own coordinates system

# Normal Mapping

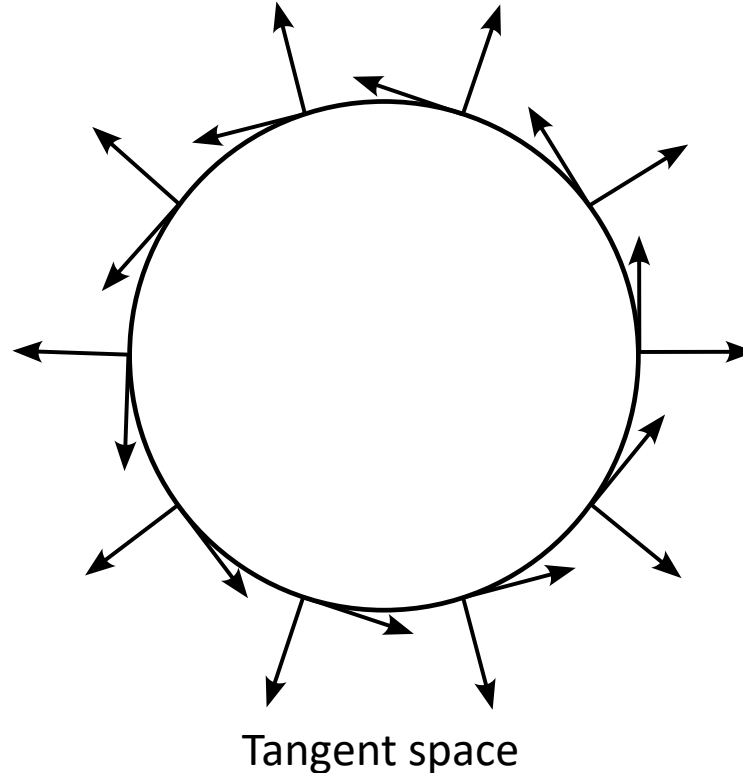
- We could express a *local* coordinate system over each vertex



- And express each normal relative to its own coordinates system
- When we unroll it into a map, it *flattens* the tangent vector

# Normal Mapping

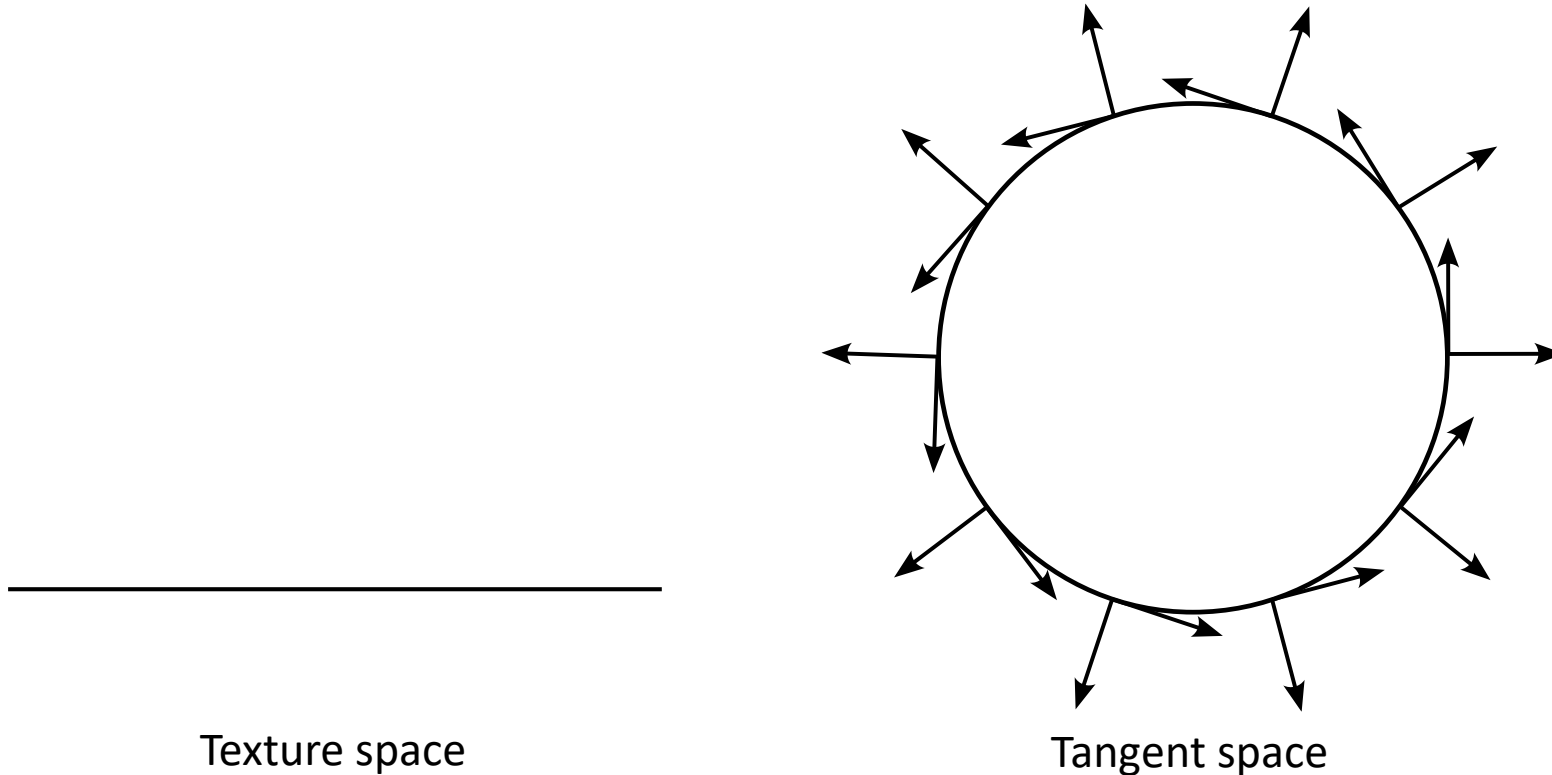
- This *local* coordinates system is called the **tangent space**.



- The tangent space is formed by *normal*, *tangent* and *bitangent* vectors as basis over each vertex.

# Normal Mapping

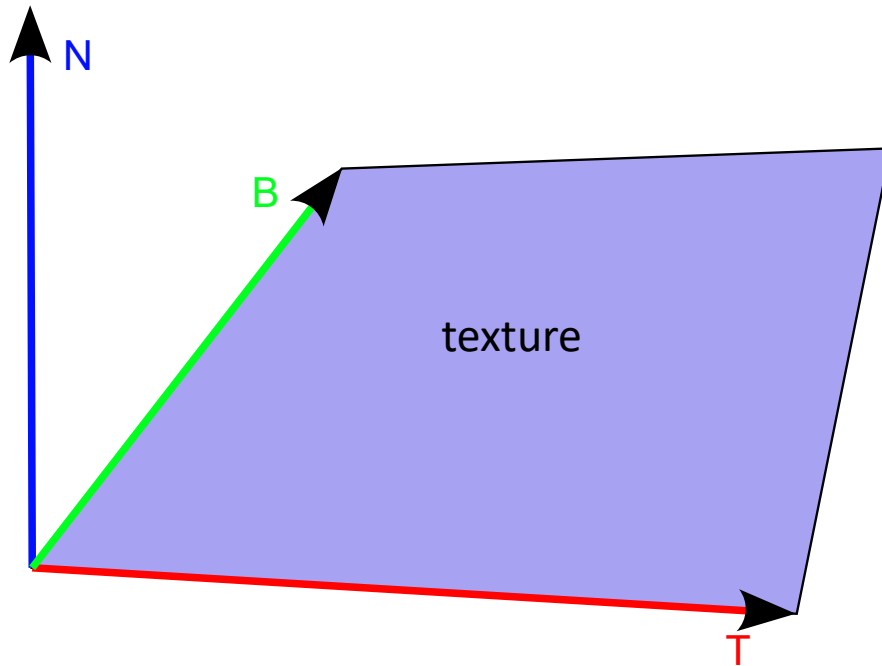
- This *local* coordinates system is called the **tangent space**.



- We just need to find a map from the texture space to the tangent space !
- And this tangent space must be coherent with the texture space.

# Normal Mapping

- In our texture, the normal points outward, the tangent and bitangent are aligned to its borders ( $u, v$  coordinates)

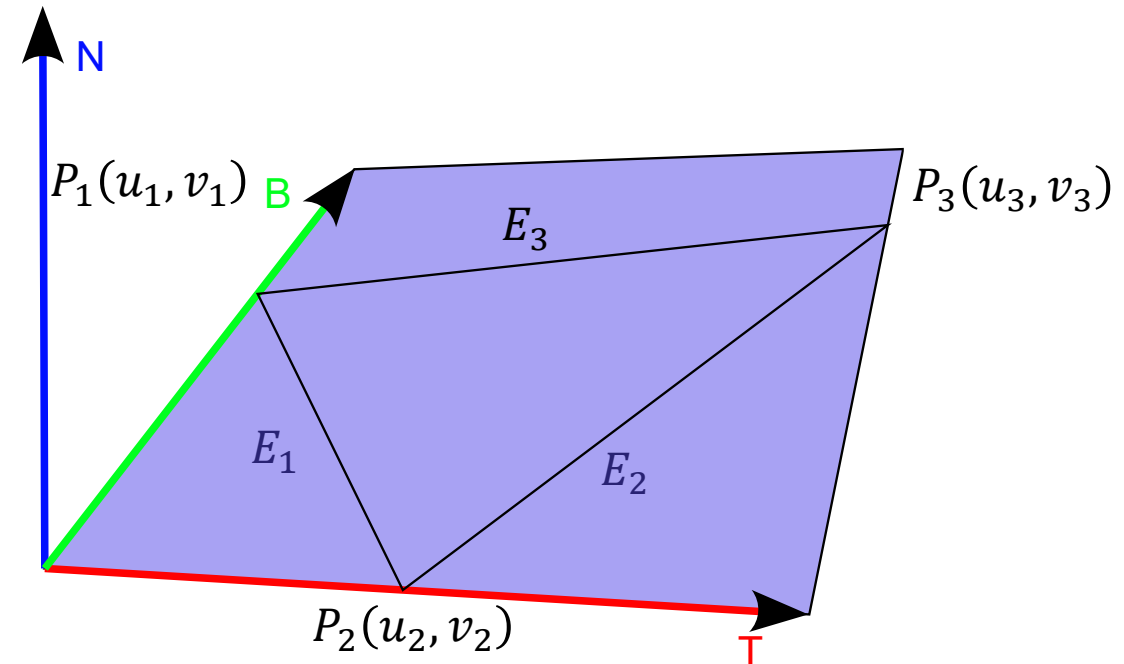
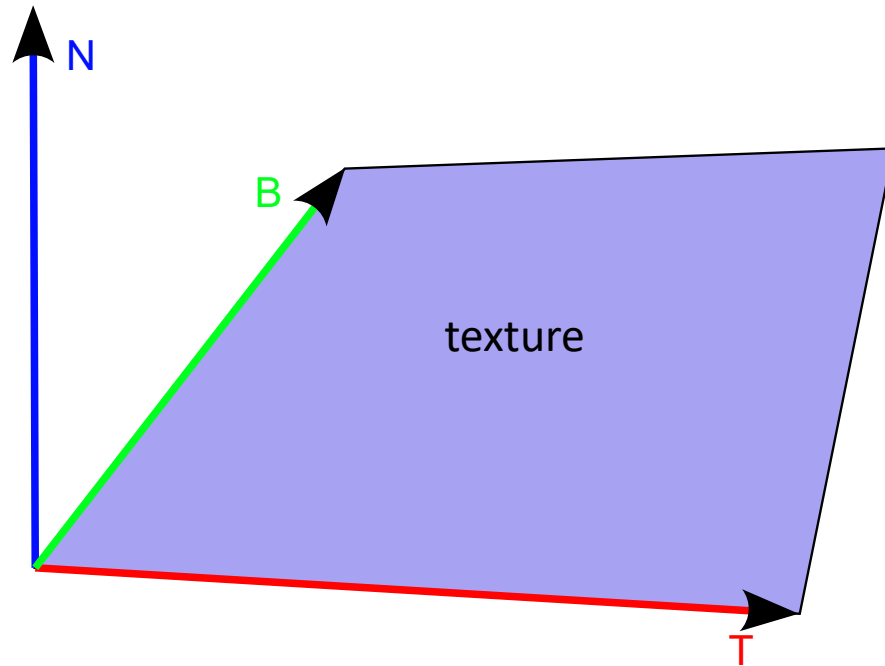


So, every point on the texture is relative to its own  $(T, B, N)$  coordinates system !

- We need a way to find the tangent and bitangent
- Luckily for us, we can use the fact that they are coplanar

# Normal Mapping

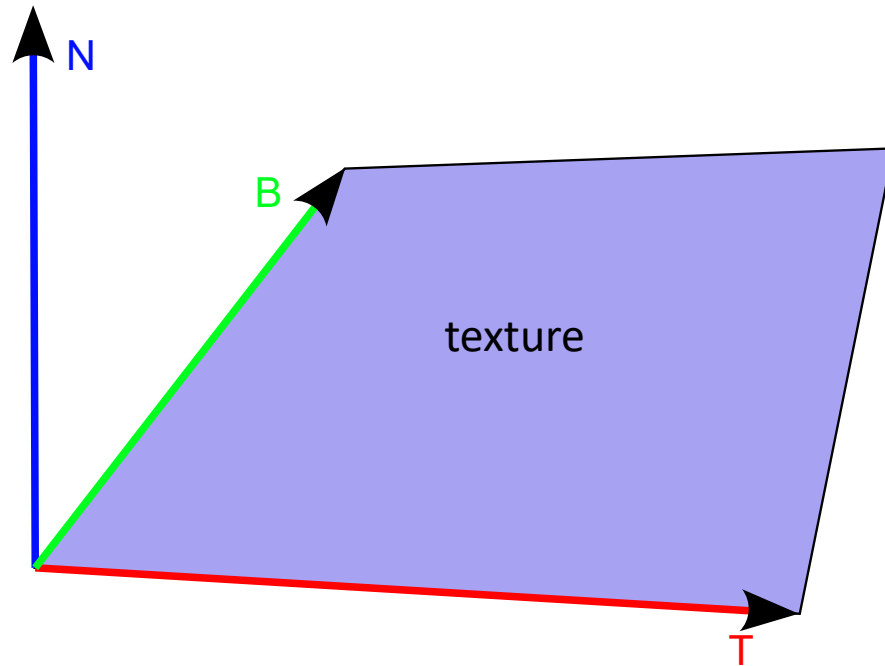
- In our texture, the normal points outward, the tangent and bitangent are aligned to its borders ( $u, v$  coordinates)



- If we take any 3 points  $P_1, P_2, P_3$  on the texture, they define a triangle with edges  $E_1, E_2, E_3$
- This triangle must be in the plane defined by  $B$  and  $T$

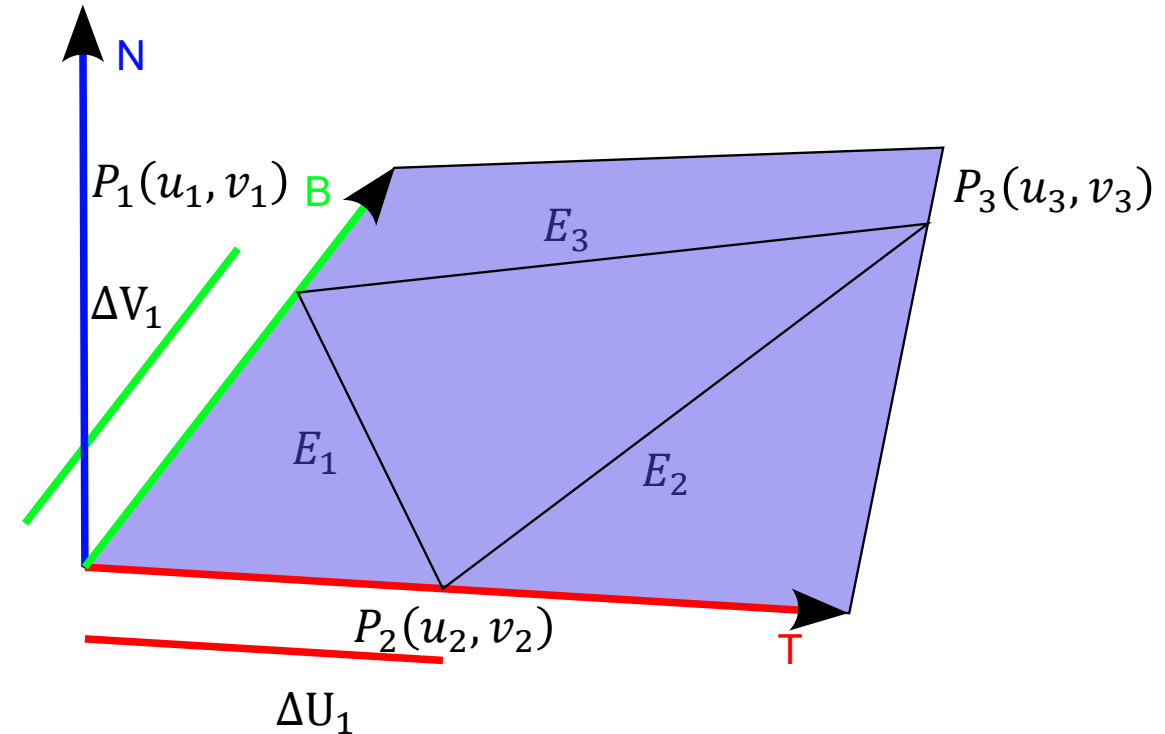
# Normal Mapping

- In our texture, the normal points outward, the tangent and bitangent are aligned to its borders ( $(u, v)$  coordinates)



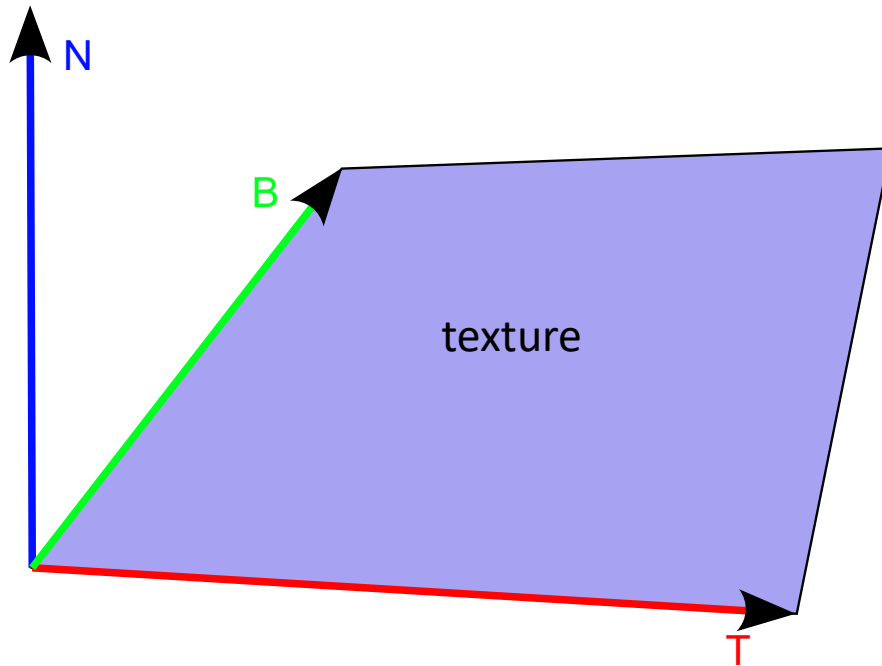
- We can express

$$E_1 = \Delta U_1 T + \Delta V_1 B$$

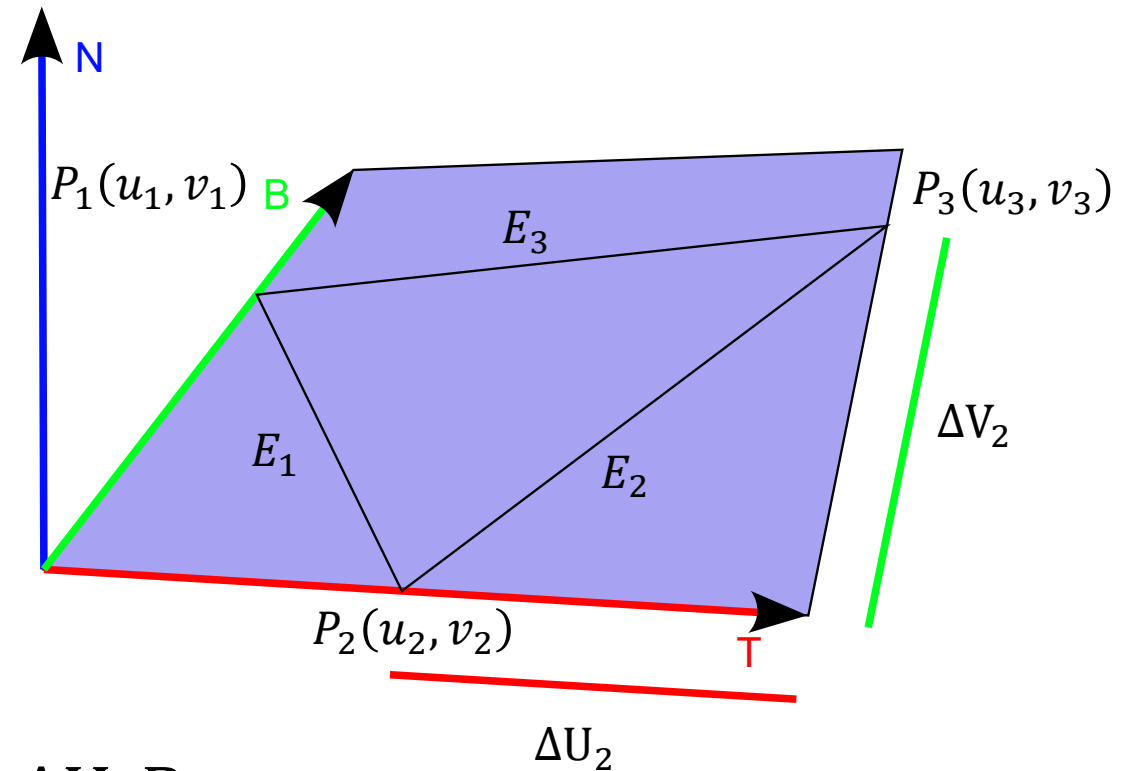


# Normal Mapping

- In our texture, the normal points outward, the tangent and bitangent are aligned to its borders ( $u, v$  coordinates)



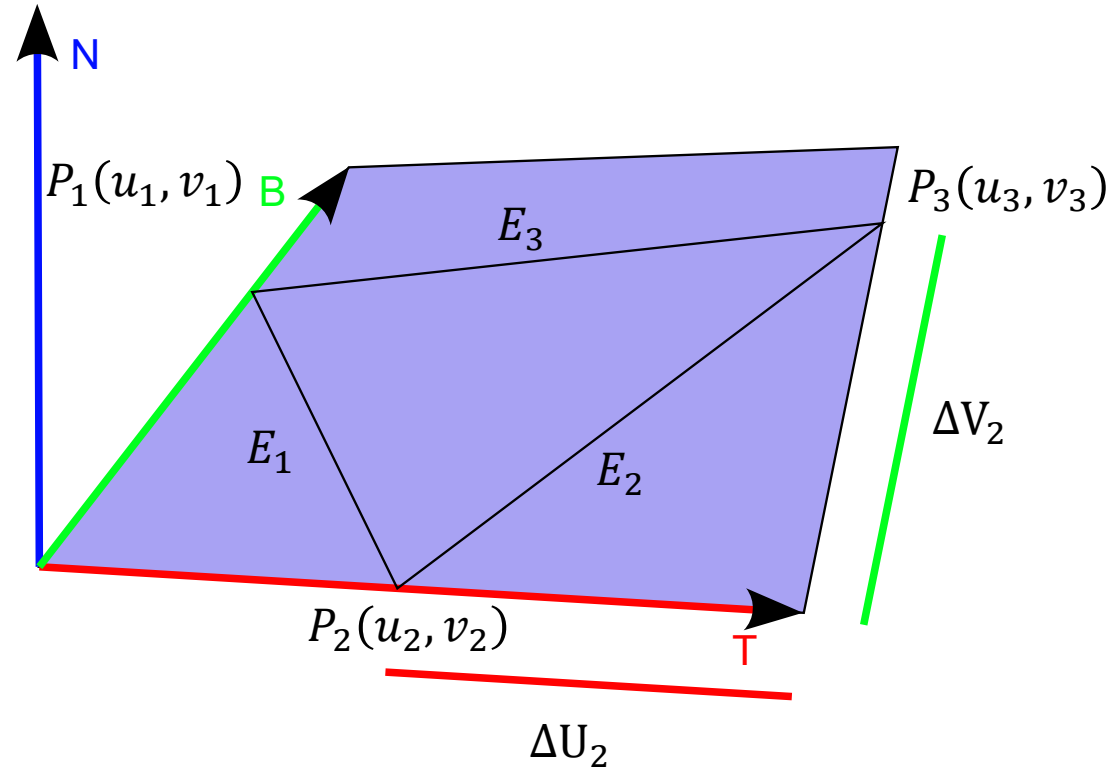
- And similarly



$$E_2 = \Delta U_2 T + \Delta V_2 B$$

# Normal Mapping

- We have a system of equations

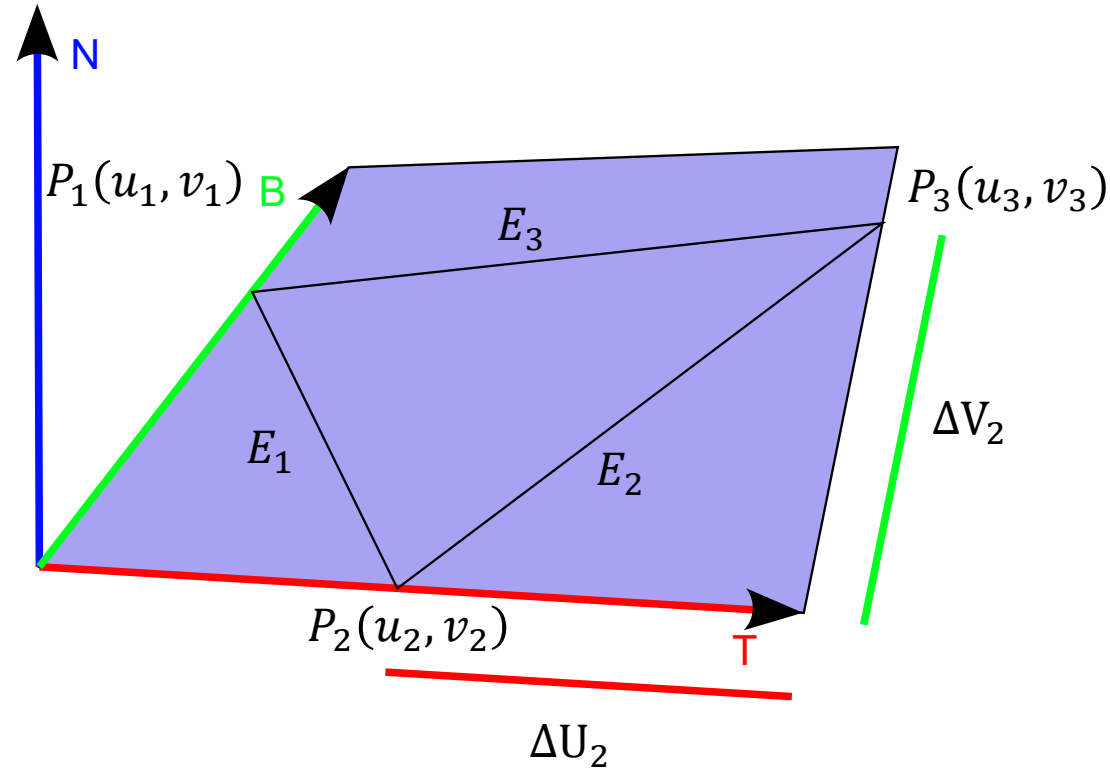


$$\begin{aligned} E_1 &= \Delta U_1 T + \Delta V_1 B \\ E_2 &= \Delta U_2 T + \Delta V_2 B \end{aligned}$$

- Two equations with two unknown ( $B, T$ ), we can solve !

# Normal Mapping

- We have a system of equations



$$\begin{pmatrix} E_1 \\ E_2 \end{pmatrix} = \begin{pmatrix} \Delta U_1 & \Delta V_1 \\ \Delta U_2 & \Delta V_2 \end{pmatrix} \begin{pmatrix} T \\ B \end{pmatrix} \Leftrightarrow \begin{pmatrix} T \\ B \end{pmatrix} = \begin{pmatrix} \Delta U_1 & \Delta V_1 \\ \Delta U_2 & \Delta V_2 \end{pmatrix}^{-1} \begin{pmatrix} E_1 \\ E_2 \end{pmatrix}$$

# Normal Mapping

- We have a system of equations

$$\begin{pmatrix} T \\ B \end{pmatrix} = \begin{pmatrix} \Delta U_1 & \Delta V_1 \\ \Delta U_2 & \Delta V_2 \end{pmatrix}^{-1} \begin{pmatrix} E_1 \\ E_2 \end{pmatrix}$$
$$\begin{pmatrix} T \\ B \end{pmatrix} = \frac{1}{\Delta U_1 \Delta V_2 - \Delta U_2 \Delta V_1} \begin{pmatrix} \Delta V_2 & -\Delta V_1 \\ -\Delta U_2 & \Delta U_1 \end{pmatrix} \begin{pmatrix} E_1 \\ E_2 \end{pmatrix}$$

- The normal is given by the normal at the object  $N$
- This transformation is described by a matrix

$$TBN_{\text{object}} = \begin{pmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{pmatrix}$$

- Why this matrix does make sense ?

# Why The $TBN$ Matrix Makes Sense

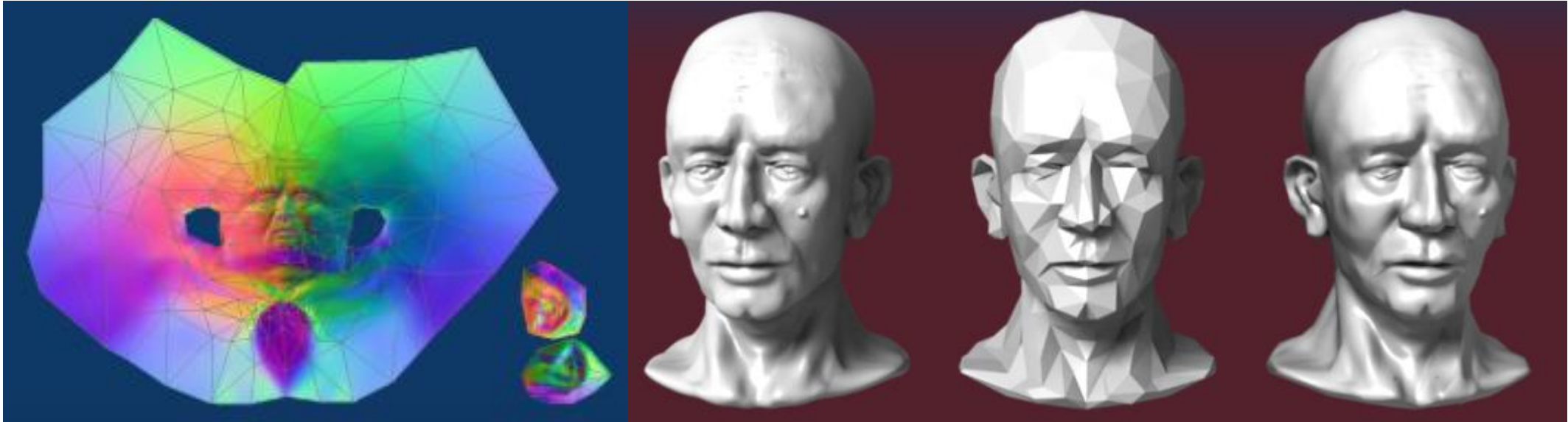
$$TBN_{\text{object}} = \begin{pmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{pmatrix}$$

- Why  $TBN_{\text{object}}$  transforms the texture space into object space?
  - By construction it takes normals in the normal map and maps them to the object
- but what does it do exactly ?
  - Note that  $(T, B, N)$  are orthogonal vectors, with length 1
  - So  $TBN_{\text{object}} TBN_{\text{object}}^T = I$  and  $\det(TBN_{\text{object}}) = 1$ 
    - Its a rotation matrix !
- This matrix just rotates the normals from object space to texture space !

# Normal Mapping

$$TBN_{\text{object}} = \begin{pmatrix} T_x & T_y & T_z \\ B_x & B_y & B_z \\ N_x & N_y & N_z \end{pmatrix}$$

- Reciprocally,  $TBN_{\text{object}}^{-1}$  transforms the object space into texture space !
- For free, we can take normals on a 3D model, and generate a normal map !



Object space normal map

High poly model

Simplified model

Rendered model

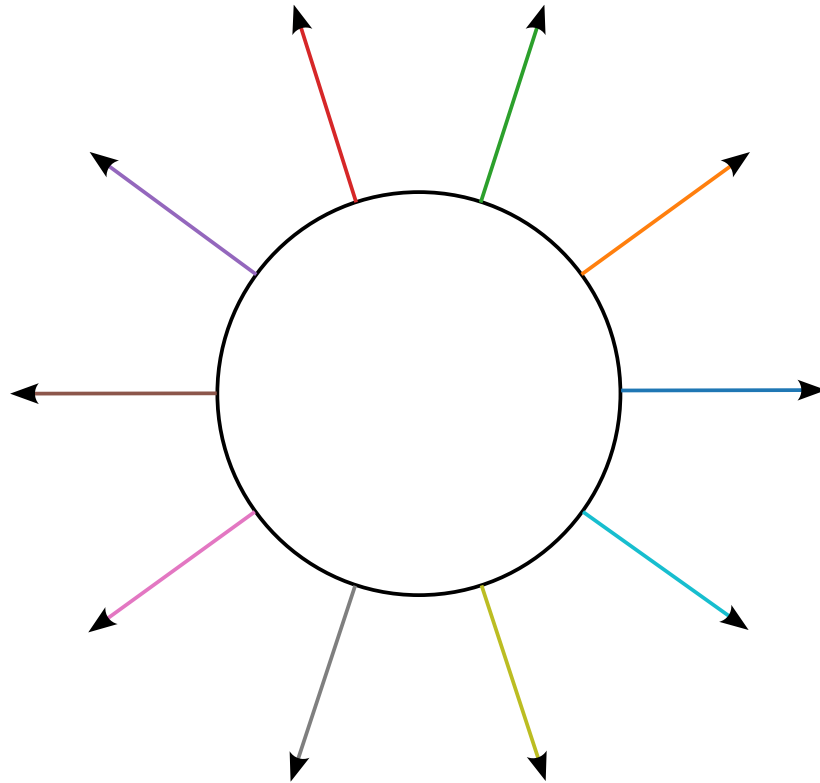
# Normal Mapping

- While the  $TBN_{object}$  allows us to transform normals from texture space to object space, we want to have normals in world space

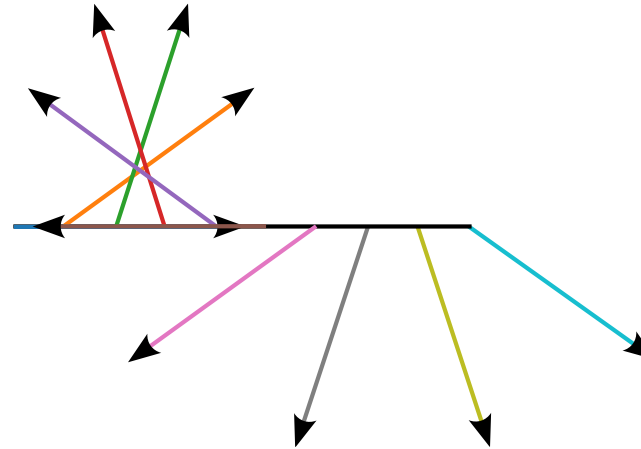
$$TBN_{world} = M_V^{-T} TBN_{object}$$

- Now we are able to compute lights !
- Take a normal map, use the  $TBN_{object}$  to map the normals to the vertices
- We have a light  $L$ , we can compute the color at each vertex by going from the normal on the object to the normal to the world using  $TBN_{world}$
- The light  $L$  and normal  $n$  are in the same space
- We can compute lights !

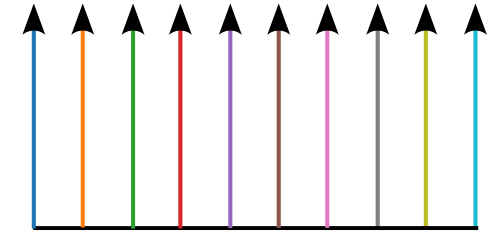
# 3D Normals to normal map



Object



Object space normal map



Tangent space normal map

- By applying  $TBN_{object}^{-1}$  to the normals of an object, we obtain the texture !

# Encoding of the Normal Map In Textures

- One last step.
- Normals are in  $[-1,1]^3$  with a norm of 1.
- But a texture can only encode positive values between 0 and 1.
- We transform our normals by mapping  $[-1,1]^3$  to  $[0,1]^3$

$$n_{rgb} = \frac{1}{2}(n + 1)$$

To retrieve it we can do the inverse operation

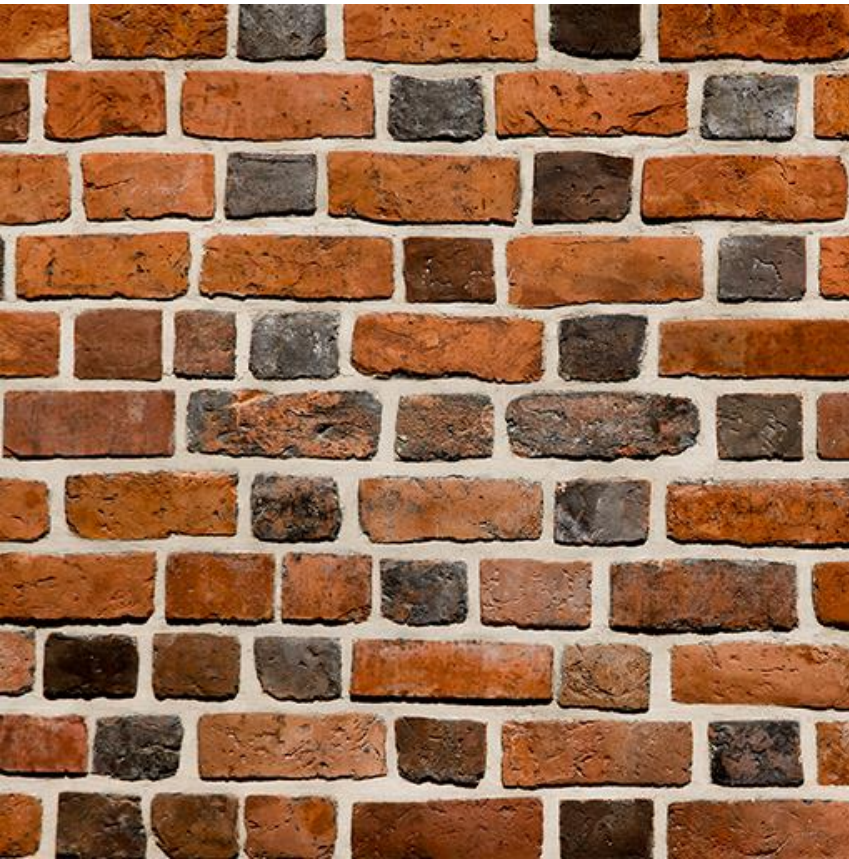
Check it works !

$$n = 2n_{rgb} - 1$$

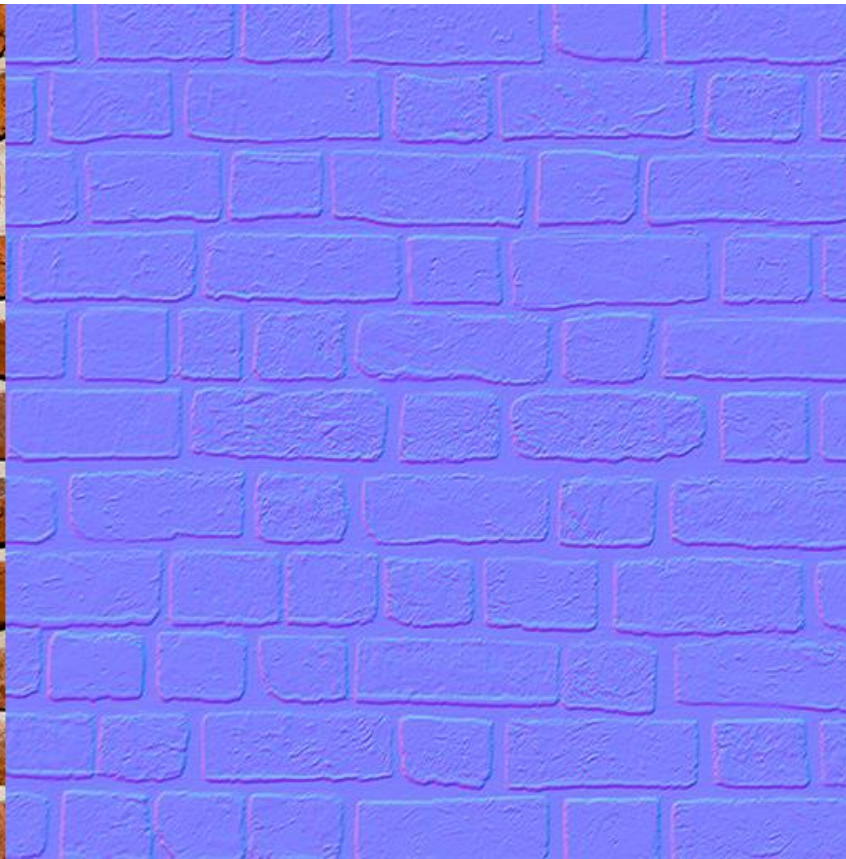
# Normal Mapping

Why those colors ?

- What does all of this look like applied to a sphere?



Texture

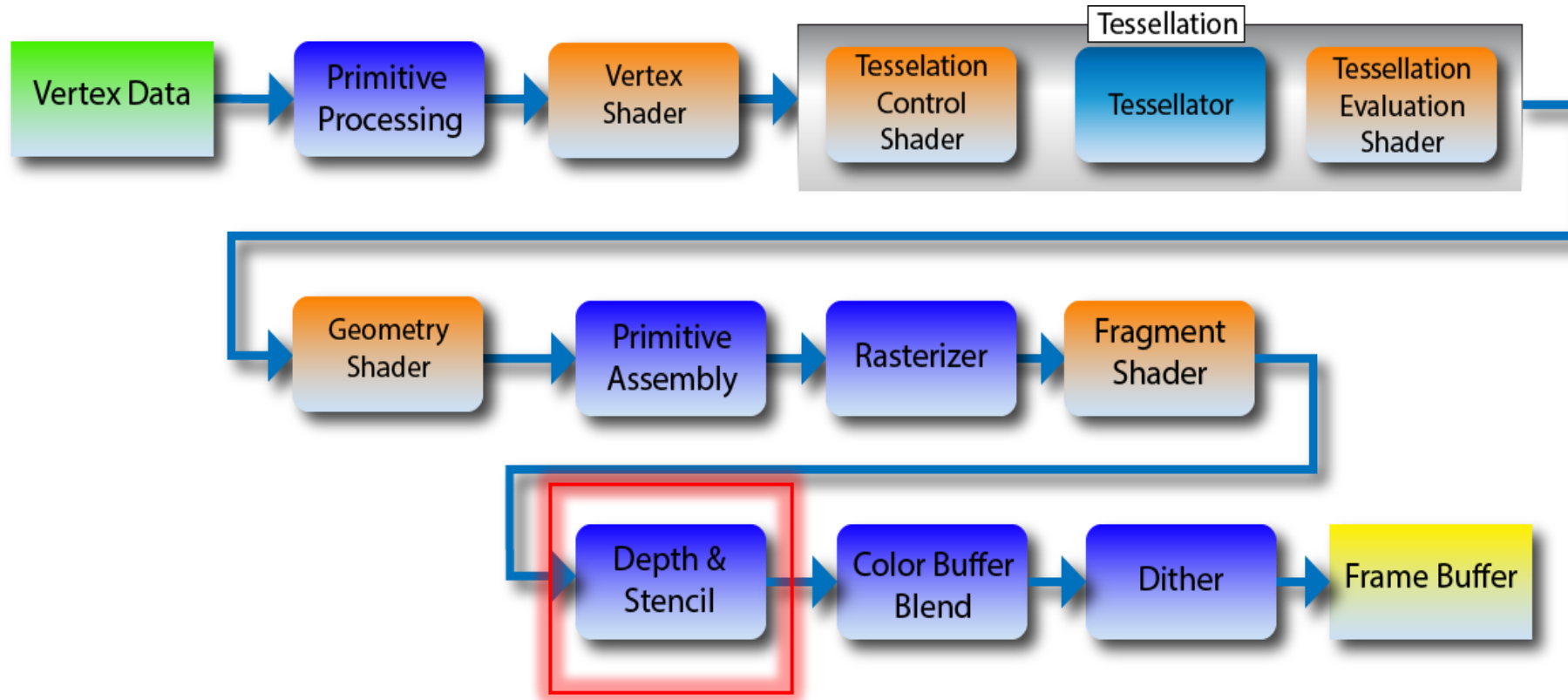


Tangent Space Normals



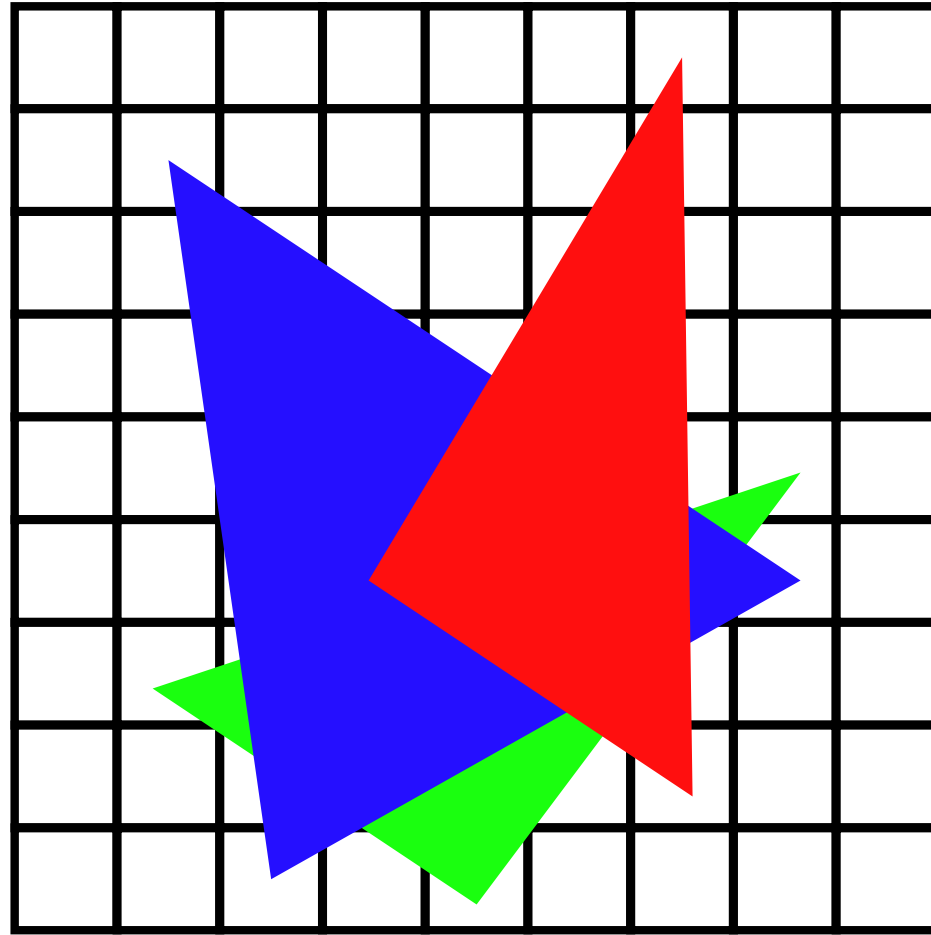
Object Space Normals

# Depth Testing



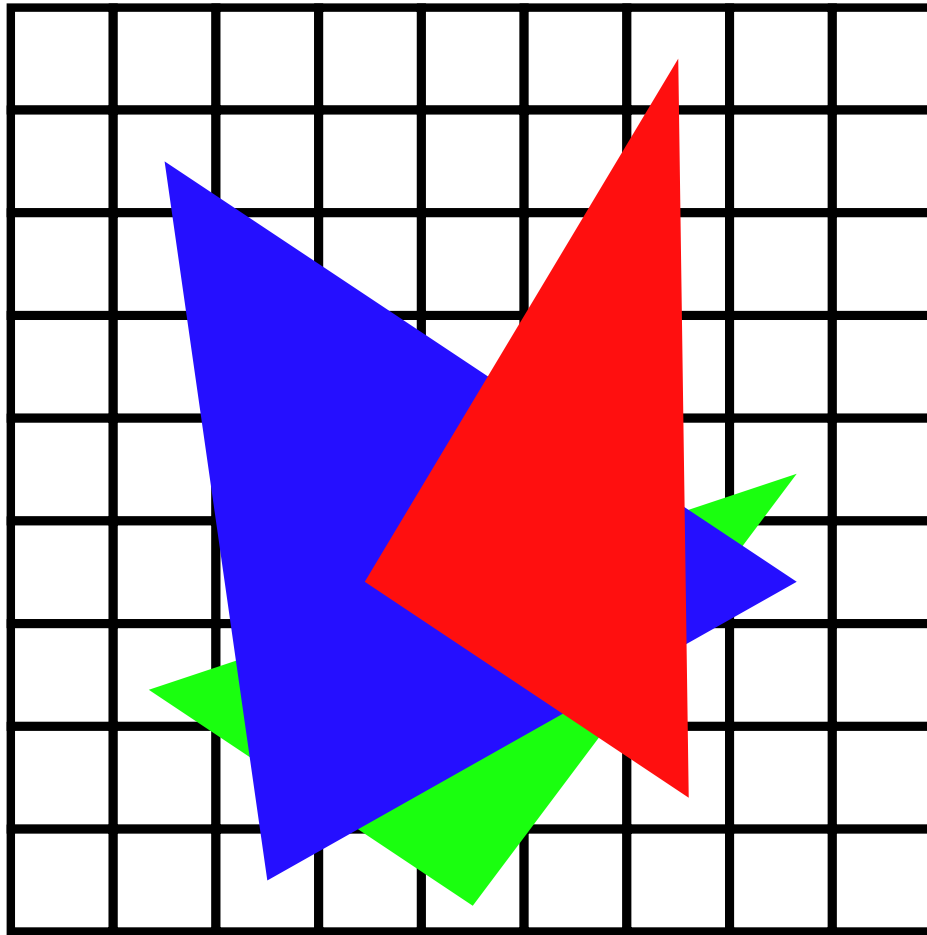
# Occlusions

- After computing the color of each fragment, some triangles are hidden behind others.

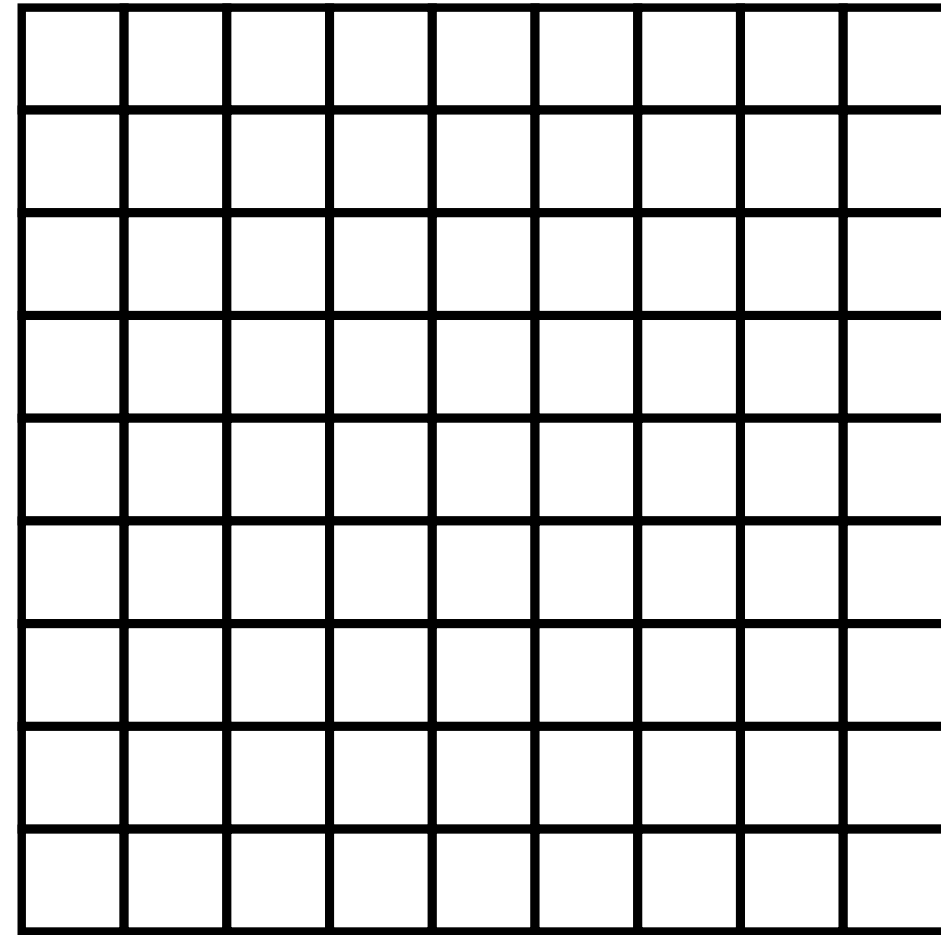


# Depth-buffer (Z-buffer)

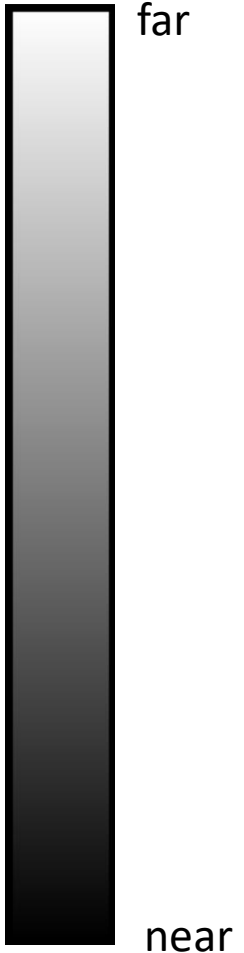
- For each sample, we store the depth of the closest triangle seen so far



Fragments



Z-Buffer

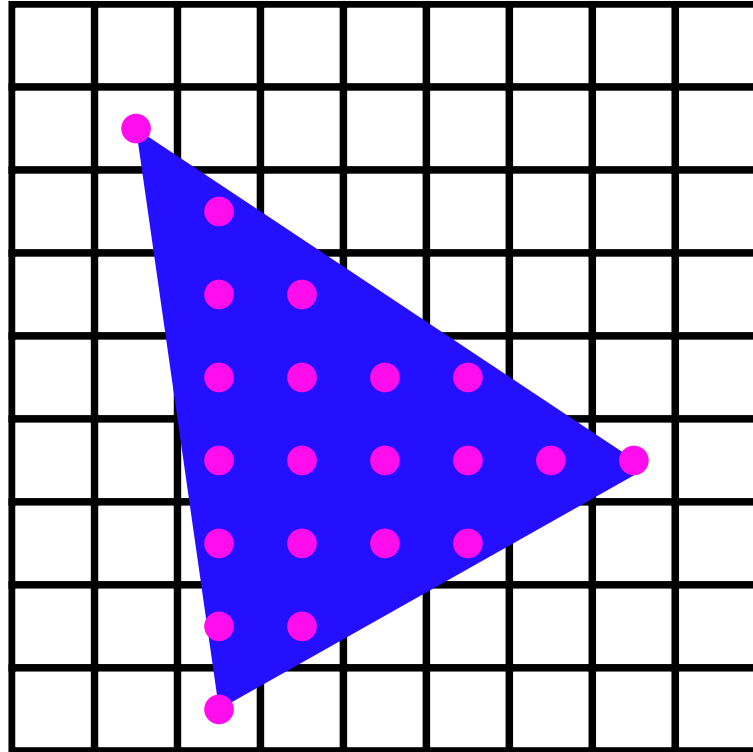
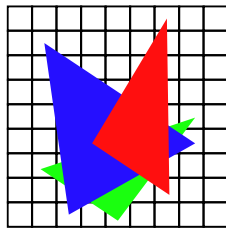


near

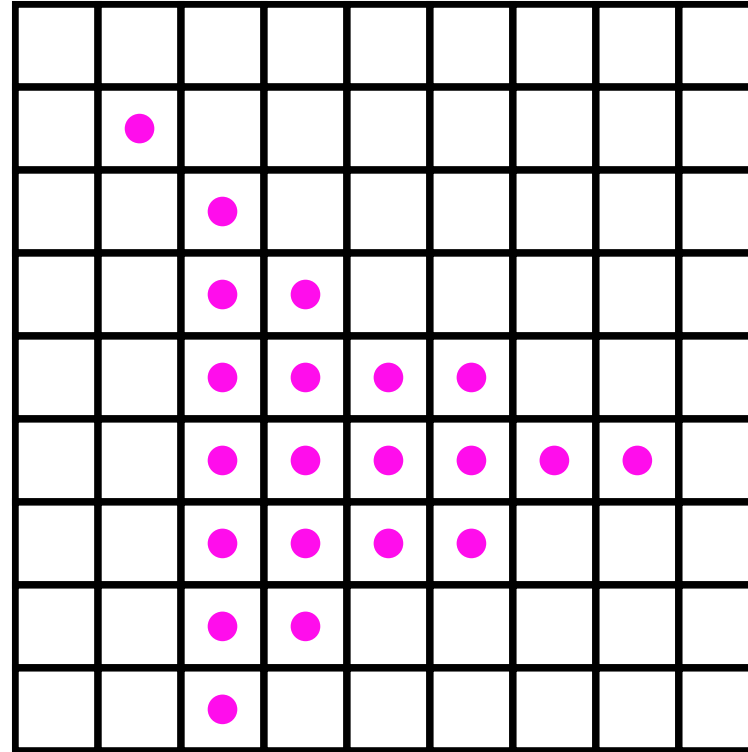
far

# Depth-buffer (Z-buffer)

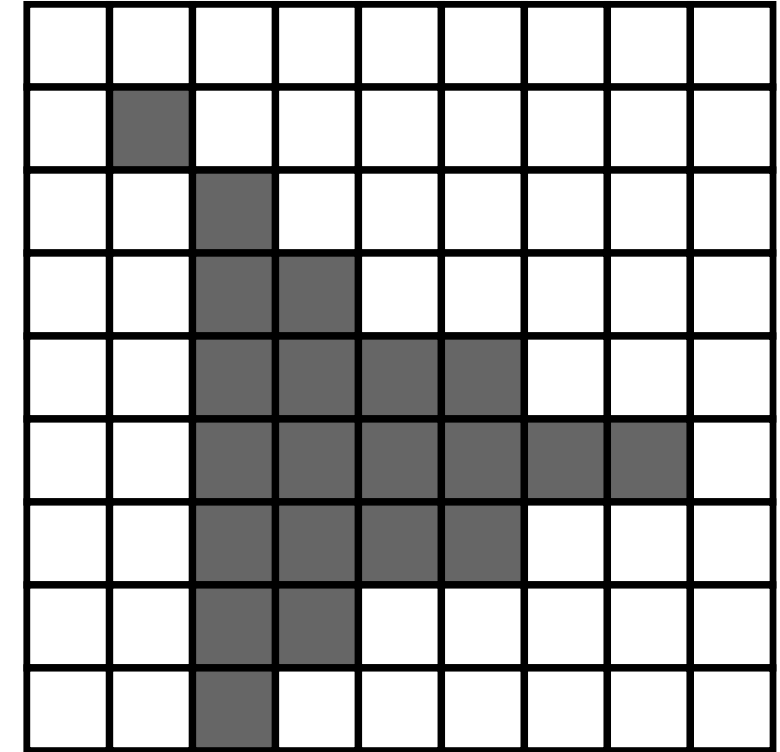
- For each sample, we store the depth of the closest triangle seen so far



Z-Buffer attribution



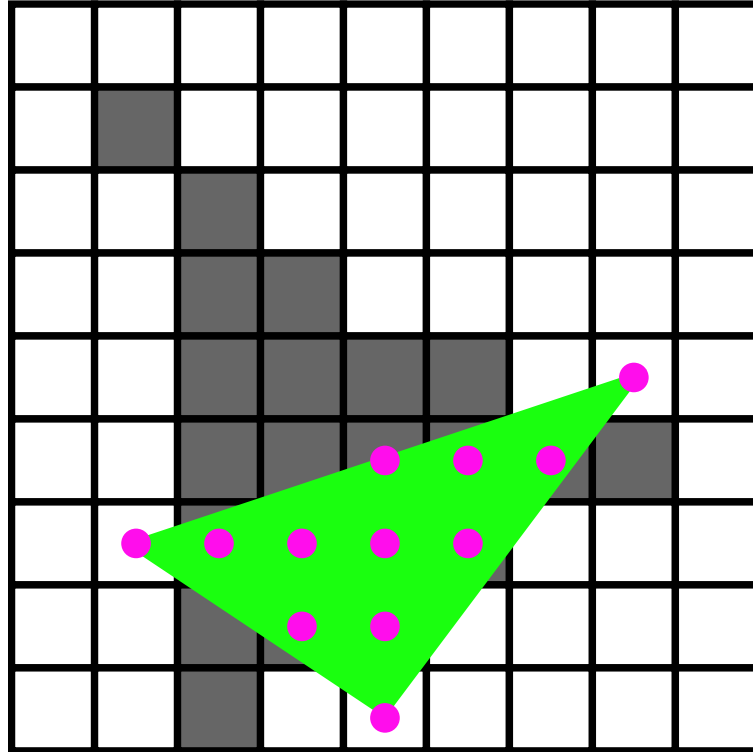
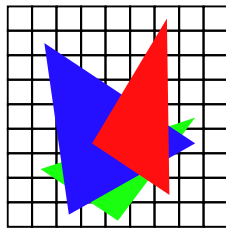
Z-Buffer test



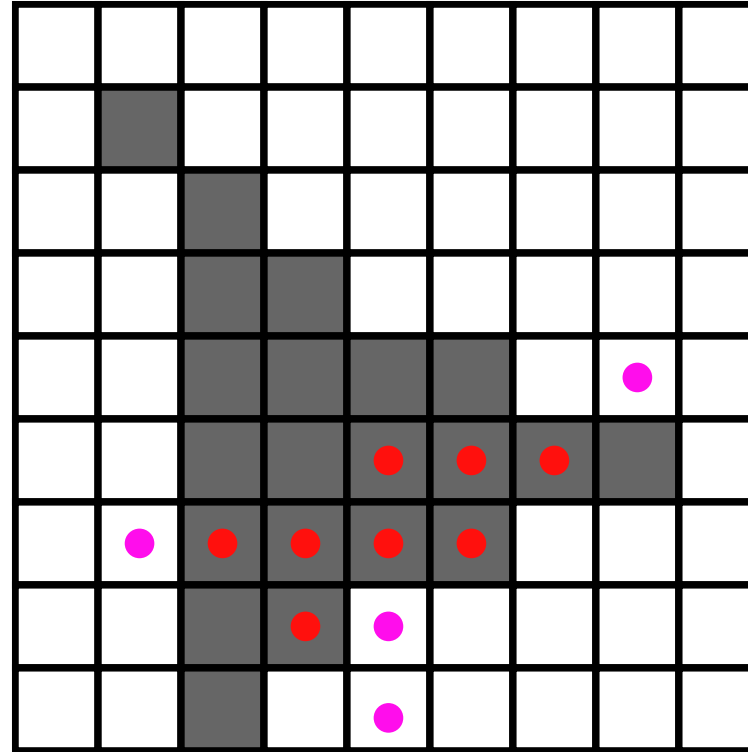
Z-Buffer

# Depth-buffer (Z-buffer)

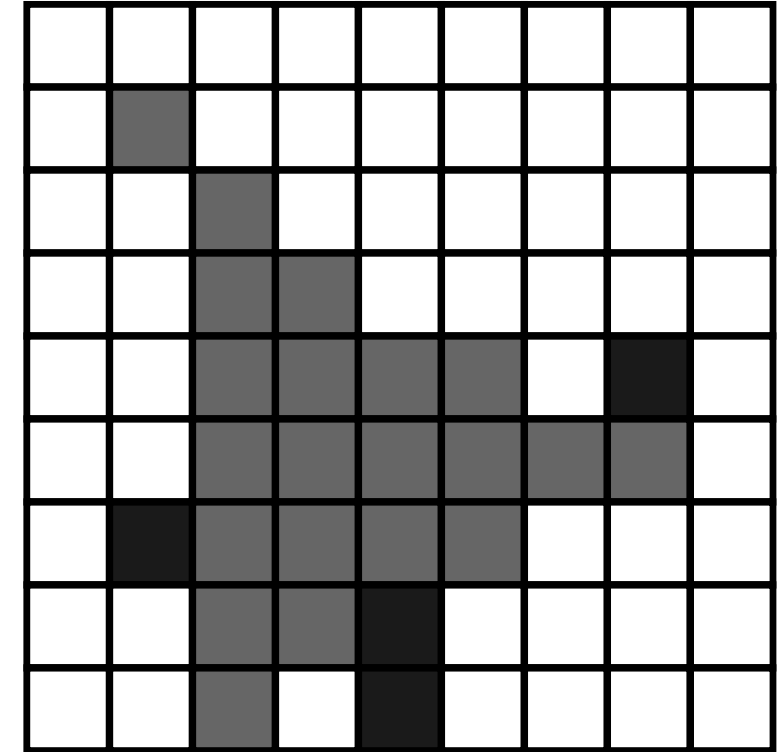
- For each sample, we store the depth of the closest triangle seen so far



Z-Buffer attribution



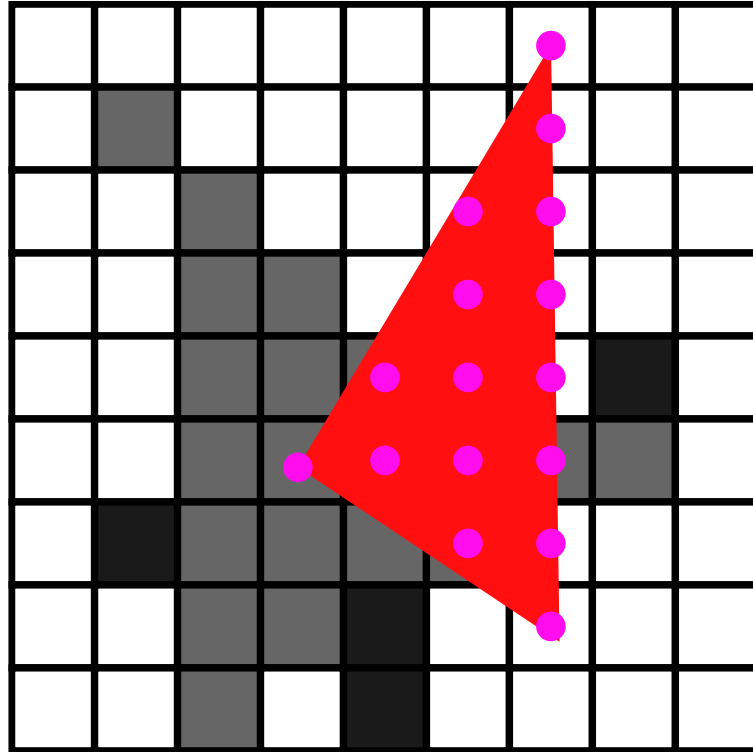
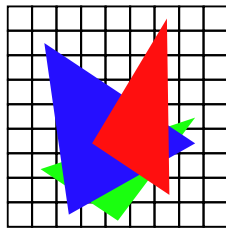
Z-Buffer test



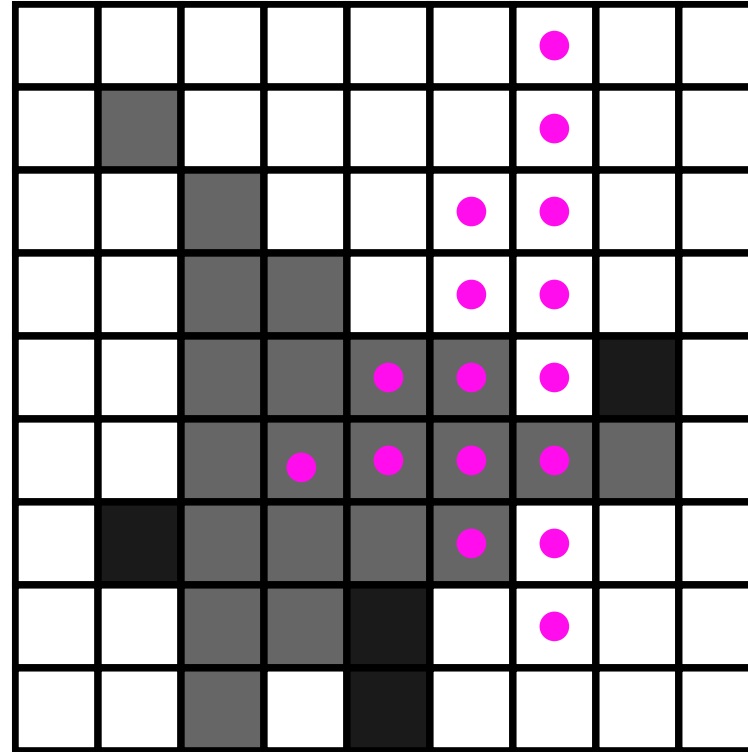
Z-Buffer

# Depth-buffer (Z-buffer)

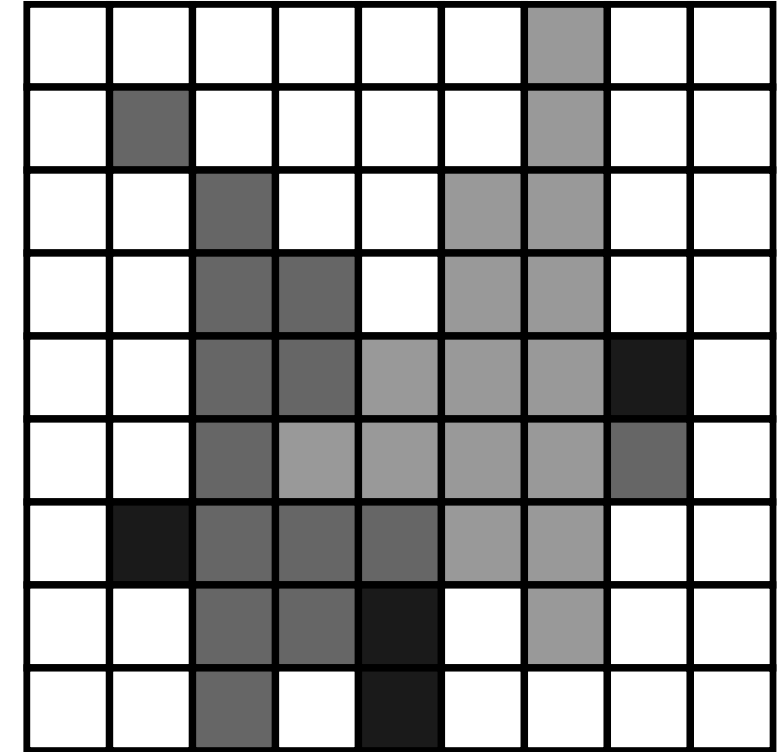
- For each sample, we store the depth of the closest triangle seen so far



Z-Buffer attribution



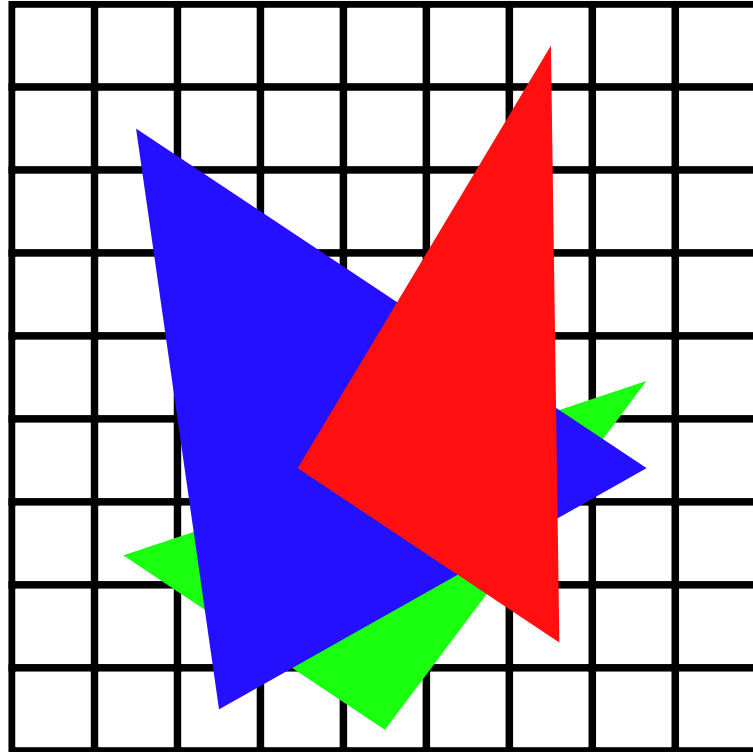
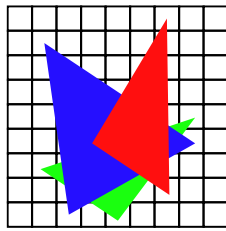
Z-Buffer test



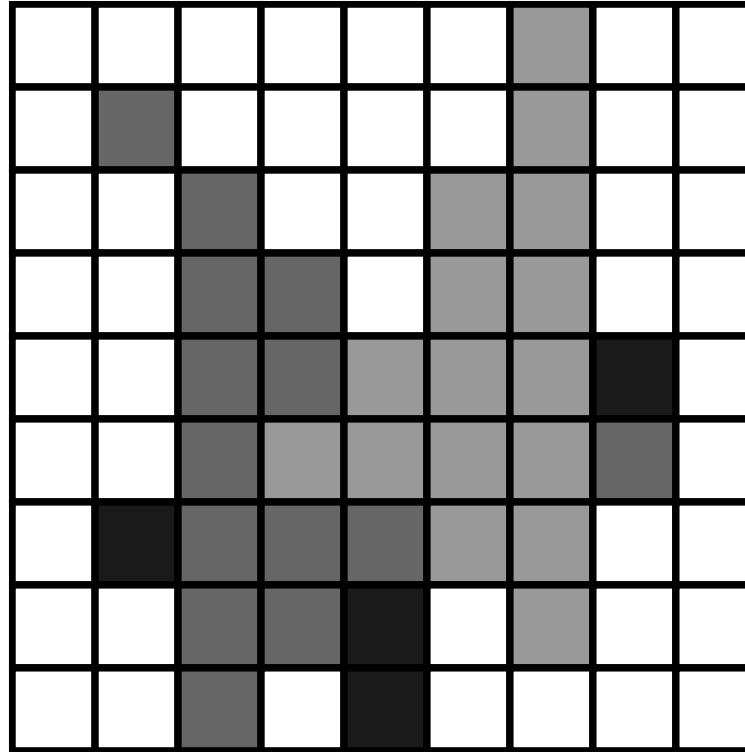
Z-Buffer

# Depth-buffer (Z-buffer)

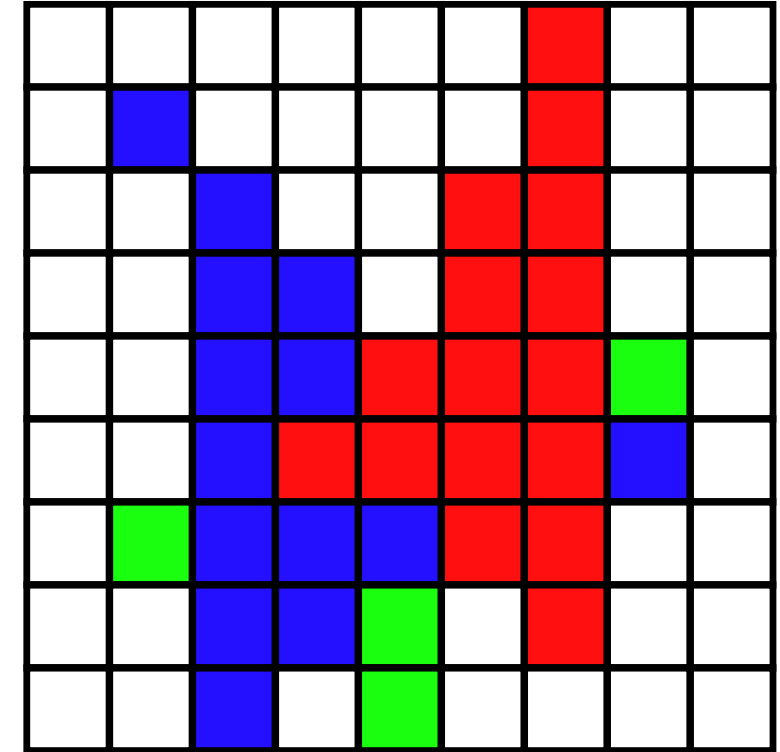
- For each sample, we store the depth of the closest triangle seen so far



Fragments



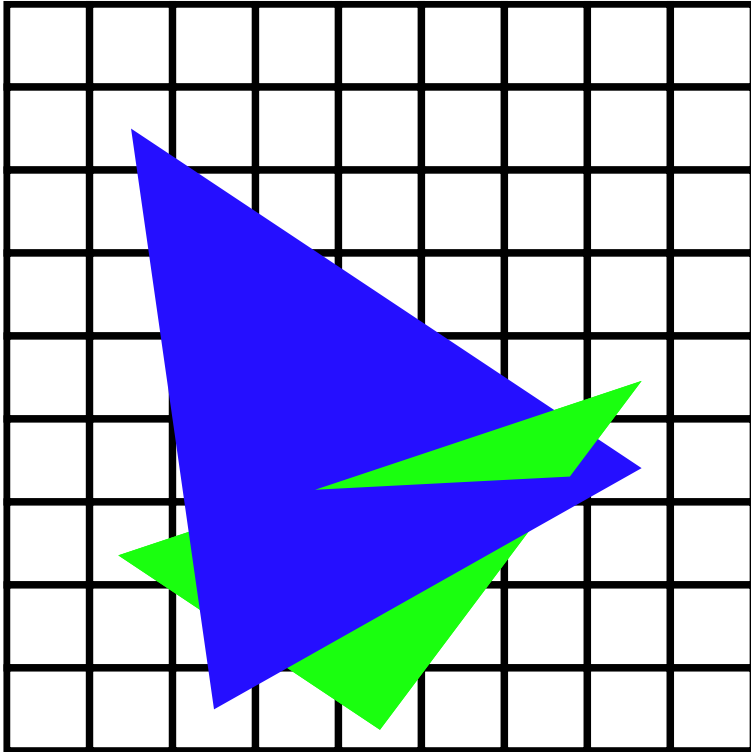
Z-Buffer



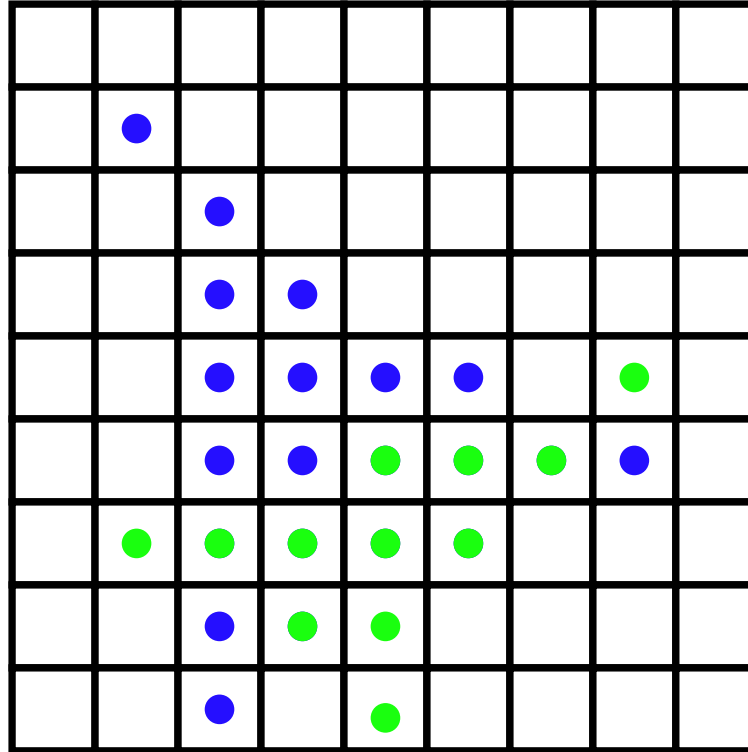
Screen

# Depth-buffer (Z-buffer) - Intersections

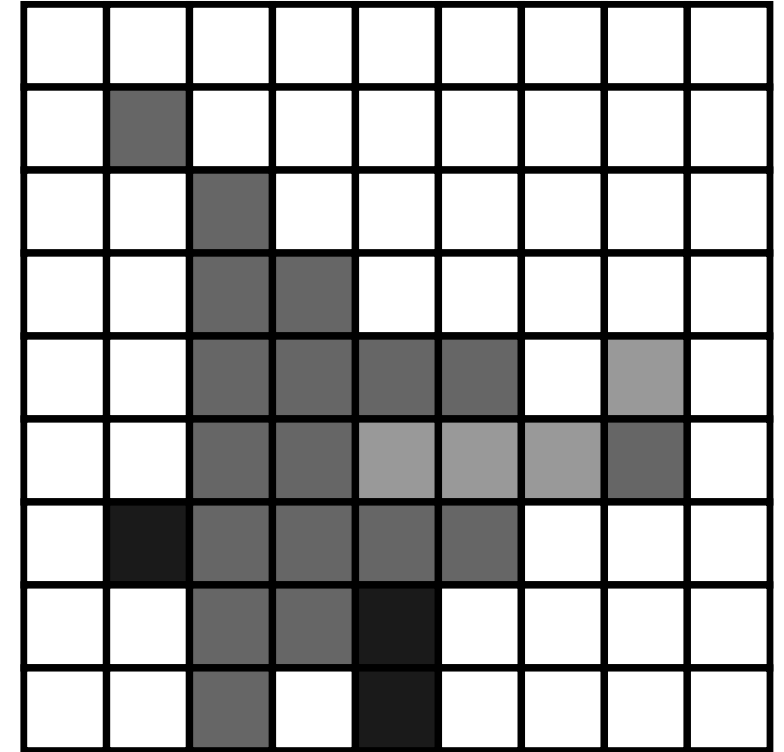
- Are the intersections handled properly ?



Fragments

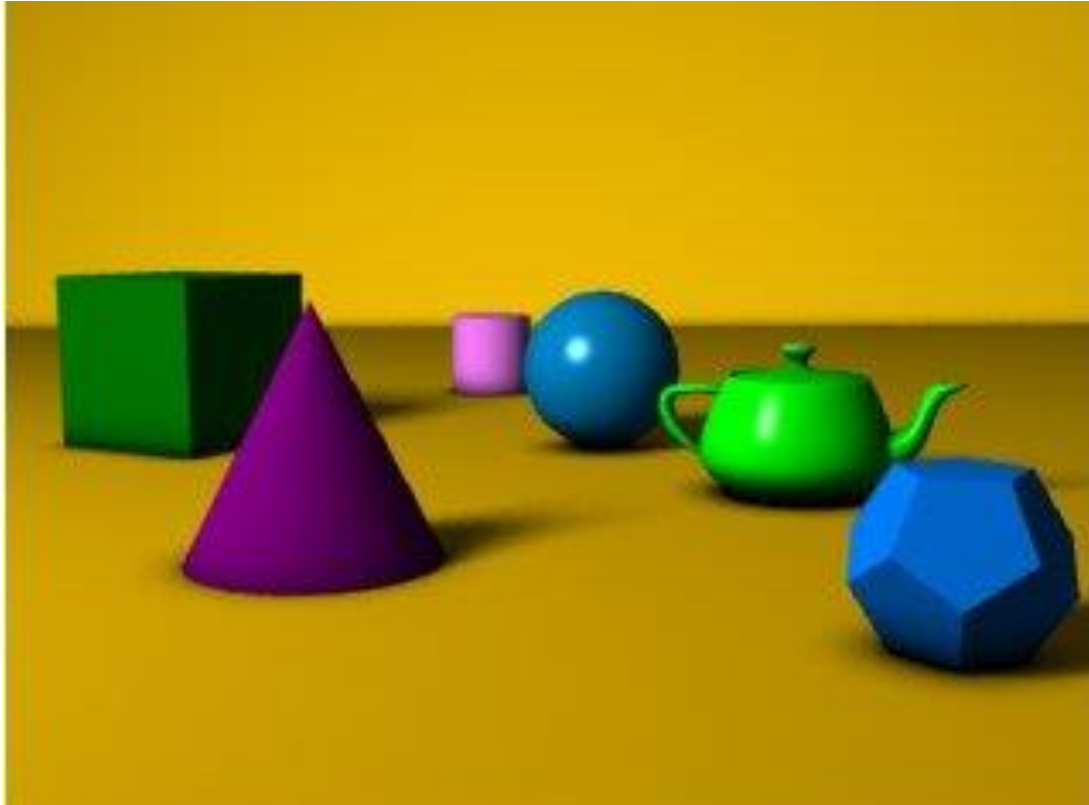


Z-Buffer tests



Z-Buffer

# Depth-buffer (Z-buffer) – Real example



Fragments



Z-Buffer

# What I Expect for the exam

- Understand how a texture is applied to a mesh
- Understand why there is aliasing
- Understand bilinear and trilinear interpolation
- Understand mipmapping
- Understand what is Normal Mapping and its tradeoffs (at least intuitively)
- Understand the colors in a normal map in object space and in tangent space
- Understand depth testing
  
- I DO NOT EXPECT
  - That you are able to create a texture from scratch
  - That you are able to rederive all Frenet equations

# Next time

- Encoding Geometry !



# Orthogonality of $TBN$

- We did not ensure that  $T$ ,  $B$  and  $N$  were orthogonal !
- They kinda are by construction, but in reality, there can be small shifts in the position.
- We can Gram-Schmidt them without affecting too much the normals
- In computer graphics, often we do not store the  $B$  component of our matrix, it can be recomputed on the fly by using
$$B = N \times T$$
- And normalizing.

# Frenet Formulas

- You already saw them in Mechanics and Geometry in bachelor !
- For a curve, they allow to construct an orthonormal axis system at each point! (very useful for bump mapping for example)
- We will define two quantities the curvature  $\kappa(t)$  and the torsion  $\tau(t)$ .
  - They respectively represent how a curve curves and twist at any point  $t$  !
- Creating a frenet frame on a point is creating a local axis system in which we can express our OpenGL tangents and normals to perform more complex algorithms !

# Frenet Formulas

- Suppose a parametric curve  $P(t)$ , twice differentiable
- The tangent at a point  $t$  is  $T(t) = \frac{d}{dt}P(t)$ , the unit tangent is  $\hat{T}(t) = \frac{T(t)}{\|T(t)\|}$
- A small (differential) length along the path is  $ds = \|T(t)\|dt$
- The curvature  $\kappa(t)$  is the magnitude of the rate at which the unit tangent changes with respect to the distance  $s$  traveled along the curve !
- $\kappa(t) = \left\| \frac{d}{ds} \hat{T} \right\| = \left\| \frac{d}{dt} \hat{T} \cdot \frac{dt}{ds} \right\| = \left\| \frac{d}{dt} \hat{T} \right\| \cdot \frac{dt}{ds} = \frac{\left\| \frac{d}{dt} \hat{T} \right\|}{\frac{ds}{dt}}$
- We have  $ds = \|T(t)\|dt$  or  $\frac{ds}{dt} = \|T(t)\|$
- Let's rewrite  $\frac{d}{dt}P(t) = T(t) = \hat{T}(t)\|T(t)\| = \hat{T}(t)\frac{ds}{dt}$

# Frenet Formulas – Normal at a point

- $P(t), T(t) = \frac{d}{dt}P(t) = \hat{T}(t) \frac{ds}{dt}, ds = ||T(t)||dt, \kappa(t) = \frac{\left\| \frac{d}{dt}\hat{T} \right\|}{\frac{ds}{dt}}$
- The curvature depends on the derivative of the tangent, which is the first derivative of the curve.
- We want to see how it relates to the second derivative.
- Note that  $\hat{T}(t) \cdot \hat{T}(t) = 1$ , then
  - $\frac{d}{dt}(\hat{T}(t) \cdot \hat{T}(t)) = \left(\frac{d}{dt}\hat{T}(t)\right) \cdot \hat{T}(t) + \hat{T}(t) \cdot \left(\frac{d}{dt}\hat{T}(t)\right) = 0 \Leftrightarrow 2\hat{T}(t) \cdot \frac{d}{dt}\hat{T}(t) = 0$
- We see that  $\frac{d}{dt}\hat{T}(t)$  is orthogonal to  $\hat{T}(t)$
- We can define  $\hat{N}(t) = \frac{d}{dt}\hat{T}(t)$  as the normal to the tangent.

# Frenet Formulas – Link between $\hat{P}(t)$ , $\hat{T}(t)$ , $\hat{N}(t)$

- $P(t), T(t) = \frac{d}{dt} P(t) = \hat{T}(t) \frac{ds}{dt}, ds = ||T(t)|| dt, \kappa(t) = \frac{||\frac{d}{dt} \hat{T}||}{\frac{ds}{dt}}, \hat{N}(t) = \frac{d}{dt} \hat{T}(t)$
- What is the link between  $\hat{P}(t)$ ,  $\hat{T}(t)$  and  $\hat{N}(t)$  ?  $\frac{d}{dx} \left( \frac{f(x)}{g(x)} \right) = \frac{f'(x)g(x) - f(x)g'(x)}{g^2(x)}$
- $\frac{d}{dt} \hat{T}(t) = \frac{d}{dt} \left( \frac{\frac{d}{dt} P(t)}{\frac{ds}{dt}} \right) = \frac{\frac{d^2}{dt^2} P(t) \cdot \frac{ds}{dt} - \frac{d}{dt} P(t) \cdot \frac{d^2 s}{dt^2}}{\left( \frac{ds}{dt} \right)^2} = \frac{\frac{d^2}{dt^2} P(t) \cdot \left( \frac{ds}{dt} \right) - \hat{T}(t) \left( \frac{ds}{dt} \right) \cdot \frac{d^2 s}{dt^2}}{\left( \frac{ds}{dt} \right)^2} =$   
 $\frac{\frac{d^2}{dt^2} P(t) - \hat{T}(t) \frac{d^2 s}{dt^2}}{\frac{ds}{dt}}$
- Solving for the second derivative of  $\hat{P}(t)$ :  $\frac{d^2}{dt^2} P(t) = \frac{d}{dt} \hat{T}(t) \frac{ds}{dt} + \hat{T}(t) \frac{d^2 s}{dt^2}$
- Since the derivative of  $\hat{T}(t)$  point in the direction of  $\hat{N}(t)$ , we write  $\frac{d}{dt} \hat{T}(t) =$   
 $\left\| \frac{d}{dt} \hat{T}(t) \right\| \hat{N}(t)$

# Frenet Formulas – Link between $\hat{P}(t)$ , $\hat{T}(t)$ , $\hat{N}(t)$

- $P(t), T(t) = \frac{d}{dt} P(t) = \hat{T}(t) \frac{ds}{dt}, ds = ||T(t)|| dt, \kappa(t) = \frac{||\frac{d}{dt} \hat{T}||}{\frac{ds}{dt}}, \hat{N}(t) = \frac{d}{dt} \hat{T}(t)$
- What is the link between  $\hat{T}(t)$  and  $\hat{N}(t)$  ?
- $\frac{d}{dt} \hat{T}(t) = \frac{\frac{d^2}{dt^2} P(t) - \hat{T}(t) \frac{d^2 s}{dt^2}}{\frac{ds}{dt}}$
- Solving for the second derivative of  $\hat{P}(t)$ :  $\frac{d^2}{dt^2} P(t) = \frac{d}{dt} \hat{T}(t) \frac{ds}{dt} + T(t) \frac{d^2 s}{dt^2}$
- Since the derivative of  $\hat{T}(t)$  point in the direction of  $\hat{N}(t)$ , we have:
  - $\frac{d}{dt} \hat{T}(t) = \left| \left| \frac{d}{dt} \hat{T}(t) \right| \right| \hat{N}(t)$
- Finally:  $\frac{d^2}{dt^2} P(t) = \left| \left| \frac{d}{dt} \hat{T}(t) \right| \right| \hat{N}(t) \frac{ds}{dt} + T(t) \frac{d^2 s}{dt^2} = \kappa(t) \frac{ds}{dt} \hat{N}(t) \frac{ds}{dt} + \hat{T}(t) \frac{d^2 s}{dt^2}$

# Frenet Formulas – A better formula for $\kappa(t)$

- $P(t), T(t) = \frac{d}{dt} P(t) = \hat{T}(t) \frac{ds}{dt}, ds = ||T(t)|| dt, \kappa(t) = \frac{||\frac{d}{dt} \hat{T}||}{\frac{ds}{dt}}, \hat{N}(t) = \frac{d}{dt} \hat{T}(t),$

$$\frac{d^2}{dt^2} P(t) = \kappa(t) \frac{ds}{dt} \hat{N}(t) \frac{ds}{dt} + \hat{T}(t) \frac{d^2 s}{dt^2}$$

- Can we find a better formula for  $\kappa(t)$ ?

- Let's take the cross product between  $\frac{d}{dt} \hat{P}(t)$  and  $\frac{d^2}{dt^2} \hat{P}(t)$

- $\frac{d}{dt} \hat{P}(t) \times \frac{d^2}{dt^2} \hat{P}(t) = \hat{T}(t) \frac{ds}{dt} \times \left( \kappa(t) \frac{ds}{dt} \hat{N}(t) \frac{ds}{dt} + \hat{T}(t) \frac{d^2 s}{dt^2} \right) = \kappa(t) \left( \frac{ds}{dt} \right)^3 \left( \hat{T}(t) \times \hat{N}(t) \right)$

- Both  $\hat{T}(t)$  and  $\hat{N}(t)$  are unit vector, so  $\hat{T}(t) \times \hat{N}(t) = 1$

- $\kappa(t) = \frac{\frac{d}{dt} \hat{P}(t) \times \frac{d^2}{dt^2} \hat{P}(t)}{\left( \frac{ds}{dt} \right)^3} = \frac{\frac{d}{dt} \hat{P}(t) \times \frac{d^2}{dt^2} \hat{P}(t)}{\left( \frac{\frac{d}{dt} P(t)}{\hat{T}(t)} \right)^3},$  finally,  $\kappa(t)$  is always positive

# Frenet Formulas – A better formula for $\kappa(t)$

- $P(t), T(t) = \frac{d}{dt}P(t) = \hat{T}(t) \frac{ds}{dt}, ds = ||T(t)||dt, \kappa(t) = \frac{||\frac{d}{dt}\hat{T}||}{\frac{ds}{dt}}, \hat{N}(t) = \frac{d}{dt}\hat{T}(t),$

$$\frac{d^2}{dt^2}P(t) = \kappa(t) \frac{ds}{dt} N(t) \frac{ds}{dt} + \hat{T}(t) \frac{d^2s}{dt^2}$$

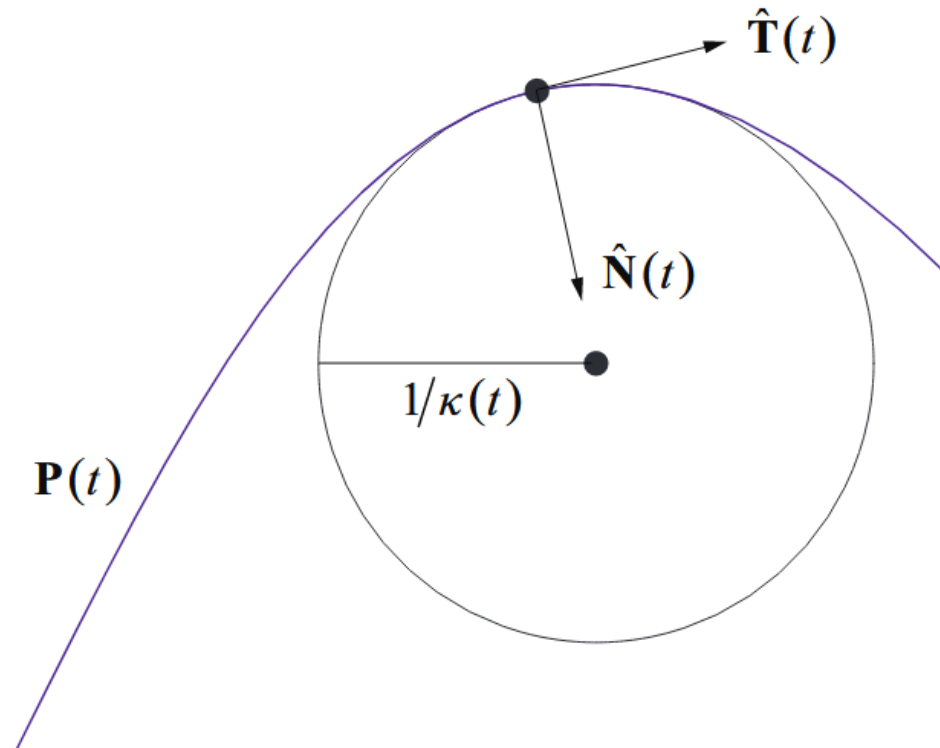
- $\kappa(t) = \frac{\frac{d}{dt}\hat{P}(t) \times \frac{d^2}{dt^2}\hat{P}(t)}{\left(\frac{ds}{dt}\right)^3} = \frac{\frac{d}{dt}\hat{P}(t) \times \frac{d^2}{dt^2}\hat{P}(t)}{\left(\frac{\frac{d}{dt}P(t)}{\hat{T}(t)}\right)^3}$

- Finally,  $\kappa(t)$  is always positive:

- $\kappa(t) = \left| \left| \frac{\frac{d}{dt}\hat{P}(t) \times \frac{d^2}{dt^2}\hat{P}(t)}{\left(\frac{\frac{d}{dt}P(t)}{\hat{T}(t)}\right)^3} \right| \right| = \left| \left| \frac{\frac{d}{dt}\hat{P}(t) \times \frac{d^2}{dt^2}\hat{P}(t)}{\left(\frac{d}{dt}P(t)\right)^3} \right| \right| = \frac{||P'(t) \times P''(t)||}{||P'(t)||^3}$

# Frenet Formulas – Binormal

- $P(t), T(t) = \frac{d}{dt}P(t) = \hat{T}(t) \frac{ds}{dt}, ds = ||T(t)||dt, \kappa(t) = \frac{||\frac{d}{dt}\hat{T}||}{\frac{ds}{dt}} = \frac{||P'(t) \times P''(t)||}{||P'(t)||^3},$   
 $\hat{N}(t) = \frac{d}{dt}\hat{T}(t), \frac{d^2}{dt^2}P(t) = \kappa(t) \frac{ds}{dt} N(t) \frac{ds}{dt} + \hat{T}(t) \frac{d^2s}{dt^2}$
- To have an orthonormal axis system at  $P(t)$ , we define the binormal:
- $\hat{B}(t) = \hat{T}(t) \times \hat{N}(t)$



What is the direction of  $\hat{B}(t)$  here?

# Appendix: Frenet formulas

- We can derive the Frenet axis along the infinitesimal path  $ds$

$$\frac{d}{ds}\hat{\mathbf{T}}(t) = \left\| \frac{d}{ds}\hat{\mathbf{T}}(t) \right\| \hat{\mathbf{N}}(t) = \kappa(t) \hat{\mathbf{N}}(t).$$

$$\frac{d}{ds}\hat{\mathbf{B}}(t) = \frac{d}{ds}\hat{\mathbf{T}}(t) \times \hat{\mathbf{N}}(t) + \hat{\mathbf{T}}(t) \times \frac{d}{ds}\hat{\mathbf{N}}(t).$$

$$\begin{aligned} \frac{d}{ds}\hat{\mathbf{B}}(t) &= \hat{\mathbf{T}}(t) \times [\alpha(t)\hat{\mathbf{T}}(t) + \tau(t)\hat{\mathbf{B}}(t)] \\ &= -\tau(t)\hat{\mathbf{N}}(t) \end{aligned}$$

$$\begin{aligned} \frac{d}{ds}\hat{\mathbf{N}}(t) &= \frac{d}{ds}[\hat{\mathbf{B}}(t) \times \hat{\mathbf{T}}(t)] \\ &= \frac{d}{ds}\hat{\mathbf{B}}(t) \times \hat{\mathbf{T}}(t) + \hat{\mathbf{B}}(t) \times \frac{d}{ds}\hat{\mathbf{T}}(t) \\ &= -\tau(t)\hat{\mathbf{N}}(t) \times \hat{\mathbf{T}}(t) + \hat{\mathbf{B}}(t) \times \kappa(t)\hat{\mathbf{N}}(t) \\ &= \tau(t)\hat{\mathbf{B}}(t) - \kappa(t)\hat{\mathbf{T}}(t). \end{aligned}$$

$$\begin{aligned} \frac{d}{ds}\hat{\mathbf{T}}(t) &= \kappa(t)\hat{\mathbf{N}}(t) \\ \frac{d}{ds}\hat{\mathbf{N}}(t) &= \tau(t)\hat{\mathbf{B}}(t) - \kappa(t)\hat{\mathbf{T}}(t) \\ \frac{d}{ds}\hat{\mathbf{B}}(t) &= -\tau(t)\hat{\mathbf{N}}(t) \end{aligned}$$